



UNIVERSIDAD
DE MÁLAGA



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA INFORMÁTICA
MASTER UNIVERSITARIO EN INGENIERÍA DEL SOFTWARE E INTELIGENCIA ARTIFICIAL

**Estudio comparativo de la calidad del código generado por LLMs comerciales y de
código abierto para el desarrollo de aplicaciones web full-stack.**

**A comparative study of code quality generated by commercial and open-source LLMs
for full-stack web application development.**

AUTOR

Javier Ramírez Rueda

TUTOR

Gabriel Jesús Luque Polo

Rubén Saborido Infantes

DEPARTAMENTO

Lenguajes y Ciencias de la Computación

UNIVERSIDAD DE MÁLAGA

MÁLAGA, JUNIO DE 2025

RESUMEN

Los recientes avances en Inteligencia Artificial han dado lugar a asistentes conversacionales enfocados al desarrollo web que pueden usar tanto LLMs comerciales (GPT-4o en GitHub Copilot) como alternativas de código abierto (Llama 3.1). Esto puede generar preguntas sobre su actual viabilidad y las diferencias en desempeño entre ambas categorías.

Dada su alta popularidad actual, se han seleccionado estos dos modelos y se han puesto en práctica desarrollando dos aplicaciones web full-stack de distinta complejidad, estudiando detalladamente el proceso y analizando posteriormente la calidad del código generado mediante SonarQube.

El procedimiento utilizado sigue el diseño de un experimento estructurado, elaborando plantillas con operaciones predefinidas que deberán completarse individualmente por cada modelo; cada operación corresponde a una instrucción, formulada siguiendo una estructura que combina el lenguaje natural con la inclusión de contexto necesario.

Los resultados obtenidos sugieren que ambas opciones pueden ser viables en el desarrollo web (aunque tienen particularidades) y presentan un desempeño razonablemente cercano, con cierta desventaja por parte de Llama 3.1. Para ambos modelos, la principal limitación observada se relaciona con la cantidad de entidades involucradas en una operación.

PALABRAS CLAVE

Desarrollo web, generación de código, Llama, GitHub Copilot, estudio comparativo

ABSTRACT

Recent advances in Artificial Intelligence have led to conversational assistants focused on web development that can use both commercial LLMs (GPT-4o in GitHub Copilot) and open-source alternatives (Llama 3.1). This may raise questions about their current viability and the performance gap between these two categories.

Given their current high popularity, these two models have been selected and put into practice by developing two full-stack web applications with different levels of complexity, thoroughly studying the process and later analyzing the quality of the generated code using SonarQube.

The procedure follows the design of a structured experiment, creating templates with predefined operations that must be completed separately for each model; each operation corresponds to an instruction, formulated using a structure that combines natural language with the inclusion of necessary context.

The findings suggest that both options can be suitable for web development (although they have their particularities) and exhibit reasonably similar performance, with a slight disadvantage for Llama 3.1. For both models, the main observed limitation is related to the number of entities involved in an operation.

KEYWORDS

Web development, code generation, Llama, GitHub Copilot, comparative study

ÍNDICE

RESUMEN.....	3
ABSTRACT.....	5
ÍNDICE.....	7
1. INTRODUCCIÓN.....	9
1.1. MOTIVACIÓN.....	9
1.2. OBJETIVOS.....	9
1.3. PREGUNTAS DE INVESTIGACIÓN.....	10
1.4. COBERTURA DEL ESTUDIO.....	11
1.5. ESTRUCTURA DEL DOCUMENTO.....	12
2. ESTADO DEL ARTE.....	13
2.1. REVISIÓN BIBLIOGRÁFICA.....	13
2.2. TECNOLOGÍAS.....	14
2.2.1. MODELOS DE LANGUAGE DE CÓDIGO ABIERTO.....	15
2.2.2. MODELOS DE LANGUAGE COMERCIALES.....	18
3. METODOLOGÍA.....	21
3.1. ESPECIFICACIÓN DE REQUISITOS.....	21
3.2. DISEÑO DE BASES DE DATOS.....	25
3.3. DISEÑO DE INSTRUCCIONES.....	26
3.3.1. PARTE DEL SERVIDOR.....	29
3.3.2. PARTE DEL CLIENTE.....	30
3.4. DESARROLLO Y REVISIÓN DE CÓDIGO.....	31
4. ANÁLISIS DE RESULTADOS.....	35
4.1. PRIMERA LECTURA DE RESULTADOS.....	35
4.2. RESULTADOS DE DESEMPEÑO.....	35
4.3. RESULTADOS DE CALIDAD.....	37
4.4. RESPUESTAS PLANTEADAS A LOS OBJETIVOS.....	37
5. CONCLUSIONES Y LÍNEAS FUTURAS.....	41
5.1. CONCLUSIONES.....	41
5.2. LÍNEAS FUTURAS.....	41
REFERENCIAS.....	43
A. INFORME DE INSTRUCCIONES.....	47

1

INTRODUCCIÓN

En este capítulo se presenta el contexto del estudio y el problema que busca abordar, proporcionando sus preguntas de investigación y una delineación inicial de su desarrollo.

1.1. MOTIVACIÓN

El reciente avance de la Inteligencia Artificial ha generado una transformación profunda en numerosos ámbitos, y uno de los más impactados ha sido indudablemente el de la programación. Esto ha permitido la incorporación de asistentes conversacionales basados en LLMs como una herramienta adicional para el desarrollo de código, cubriendo varios dominios dentro de la Ingeniería del Software como el del desarrollo web [1].

A pesar de estos avances, es evidente que la tecnología aún enfrenta ciertas limitaciones, lo que genera una interrogante constante sobre su alcance en la actualidad. En nuestro caso, surge la duda de cuán complejas pueden ser las aplicaciones web sin que esto represente una barrera para el desempeño de estos modelos al momento de entenderlas y colaborar en su desarrollo.

Por otra parte, el ecosistema de modelos de lenguaje para este tipo de herramientas presenta una dualidad clara entre las soluciones comerciales (como GPT-4o, usado en GitHub Copilot) y las alternativas de código abierto (como Llama 3.1), lo que lleva a preguntarse si estos últimos son capaces de competir con los primeros.

1.2. OBJETIVOS

En el presente Trabajo Fin de Máster (TFM), se propone realizar un estudio comparativo entre herramientas basadas en modelos de lenguaje comerciales y de código abierto, mediante el desarrollo de dos aplicaciones web full-stack y la evaluación del código generado, con el propósito de estudiar su alcance y obtener una perspectiva clara sobre sus principales diferencias.

Se buscará seleccionar un modelo entre los más relevantes en cada categoría, con el fin de realizar una comparación lo más justa y representativa posible. Y cada aplicación web

tendrá diferentes niveles de complejidad; buscando que la primera aborde una tarea simple mientras que la segunda aborde una tarea con mayor complejidad.

Para lograr alcanzar el objetivo general del estudio, se establecieron los siguientes subobjetivos específicos a tener en cuenta.

- Investigar los asistentes conversacionales basados en LLMs más relevantes y usados en la actualidad, tanto comerciales como de código abierto, y justificar la selección de los mismos con base a su actualidad, desempeño, popularidad y accesibilidad al público general.
- Diseñar y preparar el entorno necesario para que la experimentación sea estructurada y medible, incluyendo el seleccionar las métricas que se recopilarán para la posterior evaluación y el desarrollar unas plantillas de desarrollo sobre las que se definirán y ajustarán las complejidades de las dos aplicaciones web.
- Desarrollar ambas aplicaciones web a partir de las soluciones generadas por los modelos de lenguaje, proporcionándoles diferentes instrucciones en lenguaje natural contextualizadas con el código necesario en cada parte del desarrollo y documentando con exactitud todos los pasos realizados.
- Analizar los datos recopilados, complementando con los resultados de alguna herramienta de análisis, para todas las métricas seleccionadas. Estos resultados serán comparados entre sí y se elaborarán conclusiones en función de las preguntas de investigación que se planteen sobre el estudio.

1.3. PREGUNTAS DE INVESTIGACIÓN

Con el fin de proveer respuestas directas y pertinentes al objetivo general de este estudio comparativo, se han planteado las siguientes preguntas de investigación.

RQ1. ¿Es posible crear una determinada aplicación web completa con alguno de los modelos de lenguaje seleccionados mediante instrucciones en lenguaje natural?

RQ1.1. ¿Cuáles han sido las principales dificultades y desafíos observados en el proceso para cada modelo de lenguaje?

RQ2. ¿Qué tan lejos puede estar actualmente el desempeño de un modelo de lenguaje de código abierto en comparación con uno comercial en el desarrollo de aplicaciones web?

RQ3. ¿Cómo influye la complejidad de la aplicación web objetivo para cada modelo de lenguaje seleccionado en la frecuencia de errores de la generación de código?

1.4. COBERTURA DEL ESTUDIO

El estudio busca explorar las diferencias de desempeño y calidad entre un modelo comercial y otro de código abierto, tanto a nivel comparativo como individual. Aunque los hallazgos puedan ofrecer información interesante sobre estas diferencias y su potencial para facilitar el desarrollo web, también es importante reconocer sus límites.

- El estudio se centra exclusivamente en un modelo específico dentro de las dos categorías planteadas, y aunque se intente que estos sean representativos, puede limitar la generalización de los resultados a otros modelos de lenguaje; cualquier variación en aspectos como la arquitectura o los datos de entrenamiento podría influir en sus resultados realizando tareas similares.
- Aunque las tecnologías empleadas en el desarrollo de aplicaciones web son de uso frecuente en la actualidad, los resultados obtenidos tampoco son necesariamente aplicables en su totalidad a otras herramientas o lenguajes de programación.
- Dado que, en este estudio, las aplicaciones web estarán pensadas para servir como prototipos dentro de un experimento y no como aplicaciones reales; elementos secundarios, como la seguridad y la usabilidad, no serán considerados prioritarios.
- Por motivos similares, la estilización de interfaces de usuario tampoco será prioritaria y se realizará manualmente de forma simple y específica; y aunque actualmente diversos modelos puedan ser capaces de ofrecer dicha asistencia, este estudio se enfocará en el desarrollo funcional de las aplicaciones web, por lo que no se explorarán ni evaluarán ninguno de los elementos de diseño mencionados.

También es importante tener en cuenta la naturaleza no determinista del estudio, ya que la variabilidad de las repuestas generadas por los modelos dificulta en gran medida su replicación exacta en estudios posteriores. Del mismo modo, diferentes enfoques en el uso de los mismos pueden conducir a resultados diferentes.

1.5. ESTRUCTURA DEL DOCUMENTO

La estructura de esta memoria se distribuye en cinco capítulos que detallan toda la información relevante sobre su planificación, desarrollo y resultados. Posteriormente, se incluye un apéndice con un informe sobre las instrucciones formuladas para cada modelo de lenguaje y operación.

- En el primer capítulo se expone el marco general del estudio; definiendo el contexto del problema a abordar, así como las preguntas de investigación que lo guían.
- En el segundo capítulo se presentan varios conceptos técnicos fundamentales, justificando la selección de las tecnologías y modelos de lenguaje utilizados.
- En el tercer capítulo se documentan los aspectos más relevantes tanto de la planificación como del desarrollo del experimento, destacando la elaboración de la plantillas y la formulación de las instrucciones.
- En el cuarto capítulo se exponen e interpretan detalladamente los resultados obtenidos, considerando tanto el desempeño como la calidad de los modelos.
- En el quinto capítulo se establecen las conclusiones finales, además de proponer varias líneas futuras que podrían contribuir a una profundización del tema.

En este capítulo se introducen algunos de los conceptos técnicos más fundamentales relacionados con el estudio, así como las tecnologías y modelos de lenguaje empleados junto a otros estudios similares.

2.1. REVISIÓN BIBLIOGRÁFICA

Para la recolección de literatura relevante en este estudio, se utilizó principalmente la herramienta Google Scholar como motor de búsqueda académica; se emplearon combinaciones de nombres relacionados con LLMs conocidos, como GitHub Copilot, ChatGPT, Llama o Qwen, junto con palabras clave relacionadas con el desarrollo web y la calidad del código (por ejemplo, "github copilot sonarqube" o "llama full-stack").

Asimismo, también se consultó la bibliografía referenciada en varios de los artículos encontrados con el fin de ampliar la búsqueda. Por lo general, se ha observado una menor disponibilidad de estudios centrados en modelos de lenguaje de código abierto en comparación con los comerciales en el ámbito del desarrollo web, lo cual se atribuye en gran medida a la destacada presencia de los modelos publicados por OpenAI.

A continuación, se señalan algunos artículos que destacan por su relevancia y por compartir un enfoque similar al de esta investigación.

1. El estudio "Assessing the Performance of AI-Generated Code: A Case Study on GitHub Copilot" (2024) [2], mediante técnicas de análisis sobre tres benchmarks, concluye que aunque el código generado por GitHub Copilot (GPT-3) suele ser funcional, su rendimiento suele ser inferior al escrito por humanos; además, destaca que un diseño cuidadoso de las instrucciones puede mejorar su desempeño.
2. El estudio "Evaluating the Capabilities of GPT-4 in Full-Stack Web Development" (2024) [3] examina el uso de GitHub Copilot (GPT-4) mediante el desarrollo de tres aplicaciones web con ayuda parcial de instrucciones; los resultados indican que puede generar código funcional y relativamente cualitativo, pero conforme sube la complejidad del proyecto, se necesitan más instrucciones para depurar errores.

3. El estudio "Evaluating the Effectiveness of LLMs in Fixing Maintainability Issues in Real-World Projects" (2025) [4] analiza la capacidad de GitHub Copilot (GPT-4) y Llama 3.1 para abordar problemas de mantenibilidad, evaluando repositorios de GitHub mediante técnicas de *zero-shot* y *few-shot*; aunque los resultados de ambos modelos se muestran prometedores, aún enfrentan limitaciones importantes.
4. El estudio "The Impact of AI-generated Code on Web Development: A Comparative Study of ChatGPT and GitHub Copilot" (2023) [5], utilizando ChatGPT (GPT-3) y GitHub Copilot (GPT-3 reentrenado como un modelo de código, Codex), estudia la creación de una aplicación web tratando de replicar una plantilla completa; aunque ambos logran resultados adecuados, se concluye que ChatGPT es superior.

De estos cuatro estudios, aquellos que presentan una mayor similitud con la metodología adoptada (particularmente en lo que respecta a la aplicación práctica de modelos en el desarrollo de aplicaciones web) son el segundo y el cuarto.

Más allá de la comparativa entre un modelo comercial y otro de código abierto, y del enfoque actualizado que se busca aportar mediante el uso de tecnologías y modelos actuales en el desarrollo funcional; este estudio se distingue por su enfoque metodológico estructurado y medible, definiendo exactamente cada operación a realizar por los modelos.

2.2. TECNOLOGÍAS

En la actualidad, el desarrollo de aplicaciones web cuenta con un extenso abanico de tecnologías ampliamente utilizadas, tanto del lado del *back-end* como del *front-end* (Vue, NextJS, Flask, Angular...). Tras analizar distintas opciones, se ha decidido emplear las siguientes tecnologías.

- En el lado del *back-end* o servidor se empleará Django [6], un *framework* de código abierto sobre el que funcionarán los servidores de las aplicaciones, proporcionando una API. Entre sus principales atractivos, puede destacarse su base en Python (ampliamente utilizado por su legibilidad y potencia) y su sencilla integración con la base de datos PostgreSQL [7].
- En el lado del *front-end* o cliente se empleará React (Vite) [8], una biblioteca de código abierto sobre la que funcionarán los clientes de las aplicaciones, permite la construcción de interfaces de usuario dinámicas a partir de componentes; esta

biblioteca ha sido utilizada a partir de Vite, una herramienta que permite la compilación y construcción de proyectos tanto en JavaScript como en TypeScript.

Estas tecnologías fueron seleccionadas por su uso frecuente dentro del desarrollo web *full-stack* para asegurar la relevancia de los resultados del estudio. Al emplear lenguajes de programación y *frameworks* populares, se busca que los resultados sean mayormente aplicables a una amplia variedad de entornos de desarrollo.

Además de las herramientas previamente mencionadas, también se han empleado las siguientes durante distintas áreas de todo el estudio comparativo.

- **Visual Studio Code [9]**. Es un editor de código fuente flexible que funciona de forma independiente para cualquier tipo de proyecto. Se trata de la herramienta principal para el desarrollo de ambas partes de cada aplicación, además ser la interfaz para ejecutar el modelo de lenguaje comercial seleccionado.
- **Jupyter Notebook [10]**. Es una aplicación web de código abierto que funciona como editor de código fuente y tiene la especialidad de hacerlo con documentos subdivididos en celdas ejecutables, es la principal interfaz utilizada para programar la configuración y ejecución del modelo de código abierto seleccionado en Python.
- **PyTorch [11]**. Junto a Transformers, una biblioteca desarrollada por HuggingFace [12] basada en PyTorch, ambas constituyen las principales herramientas basadas en Python utilizadas para configurar y ejecutar localmente el modelo de lenguaje de código abierto seleccionado.
- **SonarQube [13]**. Es una plataforma local de código abierto que permite evaluar cualquier código fuente, soportando una amplia variedad de lenguajes. Sirve como la herramienta principal para evaluar la calidad del código generado por los modelos de lenguaje.

2.2.1. MODELOS DE LENGUAJE DE CÓDIGO ABIERTO

En términos generales, un modelo de lenguaje se considera de código abierto cuando sus pesos (los parámetros principales de la red neuronal) están disponibles públicamente, aunque otros aspectos como su código fuente no lo estén necesariamente; esto permite su uso local con mayor flexibilidad, así como su reentrenamiento (*fine-tuning*).

Si adoptamos una definición más estricta, podremos distinguir dos tipos (siendo una mayoría de ellos *open-weights*). Cabe destacar que para cada modelo de lenguaje, independientemente de su tipo, debe tenerse en cuenta también las posibles restricciones que impongan sus licencias de uso.

- Los modelos *open-weights* son aquellos que comparten sus pesos, permitiendo descargarlos, configurarlos, ejecutarlos y reentrenarlos en la máquina del usuario.
- Los modelos *open-source* son aquellos que además comparten su código fuente completo (su arquitectura, datos de entrenamiento...) y bajo una licencia permisiva.

El mercado de los modelos de lenguaje de código abierto es bastante amplio y competitivo, y aunque los modelos comerciales desarrollados por OpenAI continúan siendo los dominantes en el sector, las alternativas de código abierto impulsadas por organizaciones como Meta o DeepSeek están reduciendo progresivamente esta brecha.

La principal plataforma de referencia es HuggingFace, un sitio web que aloja modelos de lenguaje de manera análoga a cómo GitHub lo hace con código fuente.

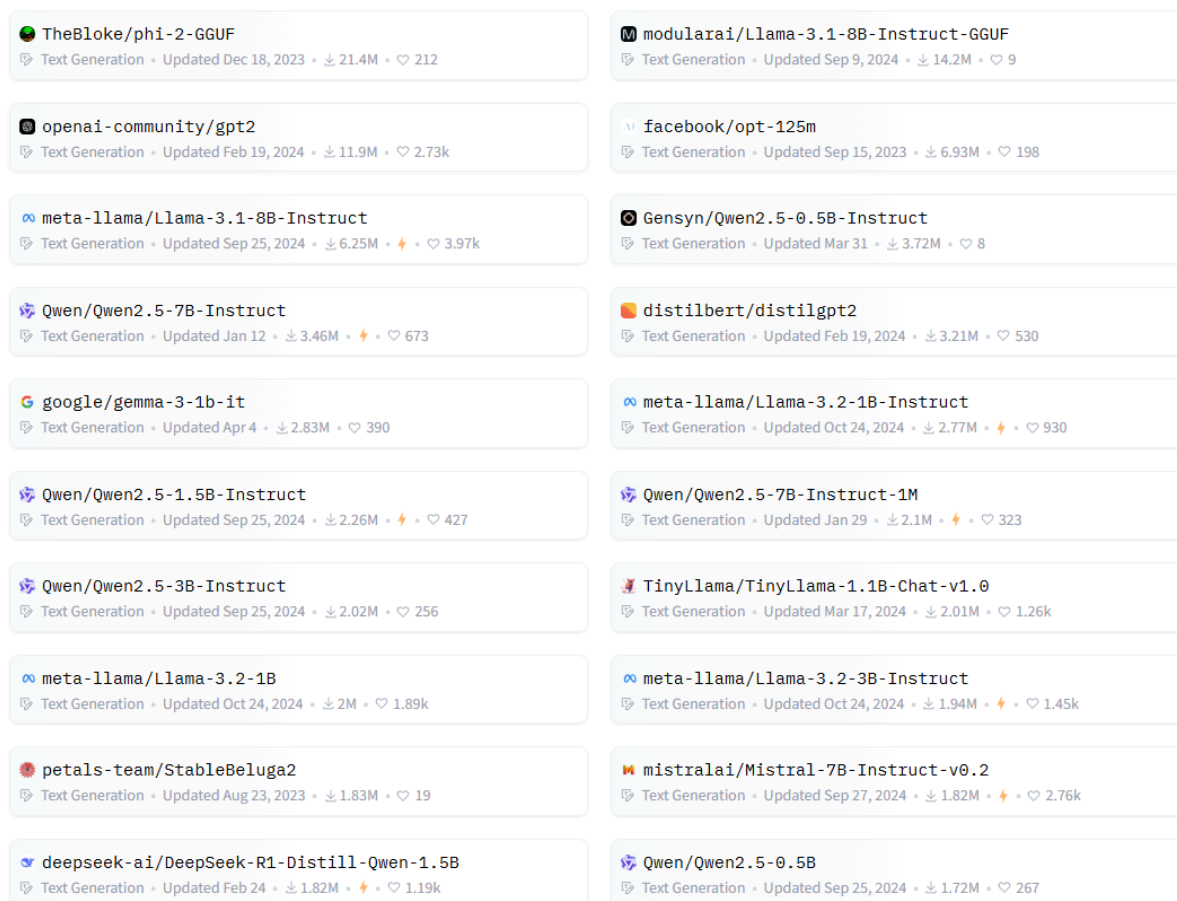


Figura 1. Los modelos de lenguaje más utilizados en HuggingFace a principios de 2025 (nótese que no todos corresponden a asistentes conversacionales) [14].

Es común ver numerosas versiones de un mismo modelo de lenguaje, que normalmente varían en números de parámetros. Si nos centramos en aquellos modelos entrenados específicamente como asistentes conversacionales más usados en la actualidad (tal y como se muestra en la Figura 1), podemos hacer una lista según su nivel de relevancia.

1. **Llama 3.1 [15].** Publicado en 2024 por Meta, su versión más utilizada con un tamaño de 8 mil millones de parámetros es "Llama-3.1-8B-Instruct". Según sus pruebas realizadas, esta versión ofrece una calidad comparable a la versión de 70 mil millones de parámetros de Llama 2; mientras que su versión más grande ofrece respuestas superiores a las de los modelos Gemini Pro 1.5 y Claude 3.
2. **Qwen 2.5 [16].** Publicado en 2024 por Alibaba, su versión más utilizada con un tamaño de 7 mil millones de parámetros es "Qwen2.5-7B-Instruct". Algunos de sus benchmarks más específicos muestran que la variante más avanzada de esta familia presenta un desempeño comparable al de GPT4-o.

3. **Gemma 3 [17]**. Publicado en 2025 por Google, su versión más utilizada con un tamaño de mil millones de parámetros es "gemma-3-1b-it". Sigue la tendencia de modelos compactos y eficientes, siendo construido a partir de la misma investigación y tecnología utilizadas para crear los modelos Gemini.
4. **TinyLlama v1.0 [18]**. Publicado en 2024 por Zhang Peiyuan, su versión más utilizada con un tamaño de mil 100 millones de parámetros es "TinyLlama-1.1B-Chat-v1.0". Se trata de un modelo con la arquitectura de Llama 2 pre-entrenado usando sólo el tamaño mencionado, ofreciendo otra versión compacta dentro de la familia.
5. **DeepSeek R1 Distill Qwen [19]**. Publicado en 2025 por DeepSeek, su versión más utilizada con un tamaño de mil 500 millones de parámetros es "DeepSeek-R1-Distill-Qwen-1.5B". Se trata de una versión destilada del reciente DeepSeek R1 (del cual se notifican respuestas comparables a las de los modelos GPT-4 y o1) a partir del reentrenamiento de Qwen 2.5.
6. **Mistral v0.2 [20]**. Publicado en 2024 por Mistral, su versión más utilizada con un tamaño de 7 mil millones de parámetros es "Mistral-7B-Instruct-v0.2". De acuerdo con sus benchmarks, supera en desempeño a la versión de 13 mil millones de parámetros de Llama 2.

Una característica en común en muchos de ellos es que sus pesos están en formato de 16 bits en lugar de los 32 bits por defecto, esto reduce teóricamente la cantidad de memoria necesaria hasta la mitad. La pérdida de precisión generalmente suele ser despreciable, pero se irá haciendo más notable conforme se siga reduciendo la cantidad de bits.

Dado que nuestro principal interés es seleccionar un modelo con base a su actualidad, desempeño, popularidad y accesibilidad al público general, se ha decidido optar por Llama 3.1, ya que continúa siendo uno de los más populares y valorados por su equilibrio entre calidad y cantidad de parámetros.

2.2.2. MODELOS DE LENGUAJE COMERCIALES

Por lo general, este tipo de modelos de lenguaje suelen destacar por su gran tamaño y elevado desempeño; no obstante, tanto sus pesos como su código fuente suelen ser de

carácter propietario, lo cual limita su transparencia y personalización, y su acceso está usualmente condicionado a esquemas comerciales.

La amplia mayoría de los modelos comerciales son asistentes conversacionales, y entre los más populares, podemos destacar los siguientes.

- **GPT-4o (ChatGPT) [21]**. Publicado en 2024 por OpenAI, es actualmente el modelo de lenguaje más avanzado y el más ampliamente utilizado. Es más rápido y tiene más capacidades que su predecesor GPT-4, además de ser capaz de procesar texto, imagen, audio y vídeo.
- **GPT-4o (GitHub Copilot) [22]**. Este mismo modelo, al igual que otros similares, ha sido incorporado en la herramienta de autocompletado de código GitHub Copilot. Inicialmente dicha herramienta se basaba en Codex, una versión reentrenada de GPT-3 enfocada en programación.
- **Gemini 2.0 Flash [23]**. Publicado en 2025 por Google, es considerado como un notable salto respecto a su predecesor Gemini 1.5 Flash. Además de mejorar su velocidad y calidad, introduce nuevas características como su manejo de eventos a tiempo real y sus capacidades de agente.
- **Claude 3.7 Sonnet [24]**. Publicado en 2025 por Anthropic, se trata de un modelo de razonamiento híbrido que permite a los usuarios elegir entre respuestas rápidas y respuestas razonadas. Este modelo integra ambas capacidades en un único marco, eliminando la necesidad de múltiples modelos.

Por motivos similares a los que motivaron la elección de Llama 3.1, se ha seleccionado la herramienta GitHub Copilot basada en GPT-4o. Esta decisión se justifica tanto por su alta popularidad y relevante papel en el desarrollo de aplicaciones web, como por la considerable cantidad de estudios observados sobre la herramienta.

En este capítulo se documentan los aspectos más relevantes tanto de la planificación como del desarrollo del experimento, destacando la elaboración de las plantillas de las aplicaciones web y la formulación de las instrucciones para los modelos de lenguaje.

3.1. ESPECIFICACIÓN DE REQUISITOS

De acuerdo con lo planteado, la evaluación de los modelos se llevará a cabo mediante su aplicación práctica en el desarrollo de dos aplicaciones web de distinta complejidad, recopilando datos necesarios en el proceso para su posterior análisis. Las funcionalidades establecidas para cada una provienen de ideas populares en el ámbito del desarrollo web.

Asimismo, establecer distintos grados de complejidad permite evaluar cómo influye el tamaño del proyecto en la capacidad de los modelos de lenguaje para producir código funcional, ya que los requisitos funcionales serán abordados principalmente por sus soluciones generadas.

La primera aplicación web (denominada "TFM0") consiste en un gestor de recetas, cuyos requisitos funcionales quedan resumidos en la Tabla 1.

RF1	La aplicación web debe permitir la visualización de recetas y sus pasos a seguir.
RF2	La aplicación web debe permitir la creación, edición y eliminación de recetas.
RF3	La aplicación web debe permitir la creación y eliminación de pasos.
RF4	La aplicación web debe mostrar los pasos de cada receta en orden ascendente.

Tabla 1. Resumen de los requisitos de la primera aplicación web (gestor de recetas).

De manera resumida, la funcionalidad principal de esta aplicación web consiste en ofrecer un sistema sencillo de gestión de recetas. Se busca representar una categoría particular de aplicaciones web de baja complejidad, con pocas entidades y una lógica principal basada en operaciones básicas de creación, lectura, actualización y eliminación (CRUD).

A través de la plataforma, el usuario puede visualizar todas las recetas que ha creado (Figura 2), así como consultar detalladamente los pasos de elaboración de cada una (Figura 3), permitiendo en todo momento la realización de modificaciones.

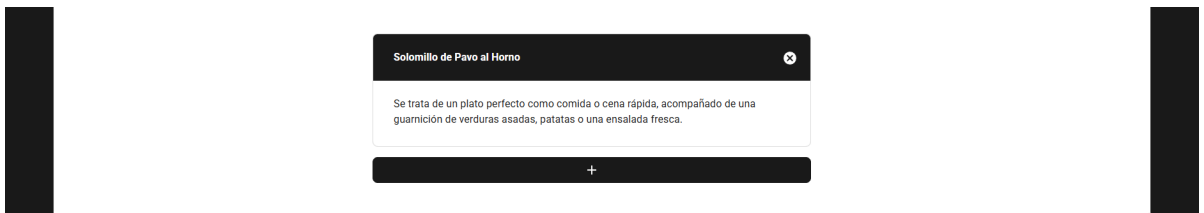


Figura 2. Página principal del gestor de recetas y ejemplo de una posible receta.

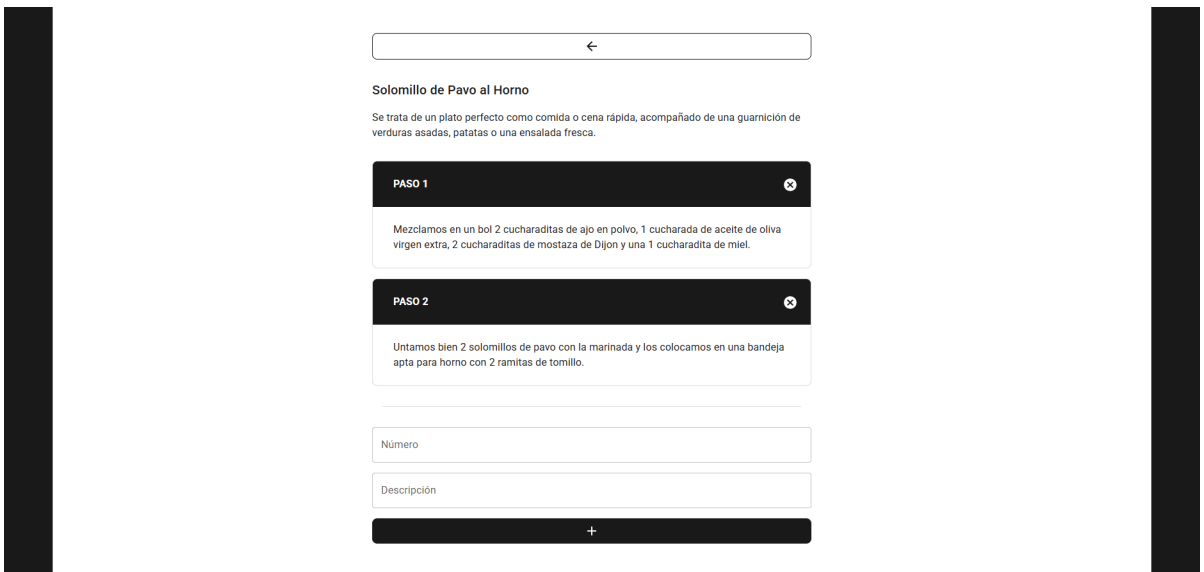


Figura 3. Ejemplo de una posible receta junto a sus pasos a seguir.

La segunda aplicación web (denominada "TFM1") consiste en una plataforma inmobiliaria, cuyos requisitos funcionales quedan resumidos en la Tabla 2.

RF1	La aplicación web debe permitir la autenticación y registro de usuarios.
RF2	La aplicación web debe permitir la edición y eliminación de usuarios.

RF3	La aplicación web debe permitir la consulta de reseñas realizadas a un usuario.
RF4	La aplicación web debe permitir la creación, edición y eliminación de reseñas a un usuario.
RF5	La aplicación web debe permitir la visualización de casas por país y por usuario.
RF6	La aplicación web debe permitir la creación, edición y eliminación de casas.
RF7	La aplicación web debe permitir la visualización de órdenes de compra, así como su aprobación o rechazo.
RF8	La aplicación web debe permitir la creación y eliminación de órdenes de compra a una casa.

Tabla 2. Resumen de los requisitos de la segunda aplicación web (plataforma inmobiliaria).

De manera resumida, la funcionalidad principal de esta aplicación web consiste en ofrecer un sistema de compraventa de inmuebles (Figuras 5 y 6). Se busca representar una categoría de aplicaciones web más complejas, con un mayor número de entidades relacionadas entre sí y con operaciones más heterogéneas.

A través de la plataforma, el usuario puede enviar una orden de compra con su oferta y datos personales para negociar una adquisición (Figura 7), con posibilidad de compartir opiniones sobre otros usuarios mediante reseñas (Figura 4).

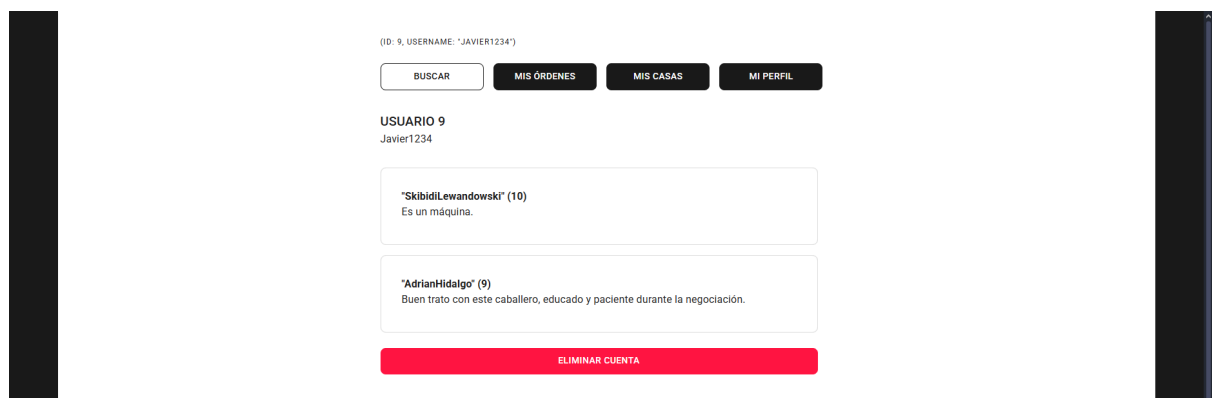


Figura 4. Ejemplo de un posible usuario junto a dos reseñas que ha recibido.

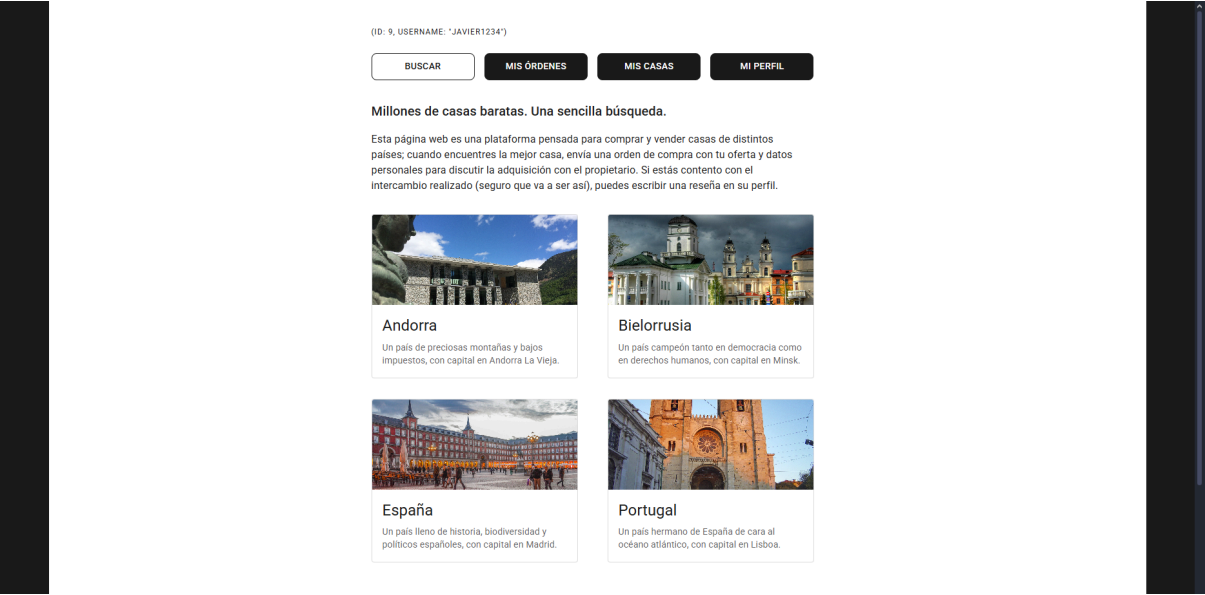


Figura 5. Página principal de la plataforma inmobiliaria.

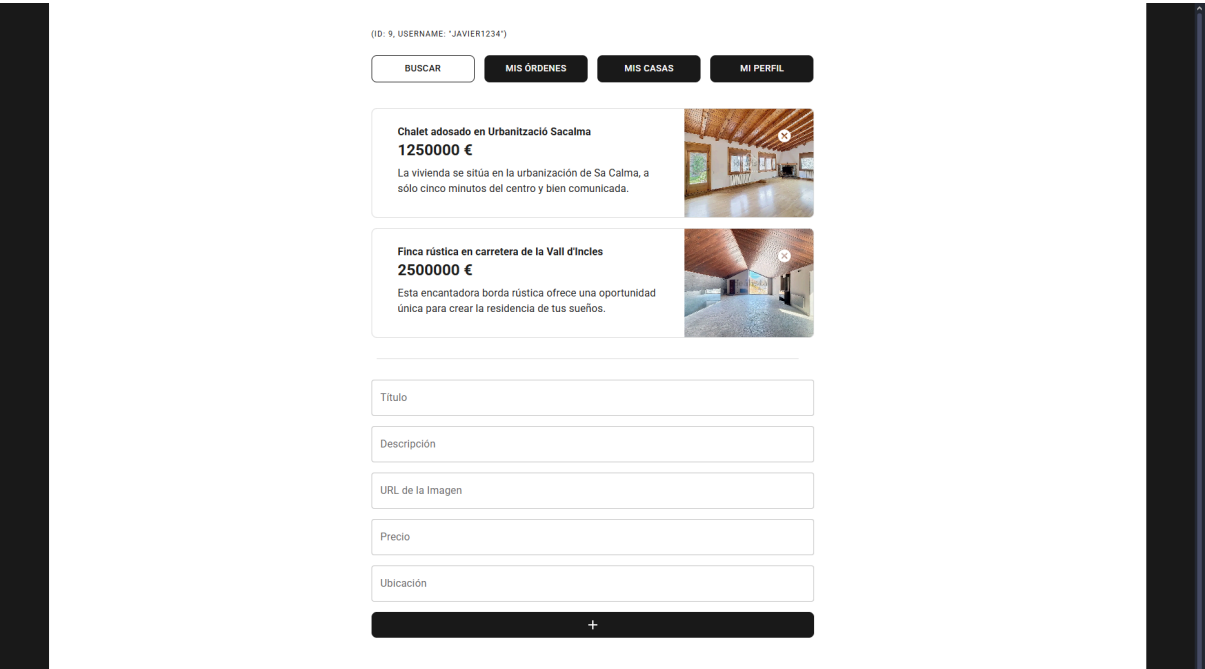


Figura 6. Sección dedicada a la gestión de casas de un usuario.

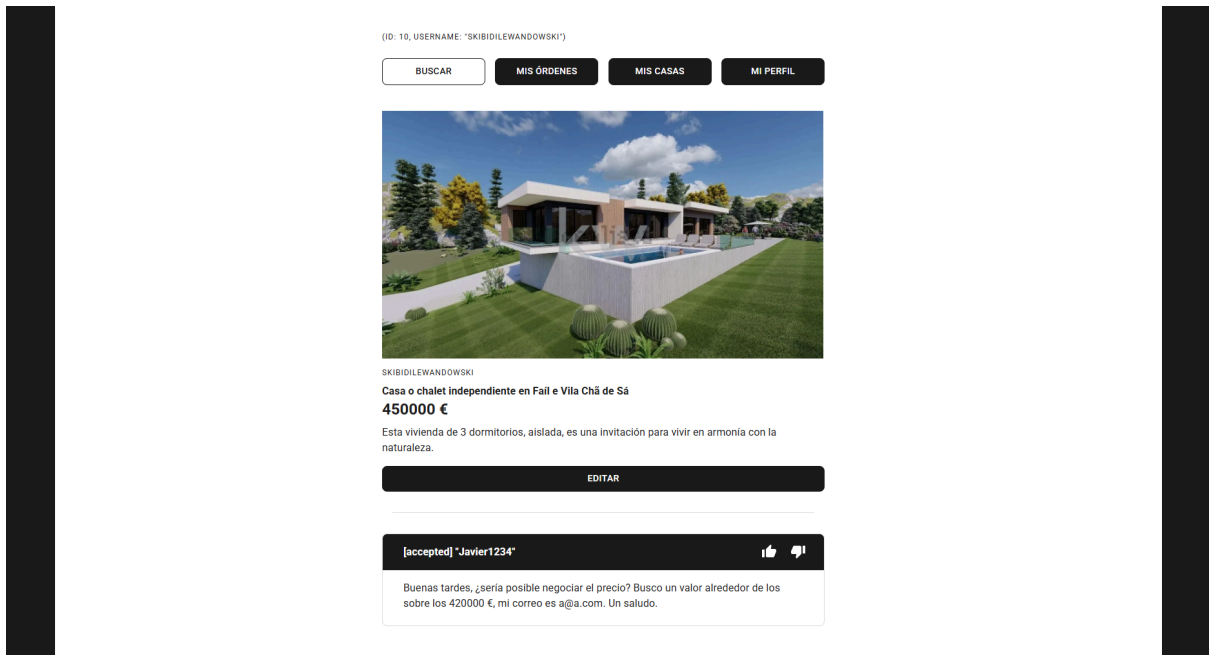


Figura 7. Ejemplo de una posible casa junto a una orden de compra que el usuario ha recibido y aceptado.

3.2. DISEÑO DE BASES DE DATOS

Cada aplicación web posee una base de datos conformada por distintos números de entidades según su complejidad, representadas respectivamente junto a sus atributos y relaciones en las Figuras 8 y 9.

En la primera aplicación web, la entidad Recipe describe a una receta (con su título y descripción) y la entidad Step describe a un paso (con su número y descripción); la relación entre ambos queda descrita en el atributo recipe de Step, que permite que una receta pueda tener múltiples pasos a seguir.

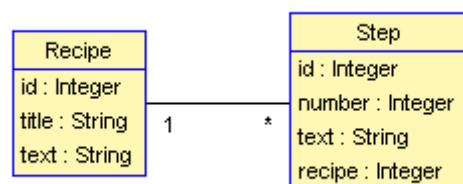


Figura 8. Diagrama de las entidades de la primera aplicación web (gestor de recetas).

En la segunda aplicación web, la entidad User describe a un usuario (con su correspondiente nombre y contraseña) y la entidad Review describe a una reseña (con su

nota y descripción); la relación entre ambos queda descrita en los atributos `reviewer` y `reviewed` de `Review`, que permiten respectivamente que un usuario pueda escribir múltiples reseñas a otros diferentes usuarios y a su vez pueda recibir múltiples reseñas.

En adición con lo anterior, la entidad `Order` describe a una orden de compra (con su descripción y estado) y la entidad `House` describe a un casa (con su título, descripción, imagen, precio y país); la relación entre ambos queda descrita en el atributo `house` de `order`, que permite que una casa pueda tener múltiples órdenes de compra.

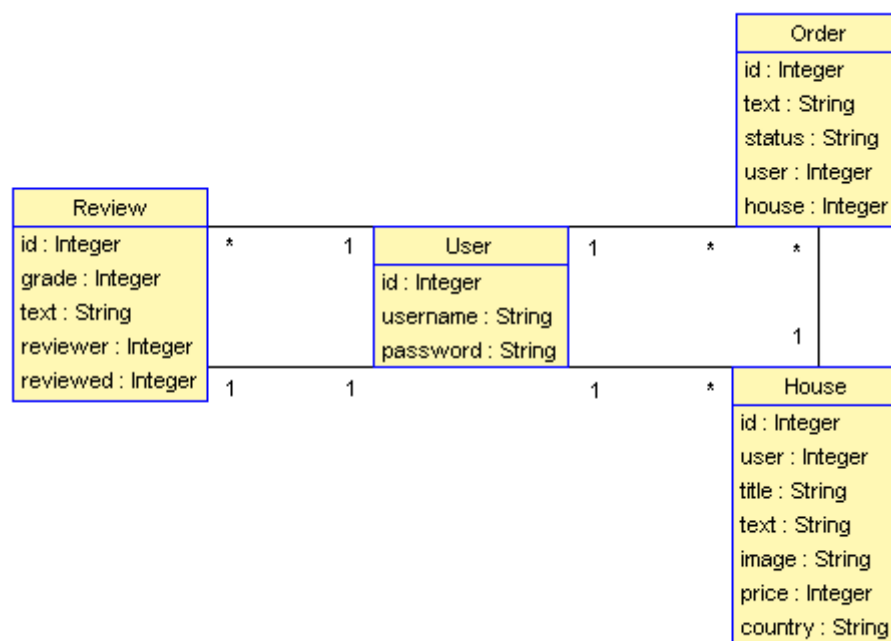


Figura 9. Diagrama de las entidades de la segunda aplicación web (plataforma inmobiliaria).

Por último, la relación entre estas dos últimas entidades y `User` quedan descritas en los atributos `user` de `Order` y `House`, que permiten respectivamente que un usuario pueda tener múltiples casas en venta y a su vez pueda hacer múltiples órdenes de compra a diferentes casas.

3.3. DISEÑO DE INSTRUCCIONES

El diseño de instrucciones para los modelos de lenguaje está profundamente ligado al de las plantillas para las aplicaciones web, ya que cada instrucción corresponde a la resolución de una operación específica. Ambos procedimientos han sido objeto de un proceso continuo de refinamiento.

Estas plantillas han sido diseñadas con el objetivo de que la experimentación sea lo más estructurada y medible posible, enfocándose en las partes más importantes del código.

- Para cada servidor, se ha configurado su correspondiente base de datos y se han modelado las entidades pertinentes; asimismo, se han delineado los puntos de acceso que serán utilizados, dejando predefinidas las operaciones que deberán ser completadas posteriormente por los modelos de lenguaje.
- Para cada cliente, se ha diseñado una interfaz gráfica simple, delineando también las operaciones específicas asociadas a cada interacción con el servidor; estas operaciones, al igual que en el caso anterior, deberán ser completadas posteriormente por los modelos de lenguaje.

Entre ambas aplicaciones web, se han elaborado un total de cuarenta instrucciones, diseñadas para ser utilizadas casi de la misma forma en ambos modelos de lenguaje. Aunque no se ha hecho un uso intensivo de técnicas complejas de ingeniería de instrucciones, todas siguen una estructura uniforme y clara; y se han redactado de manera realista, como si fueran peticiones dirigidas a una persona real.

La estructura general adoptada para el diseño de las instrucciones corresponde a la representada en la Figura 10, aunque podrá presentar algunas variaciones en función de los casos que se abordan en las secciones siguientes. De ella podemos desglosar los siguientes elementos.

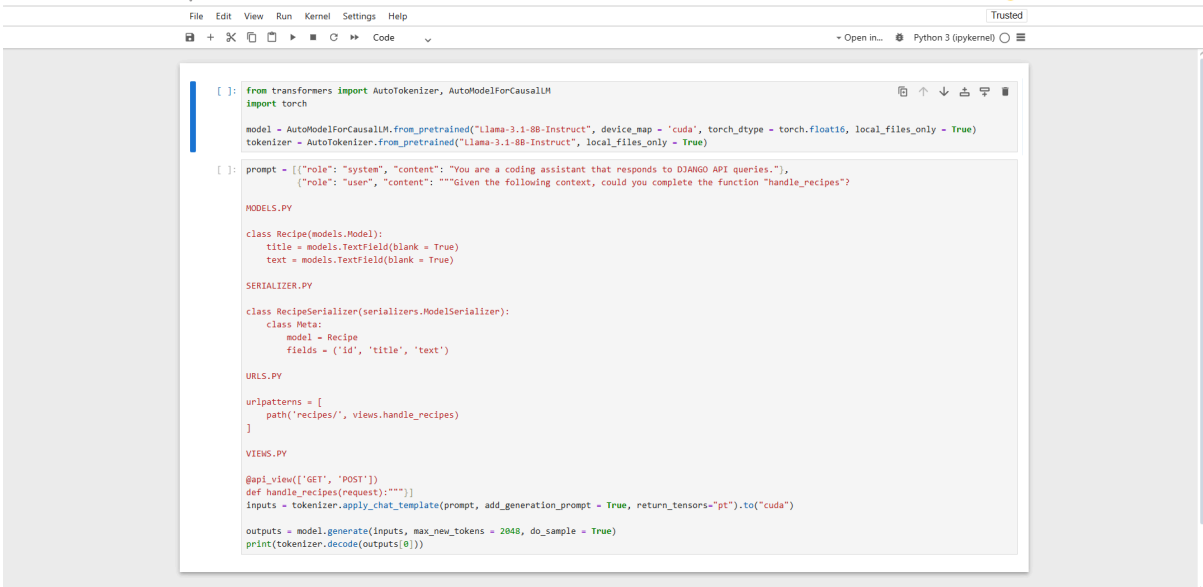
- La propia petición en sí, que incluye una simple descripción natural de la acción requerida (por ejemplo, "completa la operación `borrarEntidades`, y actualiza las variables...") a la que, según el caso, se le suma el procedimiento exacto a seguir para su correcta ejecución (por ejemplo, "usa este identificador para obtener la lista con las entidades; y mediante un bucle, usa este punto de acceso para...").
- El contexto necesario para atender adecuadamente dicha petición, que generalmente incluye fragmentos de código tanto dentro de otros archivos específicos como dentro del archivo objetivo, que es el que contiene la función objetivo que se desea completar.

PETICIÓN	DESCRIPCIÓN
	PROCEDIMIENTO EXACTO
CONTEXTO	ARCHIVOS ADICIONALES
	ARCHIVO OBJETIVO

Figura 10. Estructura general de las instrucciones formuladas.

El procedimiento utilizado para ejecutar una instrucción en Llama 3.1 es a través de la herramienta Jupyter Notebook y las bibliotecas instaladas PyTorch y Transformers. A través de ellas, se carga y configura el modelo previamente descargado desde HuggingFace, y se van ejecutando las instrucciones correspondientes para cada operación (Figura 11).

El procedimiento utilizado para ejecutar una instrucción en GitHub Copilot (GPT-4o) es a través de la herramienta Visual Studio Code y su correspondiente extensión. A través de sus interfaces, se selecciona el modelo y los archivos que forman parte del contexto, y se van ejecutando las instrucciones correspondientes para cada operación (Figura 12).



```
[ ]: from transformers import AutoTokenizer, AutoModelForCausalLM
import torch

model = AutoModelForCausalLM.from_pretrained("Llama-3.1-8B-Instruct", device_map = 'cuda', torch_dtype = torch.float16, local_files_only = True)
tokenizer = AutoTokenizer.from_pretrained("Llama-3.1-8B-Instruct", local_files_only = True)

[ ]: prompt = [{"role": "system", "content": "You are a coding assistant that responds to DJANGO API queries."},
{"role": "user", "content": ""}Given the following context, could you complete the function "handle_recipes"?

MODELS.PY

class Recipe(models.Model):
    title = models.TextField(blank = True)
    text = models.TextField(blank = True)

SERIALIZER.PY

class RecipeSerializer(serializers.ModelSerializer):
    class Meta:
        model = Recipe
        fields = ('id', 'title', 'text')

URLS.PY

urlpatterns = [
    path('recipes/', views.handle_recipes)
]

VIEWS.PY

@api_view(['GET', 'POST'])
def handle_recipes(request):"""
    inputs = tokenizer.apply_chat_template(prompt, add_generation_prompt = True, return_tensors="pt").to("cuda")

    outputs = model.generate(inputs, max_new_tokens = 2048, do_sample = True)
    print(tokenizer.decode(outputs[0]))
```

Figura 11. Procedimiento utilizado en Jupyter Notebook para la ejecución de una instrucción en Llama 3.1.

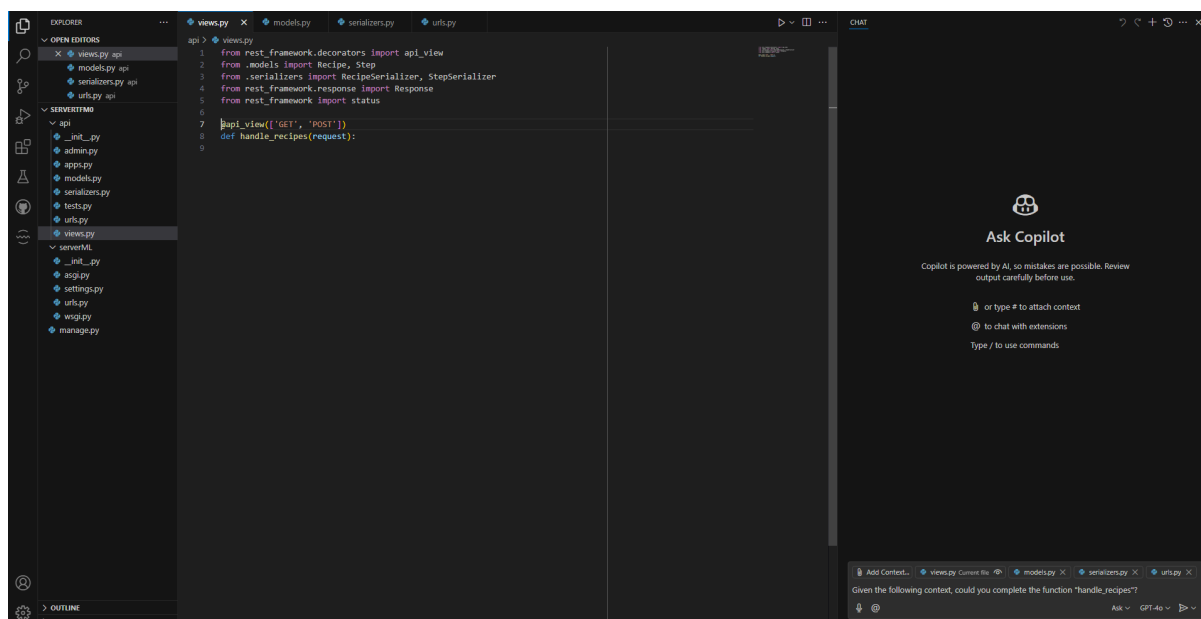


Figura 12. Procedimiento utilizado en Visual Studio Code para la ejecución de una instrucción en GitHub Copilot (GPT-4o).

3.3.1. PARTE DEL SERVIDOR

La tarea para cada modelo de lenguaje será la de desarrollar las operaciones para los puntos de acceso de cada servidor. Cada una de estas operaciones están enumeradas y etiquetadas bajo el prefijo "S" tanto en el código como en otros informes, quedándonos con un total de cuatro en la primera aplicación web y un total de ocho en la segunda.

Independientemente del modelo, el archivo objetivo se denomina "views.py" y los archivos adicionales con el resto del contexto se denominan "models.py" (que especifica las entidades), "serializers.py" (que especifica la representación de las entidades) y "urls.py" (que especifica los puntos de acceso disponibles).

Por lo general, la petición consistirá en pedir completar una operación y especificar, en su caso, si se desean añadir filtros por determinados atributos en la correspondiente entidad.

En la Figura 13 se muestra la instanciación concreta para nuestra experimentación de la mostrada anteriormente en la Figura 10. Se debe aclarar que la definición del sistema sólo es necesaria en Llama 3.1 por su forma específica de entrenamiento. De manera similar, el contexto con los fragmentos de código tampoco será necesario de incluir dentro de la instrucción en GitHub Copilot (GPT-4o), al ofrecer desde su interfaz la posibilidad de adjuntar directamente una selección de archivos enteros.

PETICIÓN	DEFINICIÓN DEL SISTEMA	"You are a coding assistant that responds to DJANGO API queries."
	DESCRIPCIÓN	
	PROCEDIMIENTO EXACTO	
CONTEXTO	ARCHIVOS ADICIONALES	MODELS.PY
		SERIALIZERS.PY
		URLS.PY
	ARCHIVO OBJETIVO	VIEWS.PY
		FUNCIÓN OBJETIVO

Figura 13. Estructura general de las instrucciones formuladas para los servidores de las aplicaciones web.

3.3.2. PARTE DEL CLIENTE

La tarea para cada modelo de lenguaje será la de desarrollar las operaciones delineadas desde la interfaz gráfica de cada cliente. Cada una de estas operaciones están enumeradas y etiquetadas bajo el prefijo "C" tanto en el código como en otros informes, quedándonos con un total de ocho en la primera aplicación web y un total de veinte en la segunda.

Independientemente del modelo, el archivo objetivo se denomina "App.tsx" y los archivos adicionales con el resto del contexto se denominan "serializers.py" (que especifica la representación de las entidades), "urls.py" (que especifica los puntos de acceso disponibles) y "views.py" (que especifica el funcionamiento de dichos puntos de acceso).

Por lo general, la petición consistirá en pedir completar una operación y especificar, en su caso, las variables que se quieran leer o modificar, así como otras funciones a llamar.

En la Figura 14 se muestra de igual forma la instanciación concreta para nuestra experimentación de la mostrada anteriormente en la Figura 10; y se reitera lo mismo que con la anterior, solo será necesario en Llama 3.1 explicitar dentro de la instrucción la definición del sistema y el contexto con los fragmentos de código necesarios, con el objetivo de utilizar la interfaz de GitHub Copilot conforme a su diseño intencionado.

PETICIÓN	DEFINICIÓN DEL SISTEMA	"You are a coding assistant that responds to REACT in TYPESCRIPT queries."
	DESCRIPCIÓN	
	PROCEDIMIENTO EXACTO	
CONTEXTO	ARCHIVOS ADICIONALES	SERIALIZERS.PY
		URLS.PY
		VIEWS.PY
	ARCHIVO OBJETIVO	APP.TSX
		FUNCIÓN OBJETIVO

Figura 14. Estructura general de las instrucciones formuladas para los clientes de las aplicaciones web.

3.4. DESARROLLO Y REVISIÓN DE CÓDIGO

Siguiendo los requisitos definidos, se procede a ejecutar cada instrucción de manera individual y aislada, estableciendo un nivel de dificultad (entre "fácil", "medio" y "difícil") según la dificultad para el modelo de lograr una solución funcional, además de unas anotaciones al respecto destacando algún comportamiento observado.

Estos datos han sido recopilados en el informe adjuntado en el apéndice, y el criterio utilizado para asignar los distintos niveles de dificultad es el siguiente.

1. En primera instancia, se formula únicamente una descripción de la petición sin incluir el procedimiento exacto, especificando simplemente la acción deseada; si el modelo de lenguaje es capaz de generar consistentemente una solución funcional sin necesidad de correcciones estructurales, se declara la instrucción como "fácil".
2. Si no se logra una solución funcional siguiendo el caso anterior, se formula el procedimiento exacto, incorporando explicaciones técnicas sobre el procedimiento a seguir, así como advertencias respecto a prácticas inadecuadas. Si bajo estas condiciones se consigue generar una solución funcional, se declara la instrucción como "medio"; en caso contrario, se termina declarando como "difícil".

El resultado final de nuestra experimentación son cuatro aplicaciones web, ya que se generaron dos versiones, una con código generado por Llama 3.1 [25][26] y otra por GitHub Copilot (GPT-4o) [27][28].

Dado que la calidad que se desea evaluar es sólo la del código generado, de cada una de estas versiones, se han tomado sus archivos objetivos ("views.py" y "App.tsx") y se les ha sometido a un análisis de calidad mediante la herramienta SonarQube, cuyos resultados obtenidos a priori se presentan en la Figura 15.

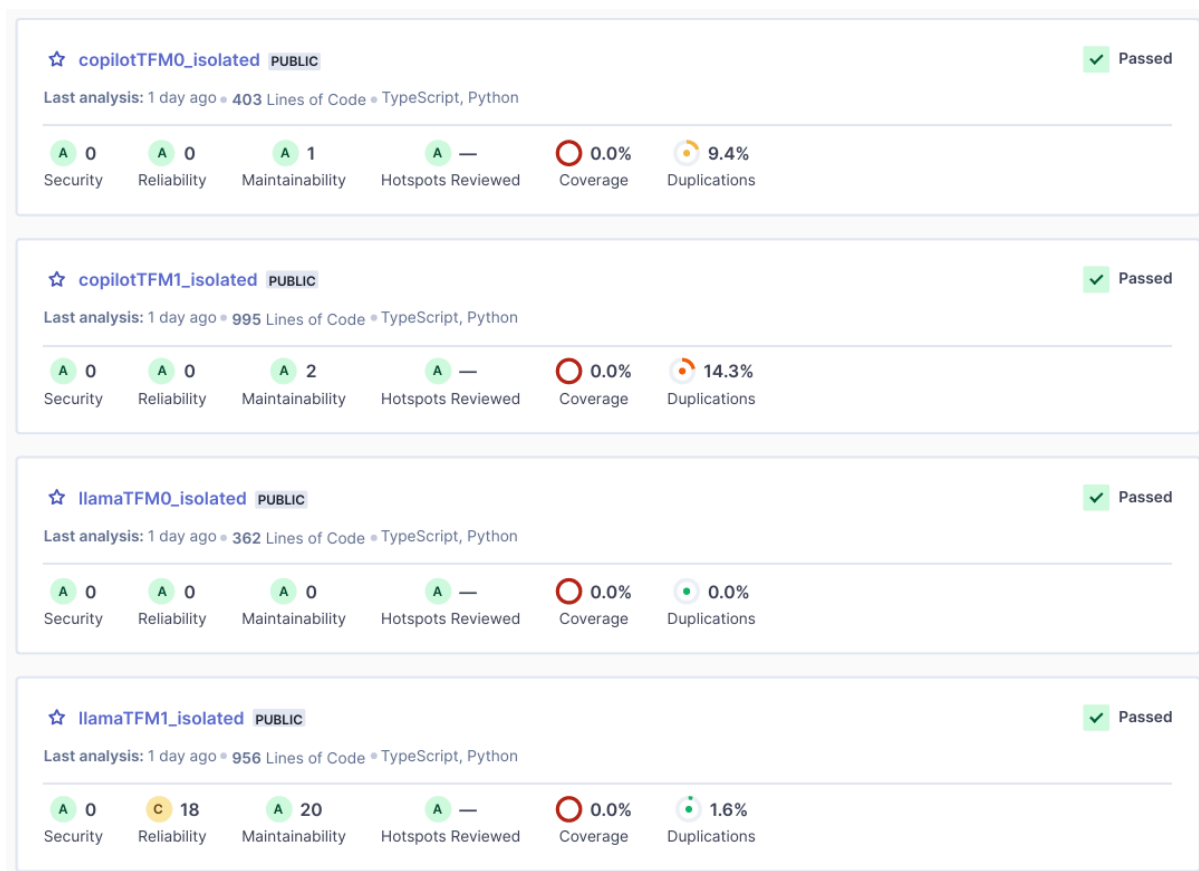


Figura 15. Resultados de los análisis de calidad del código realizados por la herramienta SonarQube para las aplicaciones web.

Aunque parte de estos resultados son relevantes para el estudio, algunos de los problemas notificados no aportan demasiado valor (por ejemplo, la herramienta confunde en una parte las funciones flecha con los cierres de las etiquetas HTML), por lo que los resultados reales serían los de la Figura 16.

De manera resumida, todas las versiones superan satisfactoriamente los criterios de calidad establecidos por SonarQube, y presentan más similitudes que diferencias entre sí.

	SECURITY	RELIABILITY	MAINTAINABILITY
copilotTFM0	0	0	1
llamaTFM0	0	0	0
copilotTFM1	0	0	0
llamaTFM1	0	0	1

Figura 16. Resultados procesados de los análisis de calidad del código realizados por la herramienta SonarQube.

4

ANÁLISIS DE RESULTADOS

En este capítulo se presentan y analizan en detalle los resultados obtenidos del experimento, considerando tanto el desempeño como la calidad de los modelos; además de proporcionar posibles respuestas a las preguntas de investigación.

4.1. PRIMERA LECTURA DE RESULTADOS

El propósito de este estudio es el de comparar el código generado por los modelos de Llama 3.1 y GitHub Copilot (GPT-4o), evaluando también la viabilidad de hacer aplicaciones web *full-stack* con ellos; los resultados obtenidos evidencian que ambas opciones son viables, aunque existen particularidades que deben tenerse en cuenta.

Durante las etapas finales del proceso de desarrollo, se observó un patrón recurrente en el comportamiento de ambos modelos de lenguaje al enfrentarse a tareas de mayor complejidad, en el que, independientemente del tipo de operación solicitada y lo que se pida, la principal causa de disminución en el desempeño está asociada a la cantidad de entidades que deben ser procesadas simultáneamente.

4.2. RESULTADOS DE DESEMPEÑO

Los datos recopilados del experimento aparecen resumidos en la Figura 17, mostrando los niveles de dificultad bajo un código de colores y el número total de líneas de código de todas las operaciones.

De las cuarenta operaciones realizadas, cuatro son problemáticas para Llama 3.1 (todas dentro de los clientes); mientras que con GitHub Copilot (GPT-4o) sólo fue una operación, que coincide con una de las problemáticas en Llama. Cabe señalar que el número de líneas de código puede incluir tanto líneas en blanco como con comentarios, y que algunas sentencias idénticas pueden haber sido redactadas con más o menos saltos de líneas.

	S1	S2	S3	S4	C1	C2	C3	C4	C5	C6	C7	C8	S1	S2	S3	S4	S5	S6	S7	S8	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16	C17	C18	C19	C20
LLAMA	12	21	21	21	12	33	22	33	32	32	34	15	42	48	50	48	50	48	54	48	26	41	16	25	26	56	25	27	32	30	28	25	28	43	31	26	63	11	27	18
COPILOT	15	25	23	25	15	35	27	33	32	32	48	18	22	22	26	22	28	22	28	22	27	39	17	30	30	48	30	27	31	33	31	31	31	33	26	26	95	17	34	18

Figura 17. Resumen de los resultados obtenidos para cada operación y aplicación web. En cada celda se presenta el número total de líneas de código resultantes, mientras que los colores verde, amarillo y rojo indican, respectivamente, los niveles de dificultad asignados "fácil", "medio" y "difícil".

Si bien no es posible establecer conclusiones sobre el desempeño de una operación únicamente a partir de su extensión en líneas de código, puede apreciarse que las operaciones de mayor complejidad tienden a requerir una cantidad de líneas superior al promedio; aunque esta correlación no siempre es constante.

- En la primera aplicación web, la única instrucción problemática es "C7" (función `deleteRecipe`) y la operación consiste en eliminar una entidad y sus hijos ordenadamente. Entre los modelos evaluados, únicamente Llama 3.1 es el que presenta problemas, con una dificultad "media" por inventarse algunas operaciones no definidas del servidor.
- En la segunda aplicación web, una de las instrucciones problemáticas es "C6" (función `getHouse`) y la operación consiste en obtener una entidad y sus hijos, y adicionalmente, obtener algunos valores de sus atributos mediante otras peticiones. Entre los modelos evaluados, únicamente Llama 3.1 es el que presenta problemas, con una dificultad "difícil" por inventarse algunos atributos de las entidades.
- En la segunda aplicación web, una de las instrucciones problemáticas es "C17" (función `deleteUser`) y la operación consiste en eliminar una entidad, sus hijos y los hijos de sus hijos ordenadamente. Llama 3.1 tiene dificultad "difícil" por no conseguir respetar los órdenes de forma exacta y GPT-4o en GitHub Copilot tiene dificultad "media" por inventarse algunas operaciones no definidas del servidor.
- En la segunda aplicación web, una de las instrucciones problemáticas es "C19" (función `deleteHouse`) y la operación consiste en eliminar una entidad y sus hijos ordenadamente. Entre los modelos evaluados, únicamente Llama 3.1 es el que presenta problemas, con una dificultad "media" por inventarse algunas operaciones no definidas del servidor.

Por lo general, los principales problemas observados pueden resumirse en las alucinaciones de atributos y operaciones inexistentes (pese a proveerse un contexto claro).

La conclusión general que se establece es que, al menos en este caso de estudio donde las operaciones son generadas de forma individual y aislada, ambos modelos de lenguaje son capaces de proveer soluciones sin mayores dificultades en la amplia mayoría de los casos (36 de 40 operaciones, aproximadamente el 90%), siendo GPT-4o en GitHub Copilot el modelo con una mayor ventaja comparativa respecto a su desempeño.

4.3. RESULTADOS DE CALIDAD

Profundizando en los resultados de SonarQube, la calidad del código generado es, en términos generales, satisfactoria. Los problemas detectados son escasos y de baja importancia, lo que sugiere que, pese al buen desempeño de ambos modelos, no son perfectos y aún pueden cometer ciertos errores menores.

- En la primera aplicación web, sólo se ha notificado un problema de mantenibilidad con nivel de severidad "alto", en la versión de GitHub Copilot (GPT-4o); se localiza en el cliente, concretamente en la instrucción "C2", relacionada con la función "getRecipe", la cual suma más de cuatro niveles de anidamiento.
- En la segunda aplicación web, sólo se ha notificado un problema de mantenibilidad con nivel de severidad "medio" en la versión de Llama 3.1; se localiza en el cliente, concretamente en la instrucción "C17", relacionada con la función "deleteUser", en la cual dentro de su procedimiento se crea una variable que se deja sin utilizar.

La conclusión general que se establece es que, al menos en este caso de estudio donde las operaciones son generadas de forma individual y aislada, ambos modelos de lenguaje son capaces de generar funciones mayoritariamente limpias en la amplia mayoría de los casos, siendo prácticamente idénticos en calidad del código.

4.4. RESPUESTAS PLANTEADAS A LOS OBJETIVOS

Una vez concluidas las etapas previas del estudio, puede procederse a responder las preguntas planteadas inicialmente en relación con los objetivos de investigación.

- RQ1. ¿Es posible crear una determinada aplicación web completa con alguno de los modelos de lenguaje seleccionados mediante instrucciones en lenguaje natural?

Se concluye que sí es posible llevar a cabo el desarrollo de una aplicación web completa mediante este enfoque, al menos bajo condiciones y tecnologías similares a las del marco del estudio.

Aunque determinadas instrucciones puedan presentar distintos desafíos (en nuestro caso, alrededor del 10%), si se proporcionan instrucciones mínimamente claras y específicas, sí es posible generalmente tanto con un modelo de código abierto como Llama 3.1 como con un modelo comercial como GPT-4o en GitHub Copilot.

RQ1.1. ¿Cuáles han sido las principales dificultades y desafíos observados en el proceso para cada modelo de lenguaje?

Se han observado toda variedad de dificultades y desafíos en todo el experimento desarrollado, particularmente en los clientes de las aplicaciones web, siendo notable que en los servidores apenas se hayan observado problemas.

Es importante resaltar el papel de estos modelos de lenguaje como herramientas de apoyo en el contexto de este estudio, y que la gravedad percibida de un problema está influida por el punto de vista de alguien con conocimientos en programación; de forma que una persona sin ese trasfondo podría tener una experiencia significativamente distinta.

- Entre los problemas de menor gravedad, se pueden destacar aquellos asociados a instrucciones con dificultad "fácil", que como se describió, requirieron entre una intervención mínima y nula. Estos problemas requirieron correcciones superficiales, como rectificar URLs mal escritas, eliminar sentencias de depuración innecesarias; o en el peor de los casos, corregir nombres de atributos incorrectos.
- Entre los problemas de mayor gravedad, suelen ser los esperables en un modelo de lenguaje, como las alucinaciones y la generación de código incoherente. En el caso de Llama 3.1, como es esperable al ser un modelo mucho más pequeño, sucede más ocasionalmente; pero ambos modelos han exhibido estos comportamientos al enfrentarse a funciones complejas.

Por otro lado, también se pueden destacar desafíos en la configuración de los LLMs. Por el lado de Llama, aunque entre la comunidad se considere "asequible" su tamaño de 8 mil millones de parámetros, puede resultar difícil de utilizar para una persona ajena. También es

necesaria una GPU con suficiente memoria VRAM para alcanzar una velocidad razonable, y dicha necesidad es mayor si las instrucciones se vuelven más largas y complejas.

En el caso de Github Copilot con GPT-4o, como es de esperarse en modelos comerciales, su uso gratuito conlleva restricciones importantes que dificultan un proceso fluido de prueba y error; se suman además las desventajas inherentes a los modelos comerciales, como la falta de transparencia, preocupaciones en torno a la privacidad, opciones limitadas de configuración o creciente dependencia con el proveedor.

RQ2. ¿Qué tan lejos puede estar actualmente el desempeño de un modelo de lenguaje de código abierto en comparación con uno comercial en el desarrollo de aplicaciones web?

En términos generales, puede afirmarse que, aunque están razonablemente próximos, aún existen diferencias entre ellos. Resulta particularmente destacable el desempeño de Llama 3.1, considerando que es un modelo significativamente más pequeño; aunque también ha de reconocerse el de GitHub Copilot y GPT-4o, ya que han logrado un desempeño superior pese a tener que atender a más contexto del necesario.

En los servidores desarrollados, ambos modelos han demostrado comportamientos casi indistinguibles, todas las instrucciones han sido cumplidas satisfactoriamente y no se ha detectado ningún problema relacionado con la calidad del código generado.

En los clientes desarrollados, de las veintiocho instrucciones evaluadas, Llama 3.1 presentó dificultades con cuatro (alrededor del 14.3%) mientras que GitHub Copilot (GPT-4o) lo hizo con solamente una (alrededor del 3.6%); en términos de calidad del código, los resultados son similares para ambos modelos, con un máximo de un problema detectado por cliente, todos ellos de baja importancia y con un impacto mínimo en las aplicaciones web.

RQ3. ¿Cómo influye la complejidad de la aplicación web objetivo para cada modelo de lenguaje seleccionado en la frecuencia de errores de la generación de código?

En base al patrón observado, se considera que el principal factor que afecta al desempeño de los modelos de lenguaje es la cantidad de entidades involucradas en una operación.

En los servidores no parece haber una influencia significativa de esto, ambos modelos han mostrado buenos resultados independientemente de cuántas entidades han de procesarse. Una posible explicación puede deberse a que, a diferencia de los clientes, en los servidores

las operaciones siempre tienden a ser del mismo tipo (variaciones de GET, POST, PUT y DELETE), lo que permite que los LLMs sepan identificarlas y manejarlas mejor.

No obstante, esto sí es notable en los clientes con el cambio de complejidad (pasamos de una instrucción problemática a tres), ya que algunos tipos de operaciones que no presentaban problemas en la primera aplicación web sí lo hacen en la segunda.

Todos los casos en los que uno de los modelos presenta dificultades con alguna de las operaciones se debe a la cantidad razonablemente alta de entidades y atributos que deben procesarse de manera simultánea, lo que implica una mayor atención necesaria al contexto proporcionado y aumenta las probabilidades de alucinaciones u otros errores relacionados.

5

CONCLUSIONES Y LÍNEAS FUTURAS

En este capítulo se reflexiona sobre los hallazgos obtenidos y se exponen las conclusiones finales, además de posibles líneas futuras que se podrían explorar en el futuro.

5.1. CONCLUSIONES

En este trabajo de investigación se ha llevado a cabo un experimento estructurado que permitiera poner a prueba, en un entorno realista y estructurado, dos modelos de lenguaje de diferentes categorías mediante el desarrollo de dos aplicaciones web full-stack de distintas complejidades, estudiando tanto a nivel comparativo como individual sus diferencias de desempeño y calidad.

A la luz de todo el trabajo realizado, se puede dar por satisfecho su propósito, ambos modelos han mostrado resultados prometedores en este tipo de entornos, y la diferencia entre las soluciones comerciales y las de código abierto es notablemente reducida. No obstante, en ambos casos persisten ciertas limitaciones y márgenes de mejora.

Cabe mencionar que, al inicio del proyecto, los conocimientos en materia de investigación eran limitados (más allá de ejercicios puntuales en ciertas asignaturas de la titulación), por lo que el proceso representó una oportunidad de aprendizaje en este ámbito; y aunque ya se contaba con algunos conocimientos previos sobre el desarrollo web con Django y React (Vite), ninguno de los dos asistentes estudiados habían sido utilizados anteriormente.

Sobre los resultados obtenidos, se concluye que los objetivos de investigación han sido alcanzados satisfactoriamente, y que los hallazgos proporcionan una visión actualizada del panorama actual de ambos tipos de LLMs en el contexto del desarrollo web, evaluando las capacidades y limitaciones de Llama 3.1 y GitHub Copilot (GPT-4o).

5.2. LÍNEAS FUTURAS

El estudio combina el análisis de modelos de lenguaje con el desarrollo web full-stack, por lo que abre múltiples posibilidades de expansión en líneas de trabajo futuras, algunas de las cuales se describen a continuación.

- En contraste con el enfoque dado al desarrollo de las aplicaciones web, sería interesante realizar un estudio comparativo similar centrado en el diseño de las mismas (incluyendo interfaces de usuario, entidades de las bases de datos...), dada la naturaleza subjetiva y poco determinista de este apartado.
- Como ya se ha visto en este estudio, el desarrollo de las aplicaciones web ha sido guiado y supervisado por una persona con conocimientos previos en Ingeniería del Software e Inteligencia Artificial; por ello, podría realizarse otro desde la perspectiva de una persona que no sabe programar, y evaluar los modelos de esta forma.
- Se podrían realizar estudios del mismo tipo con otros modelos de lenguaje u otras tecnologías de aplicaciones web, ya sean similares (como los candidatos mencionados en el estado del arte de esta memoria), o actualizar el estudio con modelos más recientes o experimentales.
- Este estudio se ha enfocado en un dominio específico dentro de la Ingeniería del Software, concretamente en el desarrollo web, por lo que existe un amplio margen para futuras investigaciones de este tipo orientadas a la aplicación de LLMs en áreas como las de ciberseguridad o los sistemas empujados.

REFERENCIAS

- [1] Xinyi, H., Yanjie, Z., Yue, L., Zhou, Y., Kailong, W., Li, L., Xiapu, L., David, L., John, G. (ACM Transactions on Software Engineering and Methodology, 2023), Large Language Models for Software Engineering: A Systematic Literature Review.
- [2] Shuang, L., Yuntao, C., Jinfu, C., Jifeng, X., Sen, H., Weiyi, S. (IEEE 35th International Symposium on Software Reliability Engineering, 2024), Assessing the Performance of AI-Generated Code: A Case Study on GitHub Copilot.
- [3] Jesper, S., Yuanyao, Z. (DiVA Dalarna University, 2024), Evaluating the Capabilities of GPT-4 in Full-Stack Web Development.
- [4] Henrique, G., Eduardo, F., Larissa, R., Sarah, N., Fischer, F., Geanderson, E. (SANER Research Papers, 2025), Evaluating the Effectiveness of LLMs in Fixing Maintainability Issues in Real-World Projects.
- [5] Edvin, F., Erik, R. (DiVA Blekinge Institute of Technology, 2023), The Impact of AI-generated Code on Web Development: A Comparative Study of ChatGPT and GitHub Copilot.
- [6] The web framework for perfectionists with deadlines - Django. <https://www.djangoproject.com/> (accedido por última vez el 27/05/2025).
- [7] PostgreSQL: The world's most advanced open source database. <https://www.postgresql.org/> (accedido por última vez el 27/05/2025).
- [8] Getting Started - Vite. <https://vite.dev/guide/> (accedido por última vez el 27/05/2025).
- [9] Visual Studio Code - Code Editing. Redefined. <https://code.visualstudio.com/> (accedido por última vez el 27/05/2025).
- [10] Project Jupyter - Home. <https://jupyter.org/> (accedido por última vez el 27/05/2025).
- [11] PyTorch. <https://pytorch.org/> (accedido por última vez el 27/05/2025).
- [12] Transformers - Hugging Face. <https://huggingface.co/docs/transformers/en/index> (accedido por última vez el 27/05/2025).

- [13] Code Quality, Security & Static Analysis Tool with SonarQube - Sonar. <https://www.sonarsource.com/products/sonarqube/> (accedido por última vez el 27/05/2025).
- [14] Models - Hugging Face. https://huggingface.co/models?pipeline_tag=text-generation&sort=downloads (accedido por última vez el 07/05/2025).
- [15] Aaron, G., Abhimanyu, D., Abhinav, J., Abhinav, P., Abhishek, K., Ahmad, A., Aiesha, L., Akhil, M., et al (551 autores adicionales no mostrados) (arXiv:2407.21783, 2024), The Llama 3 Herd of Models.
- [16] An, Y., Baosong, Y., Beichen, Z., Binyuan, H., Bo, Z., Bowen, Y., Chengyuan, L., Dayiheng, L., et al (34 autores adicionales no mostrados) (arXiv:2412.15115, 2024), Qwen2.5 Technical Report.
- [17] Aishwarya, K., Johan, F., Shreya, P., Nino, V., Ramona, M., Sarah, P., Tatiana, M., Alexandre, R., et al (207 autores adicionales no mostrados) (arXiv:2503.19786, 2025), Gemma 3 Technical Report.
- [18] Peiyuan, Z., Guangtao, Z., Tianduo, W., Wei L. (arXiv:2401.02385, 2024), TinyLlama: An Open-Source Small Language Model.
- [19] Daya, G., Dejian, Y., Haowei, Z., Junxiao, S., Ruoyu, Z., Runxin, X., Qihao, Z., Shirong M., et al (193 autores adicionales no mostrados) (arXiv:2501.12948, 2025), DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning.
- [20] Albert, Q. J., Alexandre, S., Arthur, M., Chris, B., Devendra, S. C., Diego, d. I. C., Florian, B., Gianna, L., et al (10 autores adicionales no mostrados) (arXiv:2407.21783, 2024), Mistral 7B.
- [21] Hello, GPT-4o - OpenAI. <https://openai.com/index/hello-gpt-4o/> (accedido por última vez el 27/05/2025).
- [22] GitHub Copilot - Your AI pair programmer. <https://github.com/features/copilot> (accedido por última vez el 27/05/2025).

- [23] Gemini 2.0 Flash - Generative AI on Vertex.
<https://cloud.google.com/vertex-ai/generative-ai/docs/models/gemini/2-0-flash>
(accedido por última vez el 27/05/2025).
- [24] Claude 3.7 Sonnet - Anthropic. <https://www.anthropic.com/claude/sonnet> (accedido por última vez el 27/05/2025).
- [25] 0610708584/llamaTFM0. <https://github.com/0610708584/llamaTFM0> (accedido por última vez el 27/05/2025).
- [26] 0610708584/llamaTFM1. <https://github.com/0610708584/llamaTFM1> (accedido por última vez el 27/05/2025).
- [27] 0610708584/copilotTFM0. <https://github.com/0610708584/copilotTFM0> (accedido por última vez el 27/05/2025).
- [28] 0610708584/copilotTFM1. <https://github.com/0610708584/copilotTFM1> (accedido por última vez el 27/05/2025).



INFORME DE INSTRUCCIONES

En este apéndice se adjunta el informe que recopila las instrucciones empleadas para cada una de las operaciones, formuladas conforme a la estructura previamente establecida.

En primer lugar, se describen las instrucciones correspondientes a la primera aplicación web, seguidas por las relativas a la segunda; para cada aplicación, se exponen primero las instrucciones del servidor y, posteriormente, las del cliente. Finalmente, cada instrucción es acompañada por una tabla en la que se consignan los resultados obtenidos.

1. TFMØ

1.1. SERVER

S1

Given the following context, could you complete the function "handle_recipes"?

MODELS.PY

```
class Recipe(models.Model):
    title = models.TextField(blank = True)
    text = models.TextField(blank = True)
```

SERIALIZERS.PY

```
class RecipeSerializer(serializers.ModelSerializer):
    class Meta:
        model = Recipe
        fields = ('id', 'title', 'text')
```

URLS.PY

```
urlpatterns = [
    path('recipes/', views.handle_recipes)
]
```

VIEWS.PY

```
@api_view(['GET', 'POST'])
def handle_recipes(request):
```

	DIFICULTAD	NOTAS
LLAMA	fácil.	no ha necesitado intervención.
COPILOT	fácil.	no ha necesitado intervención.

S2

Given the following context, could you complete the function "handle_recipes_by_id"?

MODELS.PY

```
class Recipe(models.Model):
    title = models.TextField()
    text = models.TextField(blank = True)
```

SERIALIZERS.PY

```
class RecipeSerializer(serializers.ModelSerializer):
    class Meta:
        model = Recipe
        fields = ('id', 'title', 'text')
```

URLS.PY

```
urlpatterns = [
    path('recipes/', views.handle_recipes),
    path('recipes/<int:pk>', views.handle_recipes_by_id)
]
```

VIEWS.PY

```
@api_view(['GET', 'POST'])
def handle_recipes(request):
    if request.method == 'GET':
        recipes = Recipe.objects.all()
        serializer = RecipeSerializer(recipes, many=True)
        return Response(serializer.data)
    elif request.method == 'POST':
        serializer = RecipeSerializer(data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data, status=status.HTTP_201_CREATED)
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

@api_view(['GET', 'PUT', 'DELETE'])
def handle_recipes_by_id(request, pk):
```

	DIFICULTAD	NOTAS
LLAMA	fácil.	no ha necesitado intervención.
COPILOT	fácil.	no ha necesitado intervención.

S3

Given the following context, could you complete the function "handle_steps" while also adding an option to query them by the attribute "recipe" (a recipe's ID)?

MODELS.PY

```
class Recipe(models.Model):
    title = models.TextField()
    text = models.TextField(blank = True)

class Step(models.Model):
    number = models.IntegerField()
    text = models.TextField()
    recipe = models.ForeignKey(Recipe, on_delete = models.DO_NOTHING)
```

SERIALIZERS.PY

```
class RecipeSerializer(serializers.ModelSerializer):
    class Meta:
        model = Recipe
        fields = ('id', 'title', 'text')

class StepSerializer(serializers.ModelSerializer):
    class Meta:
        model = Step
        fields = ('id', 'number', 'text', 'recipe')
```

URLS.PY

```
urlpatterns = [
    path('recipes/', views.handle_recipes),
    path('recipes/<int:pk>/', views.handle_recipes_by_id),
    path('steps/', views.handle_steps)
]
```

VIEWS.PY

```
@api_view(['GET', 'POST'])
def handle_recipes(request):
    if request.method == 'GET':
        recipes = Recipe.objects.all()
        serializer = RecipeSerializer(recipes, many=True)
        return Response(serializer.data)
    elif request.method == 'POST':
        serializer = RecipeSerializer(data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data, status=status.HTTP_201_CREATED)
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

@api_view(['GET', 'PUT', 'DELETE'])
def handle_recipes_by_id(request, pk):
    try:
        recipe = Recipe.objects.get(pk=pk)
    except Recipe.DoesNotExist:
        return Response(status=status.HTTP_404_NOT_FOUND)

    if request.method == 'GET':
        serializer = RecipeSerializer(recipe)
        return Response(serializer.data)

    elif request.method == 'PUT':
        serializer = RecipeSerializer(recipe, data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data)
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

    elif request.method == 'DELETE':
        recipe.delete()
        return Response(status=status.HTTP_204_NO_CONTENT)

@api_view(['GET', 'POST'])
def handle_steps(request):
```

	DIFICULTAD	NOTAS
LLAMA	fácil.	no ha necesitado intervención.
COPILOT	fácil.	no ha necesitado intervención.

S4

Given the following context, could you complete the function "handle_steps_by_id"?

MODELS.PY

```

class Recipe(models.Model):
    title = models.TextField()
    text = models.TextField(blank = True)

class Step(models.Model):
    number = models.IntegerField()
    text = models.TextField()
    recipe = models.ForeignKey(Recipe, on_delete = models.DO_NOTHING)

SERIALIZERS.PY

class RecipeSerializer(serializers.ModelSerializer):
    class Meta:
        model = Recipe
        fields = ('id', 'title', 'text')

class StepSerializer(serializers.ModelSerializer):
    class Meta:
        model = Step
        fields = ('id', 'number', 'text', 'recipe')

URLS.PY

urlpatterns = [
    path('recipes/', views.handle_recipes),
    path('recipes/<int:pk>/', views.handle_recipes_by_id),
    path('steps/', views.handle_steps),
    path('steps/<int:pk>/', views.handle_steps_by_id)
]

VIEWS.PY

@api_view(['GET', 'POST'])
def handle_recipes(request):
    if request.method == 'GET':
        recipes = Recipe.objects.all()
        serializer = RecipeSerializer(recipes, many=True)
        return Response(serializer.data)
    elif request.method == 'POST':
        serializer = RecipeSerializer(data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data, status=status.HTTP_201_CREATED)
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

@api_view(['GET', 'PUT', 'DELETE'])
def handle_recipes_by_id(request, pk):
    try:
        recipe = Recipe.objects.get(pk=pk)
    except Recipe.DoesNotExist:
        return Response(status=status.HTTP_404_NOT_FOUND)

    if request.method == 'GET':
        serializer = RecipeSerializer(recipe)
        return Response(serializer.data)

    elif request.method == 'PUT':
        serializer = RecipeSerializer(recipe, data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data)
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

    elif request.method == 'DELETE':
        recipe.delete()
        return Response(status=status.HTTP_204_NO_CONTENT)

@api_view(['GET', 'POST'])
def handle_steps(request):
    if request.method == 'GET':
        if 'recipe_id' in request.GET:
            try:
                recipe_id = int(request.GET['recipe_id'])
                steps = Step.objects.filter(recipe_id=recipe_id)
                serializer = StepSerializer(steps, many=True)
                return Response(serializer.data)
            except (ValueError, Step.DoesNotExist):
                return Response(status=status.HTTP_404_NOT_FOUND)
        else:
            steps = Step.objects.all()
            serializer = StepSerializer(steps, many=True)
            return Response(serializer.data)
    elif request.method == 'POST':
        serializer = StepSerializer(data=request.data)
        if serializer.is_valid():

```

```

        serializer.save()
        return Response(serializer.data, status=status.HTTP_201_CREATED)
    return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

@api_view(['GET', 'PUT', 'DELETE'])
def handle_steps_by_id(request, pk):

```

	DIFICULTAD	NOTAS
LLAMA	fácil.	no ha necesitado intervención.
COPILOT	fácil.	no ha necesitado intervención.

1.2. CLIENT

C1

Given the following context, could you complete the function "listRecipes" using "fetch"?

SERIALIZERS.PY

```

class RecipeSerializer(serializers.ModelSerializer):
    class Meta:
        model = Recipe
        fields = ('id', 'title', 'text')

URLS.PY

urlpatterns = [
    path('recipes/', views.handle_recipes)
]

VIEWS.PY

@api_view(['GET', 'POST'])
def handle_recipes(request):
    if request.method == 'GET':
        recipes = Recipe.objects.all()
        serializer = RecipeSerializer(recipes, many=True)
        return Response(serializer.data)
    elif request.method == 'POST':
        serializer = RecipeSerializer(data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data, status=status.HTTP_201_CREATED)
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

```

APP.TSX

```

const [recipes, setRecipes] = useState<any[]>([]);

useEffect(() => {
    listRecipes();
}, []);

const listRecipes = () => {
};

```

	DIFICULTAD	NOTAS
LLAMA	fácil.	no ha necesitado intervención.
COPILOT	fácil.	no ha necesitado intervención.

C2

Given the following context, could you complete the function "getRecipe" using "fetch"?

It's meant to be called after a button click over a specific recipe, it fills "currentRecipe" and "currentSteps" (this one list ordered by its "number" attribute), it also sets "recipes" to [""], and it must also set the variables "setEditableRecipeTitle" and "setEditableRecipeText" to false.

SERIALIZERS.PY

```

class RecipeSerializer(serializers.ModelSerializer):
    class Meta:
        model = Recipe
        fields = ('id', 'title', 'text')

class StepSerializer(serializers.ModelSerializer):
    class Meta:
        model = Step
        fields = ('id', 'number', 'text', 'recipe')

urlpatterns = [
    path('recipes/', views.handle_recipes),
    path('recipes/', views.handle_recipes_by_id),

```

```

    path('steps/', views.handle_steps)
]

VIEWS.PY

@api_view(['GET', 'POST'])
def handle_recipes(request):
    if request.method == 'GET':
        recipes = Recipe.objects.all()
        serializer = RecipeSerializer(recipes, many=True)
        return Response(serializer.data)
    elif request.method == 'POST':
        serializer = RecipeSerializer(data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data, status=status.HTTP_201_CREATED)
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

@api_view(['GET', 'PUT', 'DELETE'])
def handle_recipes_by_id(request, pk):
    try:
        recipe = Recipe.objects.get(pk=pk)
    except Recipe.DoesNotExist:
        return Response(status=status.HTTP_404_NOT_FOUND)

    if request.method == 'GET':
        serializer = RecipeSerializer(recipe)
        return Response(serializer.data)

    elif request.method == 'PUT':
        serializer = RecipeSerializer(recipe, data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data)
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

    elif request.method == 'DELETE':
        recipe.delete()
        return Response(status=status.HTTP_204_NO_CONTENT)

@api_view(['GET', 'POST'])
def handle_steps(request):
    if request.method == 'GET':
        if 'recipe_id' in request.GET:
            try:
                recipe_id = int(request.GET['recipe_id'])
                steps = Step.objects.filter(recipe_id=recipe_id)
                serializer = StepSerializer(steps, many=True)
                return Response(serializer.data)
            except (ValueError, Step.DoesNotExist):
                return Response(status=status.HTTP_404_NOT_FOUND)
        else:
            steps = Step.objects.all()
            serializer = StepSerializer(steps, many=True)
            return Response(serializer.data)
    elif request.method == 'POST':
        serializer = StepSerializer(data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data, status=status.HTTP_201_CREATED)
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

APP.TSX

const [recipes, setRecipes] = useState<any[]>([]);
const [currentRecipe, setCurrentRecipe] = useState({id: 0, title: "", text: ""});
const [currentSteps, setCurrentSteps] = useState<any[]>([]);

useEffect(() => {
    listRecipes();
}, []);

const listRecipes = async () => {
    try {
        const response = await fetch('http://127.0.0.1:8000/api/recipes/');
        if (!response.ok) {
            throw new Error('Network response was not ok');
        }
        const data = await response.json();
        setRecipes(data);
    } catch (error) {
        console.error('Error fetching recipes:', error);
    }
};

```

```
const getRecipe = () => {
}
```

	DIFICULTAD	NOTAS
LLAMA	fácil.	no ha necesitado intervención.
COPILOT	fácil.	no ha necesitado intervención.

C3

Given the following context, could you complete the function "createRecipe" using "fetch"?

It's meant to be called after a button click, it creates a new recipe with "title" as the string "Nueva receta" and "text" as "...".

SERIALIZERS.PY

```
class RecipeSerializer(serializers.ModelSerializer):
    class Meta:
        model = Recipe
        fields = ('id', 'title', 'text')
```

URLS.PY

```
urlpatterns = [
    path('recipes/', views.handle_recipes)
]
```

VIEWS.PY

```
@api_view(['GET', 'POST'])
def handle_recipes(request):
    if request.method == 'GET':
        recipes = Recipe.objects.all()
        serializer = RecipeSerializer(recipes, many=True)
        return Response(serializer.data)
    elif request.method == 'POST':
        serializer = RecipeSerializer(data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data, status=status.HTTP_201_CREATED)
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)
```

APP.TSX

```
const [recipes, setRecipes] = useState<any[]>([]);

useEffect(() => {
    listRecipes();
}, []);

const listRecipes = async () => {
    try {
        const response = await fetch('http://127.0.0.1:8000/api/recipes/');
        if (!response.ok) {
            throw new Error('Network response was not ok');
        }
        const data = await response.json();
        setRecipes(data);
    } catch (error) {
        console.error('Error fetching recipes:', error);
    }
};

const createRecipe = () => {
}
```

	DIFICULTAD	NOTAS
LLAMA	fácil.	no ha necesitado intervención.
COPILOT	fácil.	no ha necesitado intervención.

C4

Given the following context, could you complete the function "createStep" using "fetch"?

It's meant to be called after a button click, and using "newStepNumber" and "newStepText", must update "currentSteps" and recall "getRecipe" using the "currentRecipe" object.

SERIALIZERS.PY

```
class RecipeSerializer(serializers.ModelSerializer):
    class Meta:
        model = Recipe
        fields = ('id', 'title', 'text')
```

```

class StepSerializer(serializers.ModelSerializer):
    class Meta:
        model = Step
        fields = ('id', 'number', 'text', 'recipe')

urlpatterns = [
    path('recipes/', views.handle_recipes_by_id),
    path('steps/', views.handle_steps)
]

```

VIEWS.PY

```

@api_view(['GET', 'PUT', 'DELETE'])
def handle_recipes_by_id(request, pk):
    try:
        recipe = Recipe.objects.get(pk=pk)
    except Recipe.DoesNotExist:
        return Response(status=status.HTTP_404_NOT_FOUND)

    if request.method == 'GET':
        serializer = RecipeSerializer(recipe)
        return Response(serializer.data)

    elif request.method == 'PUT':
        serializer = RecipeSerializer(recipe, data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data)
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

    elif request.method == 'DELETE':
        recipe.delete()
        return Response(status=status.HTTP_204_NO_CONTENT)

@api_view(['GET', 'POST'])
def handle_steps(request):
    if request.method == 'GET':
        if 'recipe_id' in request.GET:
            try:
                recipe_id = int(request.GET['recipe_id'])
                steps = Step.objects.filter(recipe_id=recipe_id)
                serializer = StepSerializer(steps, many=True)
                return Response(serializer.data)
            except (ValueError, Step.DoesNotExist):
                return Response(status=status.HTTP_404_NOT_FOUND)
        else:
            steps = Step.objects.all()
            serializer = StepSerializer(steps, many=True)
            return Response(serializer.data)
    elif request.method == 'POST':
        serializer = StepSerializer(data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data, status=status.HTTP_201_CREATED)
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

```

APP.TSX

```

const [recipes, setRecipes] = useState<any[]>([]);
const [currentRecipe, setCurrentRecipe] = useState({id: 0, title: "", text: ""});
const [currentSteps, setCurrentSteps] = useState<any[]>([]);
const [newStepNumber, setNewStepNumber] = useState(0);
const [newStepText, setNewStepText] = useState<string>();

const getRecipe = async (recipeId: number) => {
    try {
        // Set recipes to an empty array
        setRecipes([]);

        // Fetch the recipe by ID
        const response = await fetch(`http://127.0.0.1:8000/api/recipes/${recipeId}/`);
        if (!response.ok) {
            throw new Error('Network response was not ok');
        }
        const recipeData = await response.json();

        // Fetch the steps for the recipe
        const stepsResponse = await fetch(`http://127.0.0.1:8000/api/steps/?recipe_id=${recipeId}`);
        if (!stepsResponse.ok) {
            throw new Error('Network response was not ok');
        }
        const stepsData = await stepsResponse.json();

        // Sort the steps by number

```

```

        const sortedSteps = stepsData.sort((a: { [x: string]: number; }, b: { [x: string]: number; }) =>
a["number"] - b["number"]);

        // Set the current recipe and steps
        setCurrentRecipe(recipeData);
        setCurrentSteps(sortedSteps);
    } catch (error) {
        console.error('Error fetching recipe:', error);
    }
};

const createRecipe = () => {
}

```

	DIFICULTAD	NOTAS
LLAMA	fácil.	no ha necesitado intervención.
COPILOT	fácil.	no ha necesitado intervención.

C5

Given the following context, could you complete the function "editRecipeTitle" using "fetch"?

It's meant to be called after a button click over a specific recipe, it edits the field "title" and leaves "text" with the same value, and at the end recall "getRecipe" using the "currentRecipe" object to refresh.

SERIALIZERS.PY

```

class RecipeSerializer(serializers.ModelSerializer):
    class Meta:
        model = Recipe
        fields = ('id', 'title', 'text')

class StepSerializer(serializers.ModelSerializer):
    class Meta:
        model = Step
        fields = ('id', 'number', 'text', 'recipe')

```

URLS.PY

```

urlpatterns = [
    path('recipes/', views.handle_recipes),
    path('recipes/', views.handle_recipes_by_id),
    path('steps/', views.handle_steps)
]

```

VIEWS.PY

```

@api_view(['GET', 'POST'])
def handle_recipes(request):
    if request.method == 'GET':
        recipes = Recipe.objects.all()
        serializer = RecipeSerializer(recipes, many=True)
        return Response(serializer.data)
    elif request.method == 'POST':
        serializer = RecipeSerializer(data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data, status=status.HTTP_201_CREATED)
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

@api_view(['GET', 'PUT', 'DELETE'])
def handle_recipes_by_id(request, pk):
    try:
        recipe = Recipe.objects.get(pk=pk)
    except Recipe.DoesNotExist:
        return Response(status=status.HTTP_404_NOT_FOUND)

    if request.method == 'GET':
        serializer = RecipeSerializer(recipe)
        return Response(serializer.data)

    elif request.method == 'PUT':
        serializer = RecipeSerializer(recipe, data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data)
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

    elif request.method == 'DELETE':
        recipe.delete()
        return Response(status=status.HTTP_204_NO_CONTENT)

@api_view(['GET', 'POST'])
def handle_steps(request):

```

```
if request.method == 'GET':
    if 'recipe_id' in request.GET:
        try:
            recipe_id = int(request.GET['recipe_id'])
            steps = Step.objects.filter(recipe_id=recipe_id)
            serializer = StepSerializer(steps, many=True)
            return Response(serializer.data)
        except (ValueError, Step.DoesNotExist):
            return Response(status=status.HTTP_404_NOT_FOUND)
    else:
        steps = Step.objects.all()
        serializer = StepSerializer(steps, many=True)
        return Response(serializer.data)
elif request.method == 'POST':
    serializer = StepSerializer(data=request.data)
    if serializer.is_valid():
        serializer.save()
        return Response(serializer.data, status=status.HTTP_201_CREATED)
    return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

APP.TSX

const [recipes, setRecipes] = useState<any[]>([]);
const [currentRecipe, setCurrentRecipe] = useState<{id: 0, title: "", text: ""}>();
const [currentSteps, setCurrentSteps] = useState<any[]>([]);
const [newRecipeTitle, setNewRecipeTitle] = useState<string>();

const getRecipe = async (recipeId: number) => {
    try {
        // Set recipes to an empty array
        setRecipes([]);

        // Fetch the recipe by ID
        const response = await fetch(`http://127.0.0.1:8000/api/recipes/${recipeId}/`);
        if (!response.ok) {
            throw new Error('Network response was not ok');
        }
        const recipeData = await response.json();

        // Fetch the steps for the recipe
        const stepsResponse = await fetch(`http://127.0.0.1:8000/api/steps/?recipe_id=${recipeId}`);
        if (!stepsResponse.ok) {
            throw new Error('Network response was not ok');
        }
        const stepsData = await stepsResponse.json();

        // Sort the steps by number
        const sortedSteps = stepsData.sort((a: { [x: string]: number; }, b: { [x: string]: number; }) =>
a["number"] - b["number"]);

        // Set the current recipe and steps
        setCurrentRecipe(recipeData);
        setCurrentSteps(sortedSteps);
    } catch (error) {
        console.error('Error fetching recipe:', error);
    }
};

const editRecipeTitle = () => {
}
```

	DIFICULTAD	NOTAS
LLAMA	fácil.	no ha necesitado intervención.
COPILOT	fácil.	no ha necesitado intervención.

C6

Given the following context, could you complete the function "editRecipeText" using "fetch"?

It's meant to be called after a button click over a specific recipe, it edits the field "text" and leaves "title" with the same value, and at the end recall "getRecipe" using the "currentRecipe" object to refresh.

SERIALIZERS.PY

```
class RecipeSerializer(serializers.ModelSerializer):
    class Meta:
        model = Recipe
        fields = ('id', 'title', 'text')

class StepSerializer(serializers.ModelSerializer):
    class Meta:
        model = Step
        fields = ('id', 'number', 'text', 'recipe')
```


URLS.PY

```
urlpatterns = [
    path('recipes/', views.handle_recipes),
    path('recipes/', views.handle_recipes_by_id),
    path('steps/', views.handle_steps)
]
```

VIEWS.PY

```
@api_view(['GET', 'POST'])
def handle_recipes(request):
    if request.method == 'GET':
        recipes = Recipe.objects.all()
        serializer = RecipeSerializer(recipes, many=True)
        return Response(serializer.data)
    elif request.method == 'POST':
        serializer = RecipeSerializer(data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data, status=status.HTTP_201_CREATED)
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)
```

```
@api_view(['GET', 'PUT', 'DELETE'])
def handle_recipes_by_id(request, pk):
    try:
        recipe = Recipe.objects.get(pk=pk)
    except Recipe.DoesNotExist:
        return Response(status=status.HTTP_404_NOT_FOUND)

    if request.method == 'GET':
        serializer = RecipeSerializer(recipe)
        return Response(serializer.data)

    elif request.method == 'PUT':
        serializer = RecipeSerializer(recipe, data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data)
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

    elif request.method == 'DELETE':
        recipe.delete()
        return Response(status=status.HTTP_204_NO_CONTENT)
```

```
@api_view(['GET', 'POST'])
def handle_steps(request):
    if request.method == 'GET':
        if 'recipe_id' in request.GET:
            try:
                recipe_id = int(request.GET['recipe_id'])
                steps = Step.objects.filter(recipe_id=recipe_id)
                serializer = StepSerializer(steps, many=True)
                return Response(serializer.data)
            except (ValueError, Step.DoesNotExist):
                return Response(status=status.HTTP_404_NOT_FOUND)
        else:
            steps = Step.objects.all()
            serializer = StepSerializer(steps, many=True)
            return Response(serializer.data)
    elif request.method == 'POST':
        serializer = StepSerializer(data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data, status=status.HTTP_201_CREATED)
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)
```

APP.TSX

```
const [recipes, setRecipes] = useState<any[]>([]);
const [currentRecipe, setCurrentRecipe] = useState<{id: 0, title: '', text: ''}>();
const [currentSteps, setCurrentSteps] = useState<any[]>([]);
const [newRecipeTitle, setNewRecipeTitle] = useState<string>();
const [newRecipeText, setNewRecipeText] = useState<string>();

const getRecipe = async (recipeId: number) => {
    try {
        // Set recipes to an empty array
        setRecipes([]);

        // Fetch the recipe by ID
        const response = await fetch(`http://127.0.0.1:8000/api/recipes/${recipeId}/`);
        if (!response.ok) {
            throw new Error('Network response was not ok');
        }
    }
}
```

```

const recipeData = await response.json();

// Fetch the steps for the recipe
const stepsResponse = await fetch(`http://127.0.0.1:8000/api/steps/?recipe_id=${recipeId}`);
if (!stepsResponse.ok) {
  throw new Error('Network response was not ok');
}
const stepsData = await stepsResponse.json();

// Sort the steps by number
const sortedSteps = stepsData.sort((a: { [x: string]: number; }, b: { [x: string]: number; }) =>
a["number"] - b["number"]);

// Set the current recipe and steps
setCurrentRecipe(recipeData);
setCurrentSteps(sortedSteps);
} catch (error) {
  console.error('Error fetching recipe:', error);
}
};

const editRecipeTitle = async () => {
  try {
    // Get the current recipe ID and title
    const currentRecipeId = currentRecipe.id;
    const newTitle = newRecipeTitle;

    // Fetch the updated recipe data
    const response = await fetch(`http://127.0.0.1:8000/api/recipes/${currentRecipeId}/`, {
      method: 'PUT',
      headers: {
        'Content-Type': 'application/json',
      },
      body: JSON.stringify({
        title: newTitle,
        text: currentRecipe.text, // Leave the text field unchanged
      }),
    });

    if (!response.ok) {
      throw new Error('Network response was not ok');
    }

    // Get the updated recipe data
    const updatedRecipeData = await response.json();

    // Update the current recipe and refresh the data
    setCurrentRecipe(updatedRecipeData);
    getRecipe(currentRecipeId);
  } catch (error) {
    console.error('Error editing recipe title:', error);
  }
};

const editRecipeText = async () => {
}

```

	DIFICULTAD	NOTAS
LLAMA	fácil.	no ha necesitado intervención.
COPILOT	fácil.	no ha necesitado intervención.

CZ

Given the following context, could you complete the function "deleteRecipe" using "fetch"?

It's meant to be called after a button click over a specific recipe (with the recipe's id given as a parameter) and refreshes with "listRecipes".

More specifically, it first removes each step with "recipe" as the recipe's ID, then removes the recipe itself and calls "listRecipes()".

To delete the steps, you have to use `http://127.0.0.1:8000/api/steps/?recipe\_id=\${recipeId}` in a loop; only as a GET, not as a DELETE.

SERIALIZERS.PY

```

class RecipeSerializer(serializers.ModelSerializer):
    class Meta:
        model = Recipe
        fields = ('id', 'title', 'text')

```

```

class StepSerializer(serializers.ModelSerializer):
    class Meta:
        model = Step

```

```

        fields = ('id', 'number', 'text', 'recipe')

URLS.PY

urlpatterns = [
    path('recipes/', views.handle_recipes),
    path('recipes/<int:pk>/', views.handle_recipes_by_id),
    path('steps/', views.handle_steps),
    path('steps/<int:pk>/', views.handle_steps_by_id)
]

VIEWS.PY

@api_view(['GET', 'POST'])
def handle_recipes(request):
    if request.method == 'GET':
        recipes = Recipe.objects.all()
        serializer = RecipeSerializer(recipes, many=True)
        return Response(serializer.data)
    elif request.method == 'POST':
        serializer = RecipeSerializer(data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data, status=status.HTTP_201_CREATED)
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

@api_view(['GET', 'PUT', 'DELETE'])
def handle_recipes_by_id(request, pk):
    try:
        recipe = Recipe.objects.get(pk=pk)
    except Recipe.DoesNotExist:
        return Response(status=status.HTTP_404_NOT_FOUND)

    if request.method == 'GET':
        serializer = RecipeSerializer(recipe)
        return Response(serializer.data)

    elif request.method == 'PUT':
        serializer = RecipeSerializer(recipe, data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data)
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

    elif request.method == 'DELETE':
        recipe.delete()
        return Response(status=status.HTTP_204_NO_CONTENT)

@api_view(['GET', 'POST'])
def handle_steps(request):
    if request.method == 'GET':
        if 'recipe_id' in request.GET:
            try:
                recipe_id = int(request.GET['recipe_id'])
                steps = Step.objects.filter(recipe_id=recipe_id)
                serializer = StepSerializer(steps, many=True)
                return Response(serializer.data)
            except (ValueError, Step.DoesNotExist):
                return Response(status=status.HTTP_404_NOT_FOUND)
        else:
            steps = Step.objects.all()
            serializer = StepSerializer(steps, many=True)
            return Response(serializer.data)
    elif request.method == 'POST':
        serializer = StepSerializer(data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data, status=status.HTTP_201_CREATED)
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

@api_view(['GET', 'PUT', 'DELETE'])
def handle_steps_by_id(request, pk):
    try:
        step = Step.objects.get(pk=pk)
    except Step.DoesNotExist:
        return Response(status=status.HTTP_404_NOT_FOUND)

    if request.method == 'GET':
        serializer = StepSerializer(step)
        return Response(serializer.data)

    elif request.method == 'PUT':
        serializer = StepSerializer(step, data=request.data)
        if serializer.is_valid():
            serializer.save()

```

```

        return Response(serializer.data)
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

    elif request.method == 'DELETE':
        step.delete()
        return Response(status=status.HTTP_204_NO_CONTENT)
APP.TSX
const [recipes, setRecipes] = useState<any[]>([]);

useEffect(() => {
    listRecipes();
}, []);

const listRecipes = async () => {
    try {
        const response = await fetch('http://127.0.0.1:8000/api/recipes/');
        if (!response.ok) {
            throw new Error('Network response was not ok');
        }
        const data = await response.json();
        setRecipes(data);
    } catch (error) {
        console.error('Error fetching recipes:', error);
    }
};

const deleteRecipe = () => {
}

```

	DIFICULTAD	NOTAS
LLAMA	medio.	no ha necesitado intervención; pero si no se especifica el procedimiento, el modelo se inventa las peticiones al servidor ignorando el contexto proporcionado.
COPILOT	fácil.	no ha necesitado intervención.

C8

Given the following context, could you complete the function "deleteStep" using "fetch"?

It's meant to be called after a button click over a specific step, and at the end recall "getRecipe" using the "currentRecipe" object to refresh.

SERIALIZERS.PY

```

class RecipeSerializer(serializers.ModelSerializer):
    class Meta:
        model = Recipe
        fields = ('id', 'title', 'text')

class StepSerializer(serializers.ModelSerializer):
    class Meta:
        model = Step
        fields = ('id', 'number', 'text', 'recipe')

```

URLS.PY

```

urlpatterns = [
    path('recipes/', views.handle_recipes),
    path('recipes/<int:pk>/', views.handle_recipes_by_id),
    path('steps/', views.handle_steps),
    path('steps/<int:pk>/', views.handle_steps_by_id)
]

```

VIEWS.PY

```

@api_view(['GET', 'POST'])
def handle_recipes(request):
    if request.method == 'GET':
        recipes = Recipe.objects.all()
        serializer = RecipeSerializer(recipes, many=True)
        return Response(serializer.data)
    elif request.method == 'POST':
        serializer = RecipeSerializer(data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data, status=status.HTTP_201_CREATED)
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

@api_view(['GET', 'PUT', 'DELETE'])
def handle_recipes_by_id(request, pk):
    try:
        recipe = Recipe.objects.get(pk=pk)
    except Recipe.DoesNotExist:

```

```

        return Response(status=status.HTTP_404_NOT_FOUND)

    if request.method == 'GET':
        serializer = RecipeSerializer(recipe)
        return Response(serializer.data)

    elif request.method == 'PUT':
        serializer = RecipeSerializer(recipe, data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data)
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

    elif request.method == 'DELETE':
        recipe.delete()
        return Response(status=status.HTTP_204_NO_CONTENT)

@api_view(['GET', 'POST'])
def handle_steps(request):
    if request.method == 'GET':
        if 'recipe_id' in request.GET:
            try:
                recipe_id = int(request.GET['recipe_id'])
                steps = Step.objects.filter(recipe_id=recipe_id)
                serializer = StepSerializer(steps, many=True)
                return Response(serializer.data)
            except (ValueError, Step.DoesNotExist):
                return Response(status=status.HTTP_404_NOT_FOUND)
        else:
            steps = Step.objects.all()
            serializer = StepSerializer(steps, many=True)
            return Response(serializer.data)
    elif request.method == 'POST':
        serializer = StepSerializer(data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data, status=status.HTTP_201_CREATED)
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

@api_view(['GET', 'PUT', 'DELETE'])
def handle_steps_by_id(request, pk):
    try:
        step = Step.objects.get(pk=pk)
    except Step.DoesNotExist:
        return Response(status=status.HTTP_404_NOT_FOUND)

    if request.method == 'GET':
        serializer = StepSerializer(step)
        return Response(serializer.data)

    elif request.method == 'PUT':
        serializer = StepSerializer(step, data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data)
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

    elif request.method == 'DELETE':
        step.delete()
        return Response(status=status.HTTP_204_NO_CONTENT)

```

APP.TSX

```

const [recipes, setRecipes] = useState<any[]>([]);
const [currentRecipe, setCurrentRecipe] = useState({id: 0, title: "", text: ""});
const [currentSteps, setCurrentSteps] = useState<any[]>([]);

const getRecipe = async (recipeId: number) => {
    try {
        // Set recipes to an empty array
        setRecipes([]);

        // Fetch the recipe by ID
        const response = await fetch(`http://127.0.0.1:8000/api/recipes/${recipeId}/`);
        if (!response.ok) {
            throw new Error('Network response was not ok');
        }
        const recipeData = await response.json();

        // Fetch the steps for the recipe
        const stepsResponse = await fetch(`http://127.0.0.1:8000/api/steps/?recipe_id=${recipeId}`);
        if (!stepsResponse.ok) {
            throw new Error('Network response was not ok');
        }
        const stepsData = await stepsResponse.json();
    }

```

```
        // Sort the steps by number
        const sortedSteps = stepsData.sort((a: { [x: string]: number; }, b: { [x: string]: number; }) =>
a["number"] - b["number"]);

        // Set the current recipe and steps
        setCurrentRecipe(recipeData);
        setCurrentSteps(sortedSteps);
    } catch (error) {
        console.error('Error fetching recipe:', error);
    }
};

const deleteStep = async () => {
}
```

	DIFICULTAD	NOTAS
LLAMA	fácil.	no ha necesitado intervención.
COPILOT	fácil.	no ha necesitado intervención.

2. TFM1

2.1. SERVER

S1

```
Given the following context, could you complete the function "handle_users" while also adding an option to query them by the attribute "username"?

MODELS.PY

class User(models.Model):
    username = models.TextField()
    password = models.TextField()

SERIALIZERS.PY

class UserSerializer(serializers.ModelSerializer):
    class Meta:
        model = User
        fields = ('id', 'username', 'password')

URLS.PY

urlpatterns = [
    path('users/', views.handle_users)
]

VIEWS.PY

@api_view(['GET', 'POST'])
def handle_users(request):
```

	DIFICULTAD	NOTAS
LLAMA	fácil.	no ha necesitado intervención.
COPILOT	fácil.	no ha necesitado intervención.

S2

```
Given the following context, could you complete the function "handle_users_by_id"?

MODELS.PY

class User(models.Model):
    username = models.TextField()
    password = models.TextField()

SERIALIZERS.PY

class UserSerializer(serializers.ModelSerializer):
    class Meta:
        model = User
        fields = ('id', 'username', 'password')

URLS.PY

urlpatterns = [
    path('users/', views.handle_users),
    path('users/<int:pk>/', views.handle_users_by_id)
]

VIEWS.PY
```

```

@api_view(['GET', 'POST'])
def handle_users(request):
    """
    Handles user CRUD operations.

    GET: Retrieves all users.
    POST: Creates a new user.
    """

    # Check if the request is a GET request
    if request.method == 'GET':
        # Get the query parameter 'username' from the URL
        username = request.GET.get('username')

        # If 'username' is provided, filter users by the 'username' attribute
        if username:
            users = User.objects.filter(username=username)
        else:
            # Get all users
            users = User.objects.all()

        # Serialize the users
        serializer = UserSerializer(users, many=True)

        # Return the serialized users
        return Response(serializer.data)

    # Check if the request is a POST request
    elif request.method == 'POST':
        # Create a new serializer instance
        serializer = UserSerializer(data=request.data)

        # Check if the serializer is valid
        if serializer.is_valid():
            # Save the new user
            serializer.save()

            # Return a success message
            return Response(serializer.data, status=status.HTTP_201_CREATED)
        else:
            # Return an error message
            return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

@api_view(['GET', 'PUT', 'DELETE'])
def handle_users_by_id(request, pk):

```

	DIFICULTAD	NOTAS
LLAMA	fácil.	no ha necesitado intervención.
COPILOT	fácil.	no ha necesitado intervención.

S3

Given the following context, could you complete the function "handle_reviews" while also adding an option to query them by the attributes "reviewer" (an users's ID) and "reviewed" (an users's ID)?

MODELS.PY

```

class User(models.Model):
    username = models.TextField()
    password = models.TextField()

class Review(models.Model):
    grade = models.IntegerField()
    text = models.TextField()
    reviewer = models.ForeignKey(User, on_delete = models.DO_NOTHING, related_name = "reviews_written")
    reviewed = models.ForeignKey(User, on_delete = models.DO_NOTHING, related_name = "reviews_received")

```

SERIALIZERS.PY

```

class UserSerializer(serializers.ModelSerializer):
    class Meta:
        model = User
        fields = ('id', 'username', 'password')

class ReviewSerializer(serializers.ModelSerializer):
    class Meta:
        model = Review
        fields = ('id', 'grade', 'text', 'reviewer', 'reviewed')

```

URLS.PY

```

urlpatterns = [
    path('users/', views.handle_users),
    path('users/<int:pk>/', views.handle_users_by_id),

```

```

    path('reviews/', views.handle_reviews)
]

VIEWS.PY

@api_view(['GET', 'POST'])
def handle_users(request):
    """
    Handles user CRUD operations.

    GET: Retrieves all users.
    POST: Creates a new user.
    """

    # Check if the request is a GET request
    if request.method == 'GET':
        # Get the query parameter 'username' from the URL
        username = request.GET.get('username')

        # If 'username' is provided, filter users by the 'username' attribute
        if username:
            users = User.objects.filter(username=username)
        else:
            # Get all users
            users = User.objects.all()

        # Serialize the users
        serializer = UserSerializer(users, many=True)

        # Return the serialized users
        return Response(serializer.data)

    # Check if the request is a POST request
    elif request.method == 'POST':
        # Create a new serializer instance
        serializer = UserSerializer(data=request.data)

        # Check if the serializer is valid
        if serializer.is_valid():
            # Save the new user
            serializer.save()

            # Return a success message
            return Response(serializer.data, status=status.HTTP_201_CREATED)
        else:
            # Return an error message
            return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

@api_view(['GET', 'PUT', 'DELETE'])
def handle_users_by_id(request, pk):
    """
    Handles user CRUD operations by ID.

    GET: Retrieves a user by ID.
    PUT: Updates a user by ID.
    DELETE: Deletes a user by ID.
    """

    # Try to get the user by ID
    try:
        user = User.objects.get(pk=pk)
    except User.DoesNotExist:
        # Return a 404 error if the user does not exist
        return Response({'error': 'User not found'}, status=status.HTTP_404_NOT_FOUND)

    # Check if the request is a GET request
    if request.method == 'GET':
        # Serialize the user
        serializer = UserSerializer(user)

        # Return the serialized user
        return Response(serializer.data)

    # Check if the request is a PUT request
    elif request.method == 'PUT':
        # Create a new serializer instance
        serializer = UserSerializer(user, data=request.data)

        # Check if the serializer is valid
        if serializer.is_valid():
            # Save the updated user
            serializer.save()

            # Return a success message
            return Response(serializer.data)

```



```

        else:
            # Return an error message
            return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

    # Check if the request is a DELETE request
    elif request.method == 'DELETE':
        # Delete the user
        user.delete()

        # Return a success message
        return Response({'message': 'User deleted successfully'}, status=status.HTTP_204_NO_CONTENT)

@api_view(['GET', 'POST'])
def handle_reviews(request):

```

	DIFICULTAD	NOTAS
LLAMA	fácil.	no ha necesitado intervención.
COPILOT	fácil.	no ha necesitado intervención.

S4

Given the following context, could you complete the function "handle_reviews_by_id"?

MODELS.PY

```

class User(models.Model):
    username = models.TextField()
    password = models.TextField()

class Review(models.Model):
    grade = models.IntegerField()
    text = models.TextField()
    reviewer = models.ForeignKey(User, on_delete = models.DO_NOTHING, related_name = "reviews_written")
    reviewed = models.ForeignKey(User, on_delete = models.DO_NOTHING, related_name = "reviews_received")

```

SERIALIZERS.PY

```

class UserSerializer(serializers.ModelSerializer):
    class Meta:
        model = User
        fields = ('id', 'username', 'password')

class ReviewSerializer(serializers.ModelSerializer):
    class Meta:
        model = Review
        fields = ('id', 'grade', 'text', 'reviewer', 'reviewed')

```

URLS.PY

```

urlpatterns = [
    path('users/', views.handle_users),
    path('users/<int:pk>/', views.handle_users_by_id),
    path('reviews/', views.handle_reviews),
    path('reviews/<int:pk>/', views.handle_reviews_by_id)
]

```

VIEWS.PY

```

@api_view(['GET', 'POST'])
def handle_users(request):
    """
    Handles user CRUD operations.

    GET: Retrieves all users.
    POST: Creates a new user.
    """

    # Check if the request is a GET request
    if request.method == 'GET':
        # Get the query parameter 'username' from the URL
        username = request.GET.get('username')

        # If 'username' is provided, filter users by the 'username' attribute
        if username:
            users = User.objects.filter(username=username)
        else:
            # Get all users
            users = User.objects.all()

        # Serialize the users
        serializer = UserSerializer(users, many=True)

        # Return the serialized users
        return Response(serializer.data)

```

```

# Check if the request is a POST request
elif request.method == 'POST':
    # Create a new serializer instance
    serializer = UserSerializer(data=request.data)

    # Check if the serializer is valid
    if serializer.is_valid():
        # Save the new user
        serializer.save()

        # Return a success message
        return Response(serializer.data, status=status.HTTP_201_CREATED)
    else:
        # Return an error message
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

@api_view(['GET', 'PUT', 'DELETE'])
def handle_users_by_id(request, pk):
    """
    Handles user CRUD operations by ID.

    GET: Retrieves a user by ID.
    PUT: Updates a user by ID.
    DELETE: Deletes a user by ID.
    """

    # Try to get the user by ID
    try:
        user = User.objects.get(pk=pk)
    except User.DoesNotExist:
        # Return a 404 error if the user does not exist
        return Response({'error': 'User not found'}, status=status.HTTP_404_NOT_FOUND)

    # Check if the request is a GET request
    if request.method == 'GET':
        # Serialize the user
        serializer = UserSerializer(user)

        # Return the serialized user
        return Response(serializer.data)

    # Check if the request is a PUT request
    elif request.method == 'PUT':
        # Create a new serializer instance
        serializer = UserSerializer(user, data=request.data)

        # Check if the serializer is valid
        if serializer.is_valid():
            # Save the updated user
            serializer.save()

            # Return a success message
            return Response(serializer.data)
        else:
            # Return an error message
            return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

    # Check if the request is a DELETE request
    elif request.method == 'DELETE':
        # Delete the user
        user.delete()

        # Return a success message
        return Response({'message': 'User deleted successfully'}, status=status.HTTP_204_NO_CONTENT)

@api_view(['GET', 'POST'])
def handle_reviews(request):
    """
    Handles review CRUD operations.

    GET: Retrieves all reviews.
    POST: Creates a new review.
    """

    # Check if the request is a GET request
    if request.method == 'GET':
        # Get the query parameter 'reviewed' from the URL
        reviewed = request.GET.get('reviewed')

        # If 'reviewed' is provided, filter reviews by the 'reviewed' attribute
        if reviewed:
            try:
                reviewed_id = int(reviewed)
                reviews = Review.objects.filter(reviewed_id=reviewed_id)
            except ValueError:
                # Invalid integer value for 'reviewed'
                return Response({'error': 'Invalid reviewed id'}, status=status.HTTP_400_BAD_REQUEST)
        else:
            # Return all reviews
            reviews = Review.objects.all()

        # Return the reviews
        return Response(reviews.data, status=status.HTTP_200_OK)

    # Check if the request is a POST request
    elif request.method == 'POST':
        # Create a new review
        review = Review.objects.create(**request.data)

        # Return the created review
        return Response(review.data, status=status.HTTP_201_CREATED)

```

```

        except ValueError:
            reviews = Review.objects.filter(Q(reviewed__username=reviewed) | Q(reviewed__id=reviewed))
        else:
            # Get all reviews
            reviews = Review.objects.all()

            # Serialize the reviews
            serializer = ReviewSerializer(reviews, many=True)

            # Return the serialized reviews
            return Response(serializer.data)

    # Check if the request is a POST request
    elif request.method == 'POST':
        # Create a new serializer instance
        serializer = ReviewSerializer(data=request.data)

        # Check if the serializer is valid
        if serializer.is_valid():
            # Save the new review
            serializer.save()

            # Return a success message
            return Response(serializer.data, status=status.HTTP_201_CREATED)
        else:
            # Return an error message
            return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

@api_view(['GET', 'PUT', 'DELETE'])
def handle_reviews_by_id(request, pk):

```

	DIFICULTAD	NOTAS
LLAMA	fácil.	no ha necesitado intervención.
COPILOT	fácil.	no ha necesitado intervención.

S5

Given the following context, could you complete the function "handle_houses" while also adding an option to query them by the attributes "user" (an user's ID) and "country"?

MODELS.PY

```

class User(models.Model):
    username = models.TextField()
    password = models.TextField()

class Review(models.Model):
    grade = models.IntegerField()
    text = models.TextField()
    reviewer = models.ForeignKey(User, on_delete = models.DO_NOTHING, related_name = "reviews_written")
    reviewed = models.ForeignKey(User, on_delete = models.DO_NOTHING, related_name = "reviews_received")

class House(models.Model):
    user = models.ForeignKey(User, on_delete = models.DO_NOTHING)
    title = models.TextField()
    text = models.TextField()
    image = models.TextField()
    price = models.IntegerField()
    country = models.TextField()

```

SERIALIZERS.PY

```

class UserSerializer(serializers.ModelSerializer):
    class Meta:
        model = User
        fields = ('id', 'username', 'password')

class ReviewSerializer(serializers.ModelSerializer):
    class Meta:
        model = Review
        fields = ('id', 'grade', 'text', 'reviewer', 'reviewed')

class HouseSerializer(serializers.ModelSerializer):
    class Meta:
        model = House
        fields = ('id', 'title', 'text', 'image', 'price', 'country', 'user')

```

URLS.PY

```

urlpatterns = [
    path('users/', views.handle_users),
    path('users/<int:pk>/', views.handle_users_by_id),
    path('reviews/', views.handle_reviews),
    path('reviews/<int:pk>/', views.handle_reviews_by_id),

```

```

    path('houses/', views.handle_houses)
]

VIEWS.PY

@api_view(['GET', 'POST'])
def handle_users(request):
    """
    Handles user CRUD operations.

    GET: Retrieves all users.
    POST: Creates a new user.
    """

    # Check if the request is a GET request
    if request.method == 'GET':
        # Get the query parameter 'username' from the URL
        username = request.GET.get('username')

        # If 'username' is provided, filter users by the 'username' attribute
        if username:
            users = User.objects.filter(username=username)
        else:
            # Get all users
            users = User.objects.all()

        # Serialize the users
        serializer = UserSerializer(users, many=True)

        # Return the serialized users
        return Response(serializer.data)

    # Check if the request is a POST request
    elif request.method == 'POST':
        # Create a new serializer instance
        serializer = UserSerializer(data=request.data)

        # Check if the serializer is valid
        if serializer.is_valid():
            # Save the new user
            serializer.save()

            # Return a success message
            return Response(serializer.data, status=status.HTTP_201_CREATED)
        else:
            # Return an error message
            return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

@api_view(['GET', 'PUT', 'DELETE'])
def handle_users_by_id(request, pk):
    """
    Handles user CRUD operations by ID.

    GET: Retrieves a user by ID.
    PUT: Updates a user by ID.
    DELETE: Deletes a user by ID.
    """

    # Try to get the user by ID
    try:
        user = User.objects.get(pk=pk)
    except User.DoesNotExist:
        # Return a 404 error if the user does not exist
        return Response({'error': 'User not found'}, status=status.HTTP_404_NOT_FOUND)

    # Check if the request is a GET request
    if request.method == 'GET':
        # Serialize the user
        serializer = UserSerializer(user)

        # Return the serialized user
        return Response(serializer.data)

    # Check if the request is a PUT request
    elif request.method == 'PUT':
        # Create a new serializer instance
        serializer = UserSerializer(user, data=request.data)

        # Check if the serializer is valid
        if serializer.is_valid():
            # Save the updated user
            serializer.save()

            # Return a success message
            return Response(serializer.data)

```

```

        else:
            # Return an error message
            return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

# Check if the request is a DELETE request
elif request.method == 'DELETE':
    # Delete the user
    user.delete()

    # Return a success message
    return Response({'message': 'User deleted successfully'}, status=status.HTTP_204_NO_CONTENT)

@api_view(['GET', 'POST'])
def handle_reviews(request):
    """
    Handles review CRUD operations.

    GET: Retrieves all reviews.
    POST: Creates a new review.
    """

    # Check if the request is a GET request
    if request.method == 'GET':
        # Get the query parameter 'reviewed' from the URL
        reviewed = request.GET.get('reviewed')

        # If 'reviewed' is provided, filter reviews by the 'reviewed' attribute
        if reviewed:
            try:
                reviewed_id = int(reviewed)
                reviews = Review.objects.filter(reviewed_id=reviewed_id)
            except ValueError:
                reviews = Review.objects.filter(Q(reviewed__username=reviewed) | Q(reviewed__id=reviewed))
        else:
            # Get all reviews
            reviews = Review.objects.all()

        # Serialize the reviews
        serializer = ReviewSerializer(reviews, many=True)

        # Return the serialized reviews
        return Response(serializer.data)

    # Check if the request is a POST request
    elif request.method == 'POST':
        # Create a new serializer instance
        serializer = ReviewSerializer(data=request.data)

        # Check if the serializer is valid
        if serializer.is_valid():
            # Save the new review
            serializer.save()

            # Return a success message
            return Response(serializer.data, status=status.HTTP_201_CREATED)
        else:
            # Return an error message
            return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

@api_view(['GET', 'PUT', 'DELETE'])
def handle_reviews_by_id(request, pk):
    """
    Handles review CRUD operations by ID.

    GET: Retrieves a review by ID.
    PUT: Updates a review by ID.
    DELETE: Deletes a review by ID.
    """

    # Try to get the review by ID
    try:
        review = Review.objects.get(pk=pk)
    except Review.DoesNotExist:
        # Return a 404 error if the review does not exist
        return Response({'error': 'Review not found'}, status=status.HTTP_404_NOT_FOUND)

    # Check if the request is a GET request
    if request.method == 'GET':
        # Serialize the review
        serializer = ReviewSerializer(review)

        # Return the serialized review
        return Response(serializer.data)

    # Check if the request is a PUT request

```

```

elif request.method == 'PUT':
    # Create a new serializer instance
    serializer = ReviewSerializer(review, data=request.data)

    # Check if the serializer is valid
    if serializer.is_valid():
        # Save the updated review
        serializer.save()

        # Return a success message
        return Response(serializer.data)
    else:
        # Return an error message
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

# Check if the request is a DELETE request
elif request.method == 'DELETE':
    # Delete the review
    review.delete()

    # Return a success message
    return Response({'message': 'Review deleted successfully'}, status=status.HTTP_204_NO_CONTENT)

@api_view(['GET', 'POST'])
def handle_houses(request):

```

	DIFICULTAD	NOTAS
LLAMA	fácil.	no ha necesitado intervención.
COPILOT	fácil.	no ha necesitado intervención.

S6

Given the following context, could you complete the function "handle_houses_by_id"?

MODELS.PY

```

class User(models.Model):
    username = models.TextField()
    password = models.TextField()

class Review(models.Model):
    grade = models.IntegerField()
    text = models.TextField()
    reviewer = models.ForeignKey(User, on_delete = models.DO_NOTHING, related_name = "reviews_written")
    reviewed = models.ForeignKey(User, on_delete = models.DO_NOTHING, related_name = "reviews_received")

class House(models.Model):
    user = models.ForeignKey(User, on_delete = models.DO_NOTHING)
    title = models.TextField()
    text = models.TextField()
    image = models.TextField()
    price = models.IntegerField()
    country = models.TextField()

```

SERIALIZERS.PY

```

class UserSerializer(serializers.ModelSerializer):
    class Meta:
        model = User
        fields = ('id', 'username', 'password')

class ReviewSerializer(serializers.ModelSerializer):
    class Meta:
        model = Review
        fields = ('id', 'grade', 'text', 'reviewer', 'reviewed')

class HouseSerializer(serializers.ModelSerializer):
    class Meta:
        model = House
        fields = ('id', 'user', 'title', 'text', 'image', 'price', 'country')

```

URLS.PY

```

urlpatterns = [
    path('users/', views.handle_users),
    path('users/<int:pk>/', views.handle_users_by_id),
    path('reviews/', views.handle_reviews),
    path('reviews/<int:pk>/', views.handle_reviews_by_id),
    path('houses/', views.handle_houses),
    path('houses/<int:pk>/', views.handle_houses_by_id)
]

```

VIEWS.PY

```

@api_view(['GET', 'POST'])
def handle_users(request):
    """
    Handles user CRUD operations.

    GET: Retrieves all users.
    POST: Creates a new user.
    """

    # Check if the request is a GET request
    if request.method == 'GET':
        # Get the query parameter 'username' from the URL
        username = request.GET.get('username')

        # If 'username' is provided, filter users by the 'username' attribute
        if username:
            users = User.objects.filter(username=username)
        else:
            # Get all users
            users = User.objects.all()

        # Serialize the users
        serializer = UserSerializer(users, many=True)

        # Return the serialized users
        return Response(serializer.data)

    # Check if the request is a POST request
    elif request.method == 'POST':
        # Create a new serializer instance
        serializer = UserSerializer(data=request.data)

        # Check if the serializer is valid
        if serializer.is_valid():
            # Save the new user
            serializer.save()

            # Return a success message
            return Response(serializer.data, status=status.HTTP_201_CREATED)
        else:
            # Return an error message
            return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

@api_view(['GET', 'PUT', 'DELETE'])
def handle_users_by_id(request, pk):
    """
    Handles user CRUD operations by ID.

    GET: Retrieves a user by ID.
    PUT: Updates a user by ID.
    DELETE: Deletes a user by ID.
    """

    # Try to get the user by ID
    try:
        user = User.objects.get(pk=pk)
    except User.DoesNotExist:
        # Return a 404 error if the user does not exist
        return Response({'error': 'User not found'}, status=status.HTTP_404_NOT_FOUND)

    # Check if the request is a GET request
    if request.method == 'GET':
        # Serialize the user
        serializer = UserSerializer(user)

        # Return the serialized user
        return Response(serializer.data)

    # Check if the request is a PUT request
    elif request.method == 'PUT':
        # Create a new serializer instance
        serializer = UserSerializer(user, data=request.data)

        # Check if the serializer is valid
        if serializer.is_valid():
            # Save the updated user
            serializer.save()

            # Return a success message
            return Response(serializer.data)
        else:
            # Return an error message
            return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

    # Check if the request is a DELETE request

```

```

elif request.method == 'DELETE':
    # Delete the user
    user.delete()

    # Return a success message
    return Response({'message': 'User deleted successfully'}, status=status.HTTP_204_NO_CONTENT)

@api_view(['GET', 'POST'])
def handle_reviews(request):
    """
    Handles review CRUD operations.

    GET: Retrieves all reviews.
    POST: Creates a new review.
    """

    # Check if the request is a GET request
    if request.method == 'GET':
        # Get the query parameter 'reviewed' from the URL
        reviewed = request.GET.get('reviewed')

        # If 'reviewed' is provided, filter reviews by the 'reviewed' attribute
        if reviewed:
            try:
                reviewed_id = int(reviewed)
                reviews = Review.objects.filter(reviewed_id=reviewed_id)
            except ValueError:
                reviews = Review.objects.filter(Q(reviewed__username=reviewed) | Q(reviewed__id=reviewed))
        else:
            # Get all reviews
            reviews = Review.objects.all()

        # Serialize the reviews
        serializer = ReviewSerializer(reviews, many=True)

        # Return the serialized reviews
        return Response(serializer.data)

    # Check if the request is a POST request
    elif request.method == 'POST':
        # Create a new serializer instance
        serializer = ReviewSerializer(data=request.data)

        # Check if the serializer is valid
        if serializer.is_valid():
            # Save the new review
            serializer.save()

            # Return a success message
            return Response(serializer.data, status=status.HTTP_201_CREATED)
        else:
            # Return an error message
            return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

@api_view(['GET', 'PUT', 'DELETE'])
def handle_reviews_by_id(request, pk):
    """
    Handles review CRUD operations by ID.

    GET: Retrieves a review by ID.
    PUT: Updates a review by ID.
    DELETE: Deletes a review by ID.
    """

    # Try to get the review by ID
    try:
        review = Review.objects.get(pk=pk)
    except Review.DoesNotExist:
        # Return a 404 error if the review does not exist
        return Response({'error': 'Review not found'}, status=status.HTTP_404_NOT_FOUND)

    # Check if the request is a GET request
    if request.method == 'GET':
        # Serialize the review
        serializer = ReviewSerializer(review)

        # Return the serialized review
        return Response(serializer.data)

    # Check if the request is a PUT request
    elif request.method == 'PUT':
        # Create a new serializer instance
        serializer = ReviewSerializer(review, data=request.data)

        # Check if the serializer is valid

```



```

        if serializer.is_valid():
            # Save the updated review
            serializer.save()

            # Return a success message
            return Response(serializer.data)
        else:
            # Return an error message
            return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

# Check if the request is a DELETE request
elif request.method == 'DELETE':
    # Delete the review
    review.delete()

    # Return a success message
    return Response({'message': 'Review deleted successfully'}, status=status.HTTP_204_NO_CONTENT)

@api_view(['GET', 'POST'])
def handle_houses(request):
    """
    Handles house CRUD operations.

    GET: Retrieves all houses.
    POST: Creates a new house.
    """

    # Check if the request is a GET request
    if request.method == 'GET':
        # Get the query parameters 'user' and 'country' from the URL
        user = request.GET.get('user')
        country = request.GET.get('country')

        # If 'user' is provided, filter houses by the 'user' attribute
        if user:
            try:
                user_id = int(user)
                houses = House.objects.filter(user_id=user_id)
            except ValueError:
                houses = House.objects.filter(Q(user__username=user) | Q(user__id=user))
        # If 'country' is provided, filter houses by the 'country' attribute
        elif country:
            houses = House.objects.filter(country=country)
        else:
            # Get all houses
            houses = House.objects.all()

        # Serialize the houses
        serializer = HouseSerializer(houses, many=True)

        # Return the serialized houses
        return Response(serializer.data)

    # Check if the request is a POST request
    elif request.method == 'POST':
        # Create a new serializer instance
        serializer = HouseSerializer(data=request.data)

        # Check if the serializer is valid
        if serializer.is_valid():
            # Save the new house
            serializer.save()

            # Return a success message
            return Response(serializer.data, status=status.HTTP_201_CREATED)
        else:
            # Return an error message
            return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

@api_view(['GET', 'PUT', 'DELETE'])
def handle_houses_by_id(request, pk):

```

	DIFICULTAD	NOTAS
LLAMA	fácil.	no ha necesitado intervención.
COPILOT	fácil.	no ha necesitado intervención.

SZ

Given the following context, could you complete the function "handle_orders" while also adding an option to query them by the attributes "user"(an user's ID) and "house" (a house's ID)?

MODELS.PY

```
class User(models.Model):
```

```

username = models.TextField()
password = models.TextField()

class Review(models.Model):
    grade = models.IntegerField()
    text = models.TextField()
    reviewer = models.ForeignKey(User, on_delete = models.DO_NOTHING, related_name = "reviews_written")
    reviewed = models.ForeignKey(User, on_delete = models.DO_NOTHING, related_name = "reviews_received")

class House(models.Model):
    user = models.ForeignKey(User, on_delete = models.DO_NOTHING)
    title = models.TextField()
    text = models.TextField()
    image = models.TextField()
    price = models.IntegerField()
    country = models.TextField()

class Order(models.Model):
    text = models.TextField()
    status = models.TextField()
    user = models.ForeignKey(User, on_delete = models.DO_NOTHING)
    house = models.ForeignKey(House, on_delete = models.DO_NOTHING)

SERIALIZERS.PY

class UserSerializer(serializers.ModelSerializer):
    class Meta:
        model = User
        fields = ('id', 'username', 'password')

class ReviewSerializer(serializers.ModelSerializer):
    class Meta:
        model = Review
        fields = ('id', 'grade', 'text', 'reviewer', 'reviewed')

class HouseSerializer(serializers.ModelSerializer):
    class Meta:
        model = House
        fields = ('id', 'user', 'title', 'text', 'image', 'price', 'country')

class OrderSerializer(serializers.ModelSerializer):
    class Meta:
        model = Order
        fields = ('id', 'text', 'status', 'user', 'house')

URLS.PY

urlpatterns = [
    path('users/', views.handle_users),
    path('users/<int:pk>/', views.handle_users_by_id),
    path('reviews/', views.handle_reviews),
    path('reviews/<int:pk>/', views.handle_reviews_by_id),
    path('houses/', views.handle_houses),
    path('houses/<int:pk>/', views.handle_houses_by_id),
    path('orders/', views.handle_orders)
]

VIEWS.PY

@api_view(['GET', 'POST'])
def handle_users(request):
    """
    Handles user CRUD operations.

    GET: Retrieves all users.
    POST: Creates a new user.
    """

    # Check if the request is a GET request
    if request.method == 'GET':
        # Get the query parameter 'username' from the URL
        username = request.GET.get('username')

        # If 'username' is provided, filter users by the 'username' attribute
        if username:
            users = User.objects.filter(username=username)
        else:
            # Get all users
            users = User.objects.all()

        # Serialize the users
        serializer = UserSerializer(users, many=True)

        # Return the serialized users
        return Response(serializer.data)

```

```

# Check if the request is a POST request
elif request.method == 'POST':
    # Create a new serializer instance
    serializer = UserSerializer(data=request.data)

    # Check if the serializer is valid
    if serializer.is_valid():
        # Save the new user
        serializer.save()

        # Return a success message
        return Response(serializer.data, status=status.HTTP_201_CREATED)
    else:
        # Return an error message
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

@api_view(['GET', 'PUT', 'DELETE'])
def handle_users_by_id(request, pk):
    """
    Handles user CRUD operations by ID.

    GET: Retrieves a user by ID.
    PUT: Updates a user by ID.
    DELETE: Deletes a user by ID.
    """

    # Try to get the user by ID
    try:
        user = User.objects.get(pk=pk)
    except User.DoesNotExist:
        # Return a 404 error if the user does not exist
        return Response({'error': 'User not found'}, status=status.HTTP_404_NOT_FOUND)

    # Check if the request is a GET request
    if request.method == 'GET':
        # Serialize the user
        serializer = UserSerializer(user)

        # Return the serialized user
        return Response(serializer.data)

    # Check if the request is a PUT request
    elif request.method == 'PUT':
        # Create a new serializer instance
        serializer = UserSerializer(user, data=request.data)

        # Check if the serializer is valid
        if serializer.is_valid():
            # Save the updated user
            serializer.save()

            # Return a success message
            return Response(serializer.data)
        else:
            # Return an error message
            return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

    # Check if the request is a DELETE request
    elif request.method == 'DELETE':
        # Delete the user
        user.delete()

        # Return a success message
        return Response({'message': 'User deleted successfully'}, status=status.HTTP_204_NO_CONTENT)

@api_view(['GET', 'POST'])
def handle_reviews(request):
    """
    Handles review CRUD operations.

    GET: Retrieves all reviews.
    POST: Creates a new review.
    """

    # Check if the request is a GET request
    if request.method == 'GET':
        # Get the query parameter 'reviewed' from the URL
        reviewed = request.GET.get('reviewed')

        # If 'reviewed' is provided, filter reviews by the 'reviewed' attribute
        if reviewed:
            try:
                reviewed_id = int(reviewed)
                reviews = Review.objects.filter(reviewed_id=reviewed_id)

```

```

        except ValueError:
            reviews = Review.objects.filter(Q(reviewed__username=reviewed) | Q(reviewed__id=reviewed))
        else:
            # Get all reviews
            reviews = Review.objects.all()

            # Serialize the reviews
            serializer = ReviewSerializer(reviews, many=True)

            # Return the serialized reviews
            return Response(serializer.data)

    # Check if the request is a POST request
    elif request.method == 'POST':
        # Create a new serializer instance
        serializer = ReviewSerializer(data=request.data)

        # Check if the serializer is valid
        if serializer.is_valid():
            # Save the new review
            serializer.save()

            # Return a success message
            return Response(serializer.data, status=status.HTTP_201_CREATED)
        else:
            # Return an error message
            return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

@api_view(['GET', 'PUT', 'DELETE'])
def handle_reviews_by_id(request, pk):
    """
    Handles review CRUD operations by ID.

    GET: Retrieves a review by ID.
    PUT: Updates a review by ID.
    DELETE: Deletes a review by ID.
    """

    # Try to get the review by ID
    try:
        review = Review.objects.get(pk=pk)
    except Review.DoesNotExist:
        # Return a 404 error if the review does not exist
        return Response({'error': 'Review not found'}, status=status.HTTP_404_NOT_FOUND)

    # Check if the request is a GET request
    if request.method == 'GET':
        # Serialize the review
        serializer = ReviewSerializer(review)

        # Return the serialized review
        return Response(serializer.data)

    # Check if the request is a PUT request
    elif request.method == 'PUT':
        # Create a new serializer instance
        serializer = ReviewSerializer(review, data=request.data)

        # Check if the serializer is valid
        if serializer.is_valid():
            # Save the updated review
            serializer.save()

            # Return a success message
            return Response(serializer.data)
        else:
            # Return an error message
            return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

    # Check if the request is a DELETE request
    elif request.method == 'DELETE':
        # Delete the review
        review.delete()

        # Return a success message
        return Response({'message': 'Review deleted successfully'}, status=status.HTTP_204_NO_CONTENT)

@api_view(['GET', 'POST'])
def handle_houses(request):
    """
    Handles house CRUD operations.

    GET: Retrieves all houses.
    POST: Creates a new house.
    """

```

```

# Check if the request is a GET request
if request.method == 'GET':
    # Get the query parameters 'user' and 'country' from the URL
    user = request.GET.get('user')
    country = request.GET.get('country')

    # If 'user' is provided, filter houses by the 'user' attribute
    if user:
        try:
            user_id = int(user)
            houses = House.objects.filter(user_id=user_id)
        except ValueError:
            houses = House.objects.filter(Q(user__username=user) | Q(user__id=user))
    # If 'country' is provided, filter houses by the 'country' attribute
    elif country:
        houses = House.objects.filter(country=country)
    else:
        # Get all houses
        houses = House.objects.all()

    # Serialize the houses
    serializer = HouseSerializer(houses, many=True)

    # Return the serialized houses
    return Response(serializer.data)

# Check if the request is a POST request
elif request.method == 'POST':
    # Create a new serializer instance
    serializer = HouseSerializer(data=request.data)

    # Check if the serializer is valid
    if serializer.is_valid():
        # Save the new house
        serializer.save()

        # Return a success message
        return Response(serializer.data, status=status.HTTP_201_CREATED)
    else:
        # Return an error message
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

@api_view(['GET', 'PUT', 'DELETE'])
def handle_houses_by_id(request, pk):
    """
    Handles house CRUD operations by ID.

    GET: Retrieves a house by ID.
    PUT: Updates a house by ID.
    DELETE: Deletes a house by ID.
    """

    # Try to get the house by ID
    try:
        house = House.objects.get(pk=pk)
    except House.DoesNotExist:
        # Return a 404 error if the house does not exist
        return Response({'error': 'House not found'}, status=status.HTTP_404_NOT_FOUND)

    # Check if the request is a GET request
    if request.method == 'GET':
        # Serialize the house
        serializer = HouseSerializer(house)

        # Return the serialized house
        return Response(serializer.data)

    # Check if the request is a PUT request
    elif request.method == 'PUT':
        # Create a new serializer instance
        serializer = HouseSerializer(house, data=request.data)

        # Check if the serializer is valid
        if serializer.is_valid():
            # Save the updated house
            serializer.save()

            # Return a success message
            return Response(serializer.data)
        else:
            # Return an error message
            return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

    # Check if the request is a DELETE request

```

```

        elif request.method == 'DELETE':
            # Delete the house
            house.delete()

            # Return a success message
            return Response({'message': 'House deleted successfully'}, status=status.HTTP_204_NO_CONTENT)

@api_view(['GET', 'POST'])
def handle_orders(request):

```

	DIFICULTAD	NOTAS
LLAMA	fácil.	no ha necesitado intervención.
COPILOT	fácil.	no ha necesitado intervención.

S8

Given the following context, could you complete the function "handle_orders_by_id"?

MODELS.PY

```

class User(models.Model):
    username = models.TextField()
    password = models.TextField()

class Review(models.Model):
    grade = models.IntegerField()
    text = models.TextField()
    reviewer = models.ForeignKey(User, on_delete = models.DO_NOTHING, related_name = "reviews_written")
    reviewed = models.ForeignKey(User, on_delete = models.DO_NOTHING, related_name = "reviews_received")

class House(models.Model):
    user = models.ForeignKey(User, on_delete = models.DO_NOTHING)
    title = models.TextField()
    text = models.TextField()
    image = models.TextField()
    price = models.IntegerField()
    country = models.TextField()

class Order(models.Model):
    text = models.TextField()
    status = models.TextField()
    user = models.ForeignKey(User, on_delete = models.DO_NOTHING)
    house = models.ForeignKey(House, on_delete = models.DO_NOTHING)

```

SERIALIZERS.PY

```

class UserSerializer(serializers.ModelSerializer):
    class Meta:
        model = User
        fields = ('id', 'username', 'password')

class ReviewSerializer(serializers.ModelSerializer):
    class Meta:
        model = Review
        fields = ('id', 'grade', 'text', 'reviewer', 'reviewed')

class HouseSerializer(serializers.ModelSerializer):
    class Meta:
        model = House
        fields = ('id', 'user', 'title', 'text', 'image', 'price', 'country')

class OrderSerializer(serializers.ModelSerializer):
    class Meta:
        model = Order
        fields = ('id', 'text', 'status', 'user', 'house')

```

URLS.PY

```

urlpatterns = [
    path('users/', views.handle_users),
    path('users/<int:pk>/', views.handle_users_by_id),
    path('reviews/', views.handle_reviews),
    path('reviews/<int:pk>/', views.handle_reviews_by_id),
    path('houses/', views.handle_houses),
    path('houses/<int:pk>/', views.handle_houses_by_id),
    path('orders/', views.handle_orders),
    path('orders/<int:pk>/', views.handle_orders_by_id)
]

```

VIEWS.PY

```

@api_view(['GET', 'POST'])
def handle_users(request):
    """

```

```

Handles user CRUD operations.

GET: Retrieves all users.
POST: Creates a new user.
"""

# Check if the request is a GET request
if request.method == 'GET':
    # Get the query parameter 'username' from the URL
    username = request.GET.get('username')

    # If 'username' is provided, filter users by the 'username' attribute
    if username:
        users = User.objects.filter(username=username)
    else:
        # Get all users
        users = User.objects.all()

    # Serialize the users
    serializer = UserSerializer(users, many=True)

    # Return the serialized users
    return Response(serializer.data)

# Check if the request is a POST request
elif request.method == 'POST':
    # Create a new serializer instance
    serializer = UserSerializer(data=request.data)

    # Check if the serializer is valid
    if serializer.is_valid():
        # Save the new user
        serializer.save()

        # Return a success message
        return Response(serializer.data, status=status.HTTP_201_CREATED)
    else:
        # Return an error message
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

@api_view(['GET', 'PUT', 'DELETE'])
def handle_users_by_id(request, pk):
    """
    Handles user CRUD operations by ID.

    GET: Retrieves a user by ID.
    PUT: Updates a user by ID.
    DELETE: Deletes a user by ID.
    """

    # Try to get the user by ID
    try:
        user = User.objects.get(pk=pk)
    except User.DoesNotExist:
        # Return a 404 error if the user does not exist
        return Response({'error': 'User not found'}, status=status.HTTP_404_NOT_FOUND)

    # Check if the request is a GET request
    if request.method == 'GET':
        # Serialize the user
        serializer = UserSerializer(user)

        # Return the serialized user
        return Response(serializer.data)

    # Check if the request is a PUT request
    elif request.method == 'PUT':
        # Create a new serializer instance
        serializer = UserSerializer(user, data=request.data)

        # Check if the serializer is valid
        if serializer.is_valid():
            # Save the updated user
            serializer.save()

            # Return a success message
            return Response(serializer.data)
        else:
            # Return an error message
            return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

    # Check if the request is a DELETE request
    elif request.method == 'DELETE':
        # Delete the user
        user.delete()

```

```

        # Return a success message
        return Response({'message': 'User deleted successfully'}, status=status.HTTP_204_NO_CONTENT)

@api_view(['GET', 'POST'])
def handle_reviews(request):
    """
    Handles review CRUD operations.

    GET: Retrieves all reviews.
    POST: Creates a new review.
    """

    # Check if the request is a GET request
    if request.method == 'GET':
        # Get the query parameter 'reviewed' from the URL
        reviewed = request.GET.get('reviewed')

        # If 'reviewed' is provided, filter reviews by the 'reviewed' attribute
        if reviewed:
            try:
                reviewed_id = int(reviewed)
                reviews = Review.objects.filter(reviewed_id=reviewed_id)
            except ValueError:
                reviews = Review.objects.filter(Q(reviewed__username=reviewed) | Q(reviewed__id=reviewed))
        else:
            # Get all reviews
            reviews = Review.objects.all()

        # Serialize the reviews
        serializer = ReviewSerializer(reviews, many=True)

        # Return the serialized reviews
        return Response(serializer.data)

    # Check if the request is a POST request
    elif request.method == 'POST':
        # Create a new serializer instance
        serializer = ReviewSerializer(data=request.data)

        # Check if the serializer is valid
        if serializer.is_valid():
            # Save the new review
            serializer.save()

            # Return a success message
            return Response(serializer.data, status=status.HTTP_201_CREATED)
        else:
            # Return an error message
            return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

@api_view(['GET', 'PUT', 'DELETE'])
def handle_reviews_by_id(request, pk):
    """
    Handles review CRUD operations by ID.

    GET: Retrieves a review by ID.
    PUT: Updates a review by ID.
    DELETE: Deletes a review by ID.
    """

    # Try to get the review by ID
    try:
        review = Review.objects.get(pk=pk)
    except Review.DoesNotExist:
        # Return a 404 error if the review does not exist
        return Response({'error': 'Review not found'}, status=status.HTTP_404_NOT_FOUND)

    # Check if the request is a GET request
    if request.method == 'GET':
        # Serialize the review
        serializer = ReviewSerializer(review)

        # Return the serialized review
        return Response(serializer.data)

    # Check if the request is a PUT request
    elif request.method == 'PUT':
        # Create a new serializer instance
        serializer = ReviewSerializer(review, data=request.data)

        # Check if the serializer is valid
        if serializer.is_valid():
            # Save the updated review
            serializer.save()

```



```

        # Return a success message
        return Response(serializer.data)
    else:
        # Return an error message
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

# Check if the request is a DELETE request
elif request.method == 'DELETE':
    # Delete the review
    review.delete()

    # Return a success message
    return Response({'message': 'Review deleted successfully'}, status=status.HTTP_204_NO_CONTENT)

@api_view(['GET', 'POST'])
def handle_houses(request):
    """
    Handles house CRUD operations.

    GET: Retrieves all houses.
    POST: Creates a new house.
    """

    # Check if the request is a GET request
    if request.method == 'GET':
        # Get the query parameters 'user' and 'country' from the URL
        user = request.GET.get('user')
        country = request.GET.get('country')

        # If 'user' is provided, filter houses by the 'user' attribute
        if user:
            try:
                user_id = int(user)
                houses = House.objects.filter(user_id=user_id)
            except ValueError:
                houses = House.objects.filter(Q(user__username=user) | Q(user__id=user))
        # If 'country' is provided, filter houses by the 'country' attribute
        elif country:
            houses = House.objects.filter(country=country)
        else:
            # Get all houses
            houses = House.objects.all()

        # Serialize the houses
        serializer = HouseSerializer(houses, many=True)

        # Return the serialized houses
        return Response(serializer.data)

    # Check if the request is a POST request
    elif request.method == 'POST':
        # Create a new serializer instance
        serializer = HouseSerializer(data=request.data)

        # Check if the serializer is valid
        if serializer.is_valid():
            # Save the new house
            serializer.save()

            # Return a success message
            return Response(serializer.data, status=status.HTTP_201_CREATED)
        else:
            # Return an error message
            return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

@api_view(['GET', 'PUT', 'DELETE'])
def handle_houses_by_id(request, pk):
    """
    Handles house CRUD operations by ID.

    GET: Retrieves a house by ID.
    PUT: Updates a house by ID.
    DELETE: Deletes a house by ID.
    """

    # Try to get the house by ID
    try:
        house = House.objects.get(pk=pk)
    except House.DoesNotExist:
        # Return a 404 error if the house does not exist
        return Response({'error': 'House not found'}, status=status.HTTP_404_NOT_FOUND)

    # Check if the request is a GET request
    if request.method == 'GET':

```

```

        # Serialize the house
        serializer = HouseSerializer(house)

        # Return the serialized house
        return Response(serializer.data)

    # Check if the request is a PUT request
    elif request.method == 'PUT':
        # Create a new serializer instance
        serializer = HouseSerializer(house, data=request.data)

        # Check if the serializer is valid
        if serializer.is_valid():
            # Save the updated house
            serializer.save()

            # Return a success message
            return Response(serializer.data)
        else:
            # Return an error message
            return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

    # Check if the request is a DELETE request
    elif request.method == 'DELETE':
        # Delete the house
        house.delete()

        # Return a success message
        return Response({'message': 'House deleted successfully'}, status=status.HTTP_204_NO_CONTENT)

@api_view(['GET', 'POST'])
def handle_orders(request):
    """
    Handles order CRUD operations.

    GET: Retrieves all orders.
    POST: Creates a new order.
    """

    # Check if the request is a GET request
    if request.method == 'GET':
        # Get the query parameters 'user' and 'house' from the URL
        user = request.GET.get('user')
        house = request.GET.get('house')

        # If 'user' is provided, filter orders by the 'user' attribute
        if user:
            try:
                user_id = int(user)
                orders = Order.objects.filter(user_id=user_id)
            except ValueError:
                orders = Order.objects.filter(Q(user__username=user) | Q(user__id=user))
        # If 'house' is provided, filter orders by the 'house' attribute
        elif house:
            try:
                house_id = int(house)
                orders = Order.objects.filter(house_id=house_id)
            except ValueError:
                orders = Order.objects.filter(Q(house__title=house) | Q(house__id=house))
        else:
            # Get all orders
            orders = Order.objects.all()

        # Serialize the orders
        serializer = OrderSerializer(orders, many=True)

        # Return the serialized orders
        return Response(serializer.data)

    # Check if the request is a POST request
    elif request.method == 'POST':
        # Create a new serializer instance
        serializer = OrderSerializer(data=request.data)

        # Check if the serializer is valid
        if serializer.is_valid():
            # Save the new order
            serializer.save()

            # Return a success message
            return Response(serializer.data, status=status.HTTP_201_CREATED)
        else:
            # Return an error message
            return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

```

```
@api_view(['GET', 'PUT', 'DELETE'])
def handle_orders_by_id(request, pk):
```

	DIFICULTAD	NOTAS
LLAMA	fácil.	no ha necesitado intervención.
COPILOT	fácil.	no ha necesitado intervención.

2.2. CLIENT

C1

Given the following context, could you complete the function "signIn" using "fetch"?

It's meant to be called after a button click, it checks the entered values "username" and "password"; if it's a correct user, it fills "currentUser" and sets page to "browse".

SERIALIZERS.PY

```
class UserSerializer(serializers.ModelSerializer):
    class Meta:
        model = User
        fields = ('id', 'username', 'password')
```

URLS.PY

```
urlpatterns = [
    path('users/', views.handle_users)
]
```

VIEWS.PY

```
@api_view(['GET', 'POST'])
def handle_users(request):
    """
    Handles user CRUD operations.

    GET: Retrieves all users.
    POST: Creates a new user.
    """

    # Check if the request is a GET request
    if request.method == 'GET':
        # Get the query parameter 'username' from the URL
        username = request.GET.get('username')

        # If 'username' is provided, filter users by the 'username' attribute
        if username:
            users = User.objects.filter(username=username)
        else:
            # Get all users
            users = User.objects.all()

        # Serialize the users
        serializer = UserSerializer(users, many=True)

        # Return the serialized users
        return Response(serializer.data)

    # Check if the request is a POST request
    elif request.method == 'POST':
        # Create a new serializer instance
        serializer = UserSerializer(data=request.data)

        # Check if the serializer is valid
        if serializer.is_valid():
            # Save the new user
            serializer.save()

            # Return a success message
            return Response(serializer.data, status=status.HTTP_201_CREATED)
        else:
            # Return an error message
            return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)
```

APP.TSX

```
const [username, setUsername] = useState<string>();
const [password, setPassword] = useState<string>();
const [currentUser, setCurrentUser] = useState({id: 0, username: "", password: ""});

const signIn = () => {
```

| DIFICULTAD | NOTAS

LLAMA	fácil.	no ha necesitado intervención.
COPILLOT	fácil.	no ha necesitado intervención.

C2

Given the following context, could you complete the function "getUser" using "fetch"?

It's meant to be called after a button click over a specific user; given its id, it fills "currentUser" and "currentReviews". Also, for each review in "currentReviews", use "reviewer" id to get its username and put it inside another field "reviewer_username" of the review. At the end, call "setEditableUserUsername(false)" and "setPage('profile')".

SERIALIZERS.PY

```
class UserSerializer(serializers.ModelSerializer):
    class Meta:
        model = User
        fields = ('id', 'username', 'password')

class ReviewSerializer(serializers.ModelSerializer):
    class Meta:
        model = Review
        fields = ('id', 'grade', 'text', 'reviewer', 'reviewed')
```

URLS.PY

```
urlpatterns = [
    path('users/<int:pk>/', views.handle_users_by_id),
    path('reviews/', views.handle_reviews)
]
```

VIEWS.PY

```
@api_view(['GET', 'PUT', 'DELETE'])
def handle_users_by_id(request, pk):
    """
    Handles user CRUD operations by ID.

    GET: Retrieves a user by ID.
    PUT: Updates a user by ID.
    DELETE: Deletes a user by ID.
    """

    # Try to get the user by ID
    try:
        user = User.objects.get(pk=pk)
    except User.DoesNotExist:
        # Return a 404 error if the user does not exist
        return Response({'error': 'User not found'}, status=status.HTTP_404_NOT_FOUND)

    # Check if the request is a GET request
    if request.method == 'GET':
        # Serialize the user
        serializer = UserSerializer(user)

        # Return the serialized user
        return Response(serializer.data)

    # Check if the request is a PUT request
    elif request.method == 'PUT':
        # Create a new serializer instance
        serializer = UserSerializer(user, data=request.data)

        # Check if the serializer is valid
        if serializer.is_valid():
            # Save the updated user
            serializer.save()

            # Return a success message
            return Response(serializer.data)
        else:
            # Return an error message
            return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

    # Check if the request is a DELETE request
    elif request.method == 'DELETE':
        # Delete the user
        user.delete()

        # Return a success message
        return Response({'message': 'User deleted successfully'}, status=status.HTTP_204_NO_CONTENT)

@api_view(['GET', 'POST'])
def handle_reviews(request):
```

```
"""
Handles review CRUD operations.

GET: Retrieves all reviews.
POST: Creates a new review.
"""

# Check if the request is a GET request
if request.method == 'GET':
    # Get the query parameter 'reviewed' from the URL
    reviewed = request.GET.get('reviewed')

    # If 'reviewed' is provided, filter reviews by the 'reviewed' attribute
    if reviewed:
        try:
            reviewed_id = int(reviewed)
            reviews = Review.objects.filter(reviewed_id=reviewed_id)
        except ValueError:
            reviews = Review.objects.filter(Q(reviewed__username=reviewed) | Q(reviewed__id=reviewed))
    else:
        # Get all reviews
        reviews = Review.objects.all()

    # Serialize the reviews
    serializer = ReviewSerializer(reviews, many=True)

    # Return the serialized reviews
    return Response(serializer.data)

# Check if the request is a POST request
elif request.method == 'POST':
    # Create a new serializer instance
    serializer = ReviewSerializer(data=request.data)

    # Check if the serializer is valid
    if serializer.is_valid():
        # Save the new review
        serializer.save()

        # Return a success message
        return Response(serializer.data, status=status.HTTP_201_CREATED)
    else:
        # Return an error message
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

APP.TSX

const [currentUser, setCurrentUser] = useState({id: 0, username: "", password: ""});
const [currentReviews, setCurrentReviews] = useState<any[]>([]);

const [page, setPage] = useState<string>("authentication");

const getUser = () => {
};
```

	DIFICULTAD	NOTAS
LLAMA	fácil.	no ha necesitado intervención.
COPILOT	fácil.	no ha necesitado intervención.

C3

```
Given the following context, could you complete the function "getReview" using "fetch"?

It's meant to be called after a button click over a specific review; given its id, it fills "currentReview" and
sets "page" to "review".

SERIALIZERS.PY

class ReviewSerializer(serializers.ModelSerializer):
    class Meta:
        model = Review
        fields = ('id', 'grade', 'text', 'reviewer', 'reviewed')

URLS.PY

urlpatterns = [
    path('reviews/<int:pk>/', views.handle_reviews_by_id)
]

VIEWS.PY

@api_view(['GET', 'PUT', 'DELETE'])
def handle_reviews_by_id(request, pk):
    """
```

```
Handles review CRUD operations by ID.

GET: Retrieves a review by ID.
PUT: Updates a review by ID.
DELETE: Deletes a review by ID.
"""

# Try to get the review by ID
try:
    review = Review.objects.get(pk=pk)
except Review.DoesNotExist:
    # Return a 404 error if the review does not exist
    return Response({'error': 'Review not found'}, status=status.HTTP_404_NOT_FOUND)

# Check if the request is a GET request
if request.method == 'GET':
    # Serialize the review
    serializer = ReviewSerializer(review)

    # Return the serialized review
    return Response(serializer.data)

# Check if the request is a PUT request
elif request.method == 'PUT':
    # Create a new serializer instance
    serializer = ReviewSerializer(review, data=request.data)

    # Check if the serializer is valid
    if serializer.is_valid():
        # Save the updated review
        serializer.save()

        # Return a success message
        return Response(serializer.data)
    else:
        # Return an error message
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

# Check if the request is a DELETE request
elif request.method == 'DELETE':
    # Delete the review
    review.delete()

    # Return a success message
    return Response({'message': 'Review deleted successfully'}, status=status.HTTP_204_NO_CONTENT)

APP.TSX

const [currentReviews, setCurrentReviews] = useState<any[]>([]);
const [page, setPage] = useState<string>("authentication");

const getReview = () => {
};
```

	DIFICULTAD	NOTAS
LLAMA	fácil.	no ha necesitado intervención.
COPILOT	fácil.	no ha necesitado intervención.

C4

```
Given the following context, could you complete the function "getHousesByCountry" using "fetch"?

It's meant to be called after a button click over a specific country, given its name as a parameter, it fills
"currentHouses" and sets "page" to "houses". Also, for each house in "currentHouses", use "user" id to get its
username and put it inside the field "user" of the house.

SERIALIZERS.PY

class UserSerializer(serializers.ModelSerializer):
    class Meta:
        model = User
        fields = ('id', 'username', 'password')

class HouseSerializer(serializers.ModelSerializer):
    class Meta:
        model = House
        fields = ('id', 'user', 'title', 'text', 'image', 'price', 'country')

URLS.PY

urlpatterns = [
    path('users/<int:pk>/', views.handle_users_by_id),
    path('houses/', views.handle_houses)
]
```

VIEWS.PY

```
@api_view(['GET', 'PUT', 'DELETE'])
def handle_users_by_id(request, pk):
    """
    Handles user CRUD operations by ID.

    GET: Retrieves a user by ID.
    PUT: Updates a user by ID.
    DELETE: Deletes a user by ID.
    """

    # Try to get the user by ID
    try:
        user = User.objects.get(pk=pk)
    except User.DoesNotExist:
        # Return a 404 error if the user does not exist
        return Response({'error': 'User not found'}, status=status.HTTP_404_NOT_FOUND)

    # Check if the request is a GET request
    if request.method == 'GET':
        # Serialize the user
        serializer = UserSerializer(user)

        # Return the serialized user
        return Response(serializer.data)

    # Check if the request is a PUT request
    elif request.method == 'PUT':
        # Create a new serializer instance
        serializer = UserSerializer(user, data=request.data)

        # Check if the serializer is valid
        if serializer.is_valid():
            # Save the updated user
            serializer.save()

            # Return a success message
            return Response(serializer.data)
        else:
            # Return an error message
            return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

    # Check if the request is a DELETE request
    elif request.method == 'DELETE':
        # Delete the user
        user.delete()

        # Return a success message
        return Response({'message': 'User deleted successfully'}, status=status.HTTP_204_NO_CONTENT)

@api_view(['GET', 'POST'])
def handle_houses(request):
    """
    Handles house CRUD operations.

    GET: Retrieves all houses.
    POST: Creates a new house.
    """

    # Check if the request is a GET request
    if request.method == 'GET':
        # Get the query parameters 'user' and 'country' from the URL
        user = request.GET.get('user')
        country = request.GET.get('country')

        # If 'user' is provided, filter houses by the 'user' attribute
        if user:
            try:
                user_id = int(user)
                houses = House.objects.filter(user_id=user_id)
            except ValueError:
                houses = House.objects.filter(Q(user__username=user) | Q(user__id=user))
        # If 'country' is provided, filter houses by the 'country' attribute
        elif country:
            houses = House.objects.filter(country=country)
        else:
            # Get all houses
            houses = House.objects.all()

        # Serialize the houses
        serializer = HouseSerializer(houses, many=True)

        # Return the serialized houses
```

```

        return Response(serializer.data)

    # Check if the request is a POST request
    elif request.method == 'POST':
        # Create a new serializer instance
        serializer = HouseSerializer(data=request.data)

        # Check if the serializer is valid
        if serializer.is_valid():
            # Save the new house
            serializer.save()

            # Return a success message
            return Response(serializer.data, status=status.HTTP_201_CREATED)
        else:
            # Return an error message
            return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

APP.TSX

const [currentHouses, setCurrentHouses] = useState<any[]>([]);
const [page, setPage] = useState<string>("authentication");

const getHousesByCountry = () => {
};

```

	DIFICULTAD	NOTAS
LLAMA	fácil.	no ha necesitado intervención.
COPILOT	fácil.	no ha necesitado intervención.

C5

Given the following context, could you complete the function "getHousesByUser" using "fetch"?

It's meant to be called after a button click over a specific user, given its id as a parameter, it fills "currentHouses" and sets "page" to "my_houses". Also, for each house in "currentHouses", you should also do fill the attribute "user" with the username).

SERIALIZERS.PY

```

class UserSerializer(serializers.ModelSerializer):
    class Meta:
        model = User
        fields = ('id', 'username', 'password')

class HouseSerializer(serializers.ModelSerializer):
    class Meta:
        model = House
        fields = ('id', 'user', 'title', 'text', 'image', 'price', 'country')

```

URLS.PY

```

urlpatterns = [
    path('users/<int:pk>/', views.handle_users_by_id),
    path('houses/', views.handle_houses)
]

```

VIEWS.PY

```

@api_view(['GET', 'PUT', 'DELETE'])
def handle_users_by_id(request, pk):
    """
    Handles user CRUD operations by ID.

    GET: Retrieves a user by ID.
    PUT: Updates a user by ID.
    DELETE: Deletes a user by ID.
    """

    # Try to get the user by ID
    try:
        user = User.objects.get(pk=pk)
    except User.DoesNotExist:
        # Return a 404 error if the user does not exist
        return Response({'error': 'User not found'}, status=status.HTTP_404_NOT_FOUND)

    # Check if the request is a GET request
    if request.method == 'GET':
        # Serialize the user
        serializer = UserSerializer(user)

        # Return the serialized user
        return Response(serializer.data)

```



```

# Check if the request is a PUT request
elif request.method == 'PUT':
    # Create a new serializer instance
    serializer = UserSerializer(user, data=request.data)

    # Check if the serializer is valid
    if serializer.is_valid():
        # Save the updated user
        serializer.save()

        # Return a success message
        return Response(serializer.data)
    else:
        # Return an error message
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

# Check if the request is a DELETE request
elif request.method == 'DELETE':
    # Delete the user
    user.delete()

    # Return a success message
    return Response({'message': 'User deleted successfully'}, status=status.HTTP_204_NO_CONTENT)

```

```
@api_view(['GET', 'POST'])
```

```
def handle_houses(request):
```

```
    """
```

```
    Handles house CRUD operations.
```

```
    GET: Retrieves all houses.
```

```
    POST: Creates a new house.
```

```
    """
```

```
    # Check if the request is a GET request
```

```
    if request.method == 'GET':
```

```
        # Get the query parameters 'user' and 'country' from the URL
```

```
        user = request.GET.get('user')
```

```
        country = request.GET.get('country')
```

```
        # If 'user' is provided, filter houses by the 'user' attribute
```

```
        if user:
```

```
            try:
```

```
                user_id = int(user)
```

```
                houses = House.objects.filter(user_id=user_id)
```

```
            except ValueError:
```

```
                houses = House.objects.filter(Q(user__username=user) | Q(user__id=user))
```

```
        # If 'country' is provided, filter houses by the 'country' attribute
```

```
        elif country:
```

```
            houses = House.objects.filter(country=country)
```

```
        else:
```

```
            # Get all houses
```

```
            houses = House.objects.all()
```

```
        # Serialize the houses
```

```
        serializer = HouseSerializer(houses, many=True)
```

```
        # Return the serialized houses
```

```
        return Response(serializer.data)
```

```
    # Check if the request is a POST request
```

```
    elif request.method == 'POST':
```

```
        # Create a new serializer instance
```

```
        serializer = HouseSerializer(data=request.data)
```

```
        # Check if the serializer is valid
```

```
        if serializer.is_valid():
```

```
            # Save the new house
```

```
            serializer.save()
```

```
            # Return a success message
```

```
            return Response(serializer.data, status=status.HTTP_201_CREATED)
```

```
        else:
```

```
            # Return an error message
```

```
            return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)
```

```
APP.TSX
```

```
const [currentHouses, setCurrentHouses] = useState<any[]>([]);
```

```
const [page, setPage] = useState<string>("authentication");
```

```
const getHousesByCountry = async (country: string) => {
```

```
    try {
```

```
        const response = await fetch(`http://127.0.0.1:8000/api/houses/?country=${country}`);
```

```
        const houses = await response.json();
```

```

        // Loop through each house and fetch the user's username
        const promises = houses.map((house: { user: any; }) => {
        return fetch(`http://127.0.0.1:8000/api/users/${house.user}/`)
            .then((response) => response.json())
            .then((user) => {
                house.user = user.username;
                return house;
            });
        });

        // Wait for all promises to resolve
        const updatedHouses = await Promise.all(promises);

        // Update the state with the updated houses
        setCurrentHouses(updatedHouses);
        setPage("houses");
    } catch (error) {
        console.error(error);
    }
};

const getHousesByUser = () => {
};

```

	DIFICULTAD	NOTAS
LLAMA	fácil.	no ha necesitado intervención.
COPILOT	fácil.	no ha necesitado intervención.

C6

Given the following context, could you complete the function "getHouse" using "fetch"?

It's meant to be called after a button click over a specific house; it fills "currentHouse" (with the house's ID given as a parameter), "currentOrders" (with the house's user's ID) and sets "page" to "house". In "currentHouse", you need to get its user's username and put it inside an attribute "username"; and for every order in "currentOrders", you need to get its user's username and put it inside an attribute "user_username".

SERIALIZERS.PY

```

class UserSerializer(serializers.ModelSerializer):
    class Meta:
        model = User
        fields = ('id', 'username', 'password')

class HouseSerializer(serializers.ModelSerializer):
    class Meta:
        model = House
        fields = ('id', 'user', 'title', 'text', 'image', 'price', 'country')

class OrderSerializer(serializers.ModelSerializer):
    class Meta:
        model = Order
        fields = ('id', 'text', 'status', 'user', 'house')

```

URLS.PY

```

urlpatterns = [
    path('users/<int:pk>/', views.handle_users_by_id),
    path('houses/<int:pk>/', views.handle_houses_by_id),
    path('orders/', views.handle_orders)
]

```

VIEWS.PY

```

@api_view(['GET', 'PUT', 'DELETE'])
def handle_users_by_id(request, pk):
    """
    Handles user CRUD operations by ID.

    GET: Retrieves a user by ID.
    PUT: Updates a user by ID.
    DELETE: Deletes a user by ID.
    """

    # Try to get the user by ID
    try:
        user = User.objects.get(pk=pk)
    except User.DoesNotExist:
        # Return a 404 error if the user does not exist
        return Response({'error': 'User not found'}, status=status.HTTP_404_NOT_FOUND)

    # Check if the request is a GET request
    if request.method == 'GET':
        # Serialize the user

```

```

        serializer = UserSerializer(user)

        # Return the serialized user
        return Response(serializer.data)

    # Check if the request is a PUT request
    elif request.method == 'PUT':
        # Create a new serializer instance
        serializer = UserSerializer(user, data=request.data)

        # Check if the serializer is valid
        if serializer.is_valid():
            # Save the updated user
            serializer.save()

            # Return a success message
            return Response(serializer.data)
        else:
            # Return an error message
            return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

    # Check if the request is a DELETE request
    elif request.method == 'DELETE':
        # Delete the user
        user.delete()

        # Return a success message
        return Response({'message': 'User deleted successfully'}, status=status.HTTP_204_NO_CONTENT)

@api_view(['GET', 'PUT', 'DELETE'])
def handle_houses_by_id(request, pk):
    """
    Handles house CRUD operations by ID.

    GET: Retrieves a house by ID.
    PUT: Updates a house by ID.
    DELETE: Deletes a house by ID.
    """

    # Try to get the house by ID
    try:
        house = House.objects.get(pk=pk)
    except House.DoesNotExist:
        # Return a 404 error if the house does not exist
        return Response({'error': 'House not found'}, status=status.HTTP_404_NOT_FOUND)

    # Check if the request is a GET request
    if request.method == 'GET':
        # Serialize the house
        serializer = HouseSerializer(house)

        # Return the serialized house
        return Response(serializer.data)

    # Check if the request is a PUT request
    elif request.method == 'PUT':
        # Create a new serializer instance
        serializer = HouseSerializer(house, data=request.data)

        # Check if the serializer is valid
        if serializer.is_valid():
            # Save the updated house
            serializer.save()

            # Return a success message
            return Response(serializer.data)
        else:
            # Return an error message
            return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

    # Check if the request is a DELETE request
    elif request.method == 'DELETE':
        # Delete the house
        house.delete()

        # Return a success message
        return Response({'message': 'House deleted successfully'}, status=status.HTTP_204_NO_CONTENT)

@api_view(['GET', 'POST'])
def handle_orders(request):
    """
    Handles order CRUD operations.

    GET: Retrieves all orders.
    POST: Creates a new order.

```

```
"""

# Check if the request is a GET request
if request.method == 'GET':
    # Get the query parameters 'user' and 'house' from the URL
    user = request.GET.get('user')
    house = request.GET.get('house')

    # If 'user' is provided, filter orders by the 'user' attribute
    if user:
        try:
            user_id = int(user)
            orders = Order.objects.filter(user_id=user_id)
        except ValueError:
            orders = Order.objects.filter(Q(user__username=user) | Q(user__id=user))
    # If 'house' is provided, filter orders by the 'house' attribute
    elif house:
        try:
            house_id = int(house)
            orders = Order.objects.filter(house_id=house_id)
        except ValueError:
            orders = Order.objects.filter(Q(house__title=house) | Q(house__id=house))
    else:
        # Get all orders
        orders = Order.objects.all()

    # Serialize the orders
    serializer = OrderSerializer(orders, many=True)

    # Return the serialized orders
    return Response(serializer.data)

# Check if the request is a POST request
elif request.method == 'POST':
    # Create a new serializer instance
    serializer = OrderSerializer(data=request.data)

    # Check if the serializer is valid
    if serializer.is_valid():
        # Save the new order
        serializer.save()

        # Return a success message
        return Response(serializer.data, status=status.HTTP_201_CREATED)
    else:
        # Return an error message
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

APP.TSX

const [currentHouse, setCurrentHouse] = useState({id: 0, user: "", title: "", text: "", image: "", price: 0,
country: ""});
const [currentOrders, setCurrentOrders] = useState<any[]>([]);

const [page, setPage] = useState<string>("authentication");

const getHouse = () => {
};
```

	DIFICULTAD	NOTAS
LLAMA	difícil.	sí ha necesitado intervención; el modelo genera código parcialmente funcional, pero se inventa los atributos de las entidades ignorando el contexto proporcionado.
COPILOT	fácil.	no ha necesitado intervención.

C7

```
Given the following context, could you complete the function "getOrders" using "fetch"?

It's meant to be called after a button click over a specific user; it fills "currentOrders" (with the user's ID
given as a parameter) and sets "page" to "orders". And for every order in "currentOrders", you need to get its
user's username and put it inside an attribute "user_username".

SERIALIZERS.PY

class UserSerializer(serializers.ModelSerializer):
class Meta:
    model = User
    fields = ('id', 'username', 'password')

class OrderSerializer(serializers.ModelSerializer):
class Meta:
    model = Order
    fields = ('id', 'text', 'status', 'user', 'house')
```

URLS.PY

```
urlpatterns = [
    path('users/<int:pk>/', views.handle_users_by_id),
    path('orders/', views.handle_orders)
]
```

VIEWS.PY

```
@api_view(['GET', 'PUT', 'DELETE'])
def handle_users_by_id(request, pk):
    """
    Handles user CRUD operations by ID.

    GET: Retrieves a user by ID.
    PUT: Updates a user by ID.
    DELETE: Deletes a user by ID.
    """

    # Try to get the user by ID
    try:
        user = User.objects.get(pk=pk)
    except User.DoesNotExist:
        # Return a 404 error if the user does not exist
        return Response({'error': 'User not found'}, status=status.HTTP_404_NOT_FOUND)

    # Check if the request is a GET request
    if request.method == 'GET':
        # Serialize the user
        serializer = UserSerializer(user)

        # Return the serialized user
        return Response(serializer.data)

    # Check if the request is a PUT request
    elif request.method == 'PUT':
        # Create a new serializer instance
        serializer = UserSerializer(user, data=request.data)

        # Check if the serializer is valid
        if serializer.is_valid():
            # Save the updated user
            serializer.save()

            # Return a success message
            return Response(serializer.data)
        else:
            # Return an error message
            return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

    # Check if the request is a DELETE request
    elif request.method == 'DELETE':
        # Delete the user
        user.delete()

        # Return a success message
        return Response({'message': 'User deleted successfully'}, status=status.HTTP_204_NO_CONTENT)

@api_view(['GET', 'POST'])
def handle_orders(request):
    """
    Handles order CRUD operations.

    GET: Retrieves all orders.
    POST: Creates a new order.
    """

    # Check if the request is a GET request
    if request.method == 'GET':
        # Get the query parameters 'user' and 'house' from the URL
        user = request.GET.get('user')
        house = request.GET.get('house')

        # If 'user' is provided, filter orders by the 'user' attribute
        if user:
            try:
                user_id = int(user)
                orders = Order.objects.filter(user_id=user_id)
            except ValueError:
                orders = Order.objects.filter(Q(user__username=user) | Q(user__id=user))
        # If 'house' is provided, filter orders by the 'house' attribute
        elif house:
            try:
                house_id = int(house)
                orders = Order.objects.filter(house_id=house_id)
```

```
except ValueError:
    orders = Order.objects.filter(Q(house__title=house) | Q(house__id=house))
else:
    # Get all orders
    orders = Order.objects.all()

    # Serialize the orders
    serializer = OrderSerializer(orders, many=True)

    # Return the serialized orders
    return Response(serializer.data)

# Check if the request is a POST request
elif request.method == 'POST':
    # Create a new serializer instance
    serializer = OrderSerializer(data=request.data)

    # Check if the serializer is valid
    if serializer.is_valid():
        # Save the new order
        serializer.save()

        # Return a success message
        return Response(serializer.data, status=status.HTTP_201_CREATED)
    else:
        # Return an error message
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

APP.TSX

const [currentOrders, setCurrentOrders] = useState<any[]>([]);
const [page, setPage] = useState<string>("authentication");

const getOrders = () => {
};
```

	DIFICULTAD	NOTAS
LLAMA	fácil.	no ha necesitado intervención.
COPILOT	fácil.	no ha necesitado intervención.

C8

Given the following context, could you complete the function "register" using "fetch"?

It's meant to be called after a button click, it uses the entered values "username" and "password" to create a new user, fills "loggedUser" and sets "page" to "browse".

SERIALIZERS.PY

```
class UserSerializer(serializers.ModelSerializer):
    class Meta:
        model = User
        fields = ('id', 'username', 'password')
```

URLS.PY

```
urlpatterns = [
    path('users/', views.handle_users)
]
```

VIEWS.PY

```
@api_view(['GET', 'POST'])
def handle_users(request):
    """
    Handles user CRUD operations.

    GET: Retrieves all users.
    POST: Creates a new user.
    """

    # Check if the request is a GET request
    if request.method == 'GET':
        # Get the query parameter 'username' from the URL
        username = request.GET.get('username')

        # If 'username' is provided, filter users by the 'username' attribute
        if username:
            users = User.objects.filter(username=username)
        else:
            # Get all users
            users = User.objects.all()

    # Serialize the users
```

```

        serializer = UserSerializer(users, many=True)

        # Return the serialized users
        return Response(serializer.data)

    # Check if the request is a POST request
    elif request.method == 'POST':
        # Create a new serializer instance
        serializer = UserSerializer(data=request.data)

        # Check if the serializer is valid
        if serializer.is_valid():
            # Save the new user
            serializer.save()

            # Return a success message
            return Response(serializer.data, status=status.HTTP_201_CREATED)
        else:
            # Return an error message
            return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

APP.TSX

const [username, setUsername] = useState<string>();
const [password, setPassword] = useState<string>();
const [loggedUser, setLoggedUser] = useState({id: 0, username: "", password: ""});

const register = () => {
};

```

	DIFICULTAD	NOTAS
LLAMA	fácil.	no ha necesitado intervención.
COPILOT	fácil.	no ha necesitado intervención.

C9

Given the following context, could you complete the function "createReview" using "fetch"?

It's meant to be called after a button click, it uses the entered values "newReviewGrade" and "newReviewText" and the values "loggedUser['id']" and "currentUser['id']" to create a new review, and should recall "getUser(currentUser['id'])".

SERIALIZERS.PY

```

class ReviewSerializer(serializers.ModelSerializer):
    class Meta:
        model = Review
        fields = ('id', 'grade', 'text', 'reviewer', 'reviewed')

```

URLS.PY

```

urlpatterns = [
    path('reviews/', views.handle_reviews),
]

```

VIEWS.PY

```

@api_view(['GET', 'POST'])
def handle_reviews(request):
    """
    Handles review CRUD operations.

    GET: Retrieves all reviews.
    POST: Creates a new review.
    """

    # Check if the request is a GET request
    if request.method == 'GET':
        # Get the query parameter 'reviewed' from the URL
        reviewed = request.GET.get('reviewed')

        # If 'reviewed' is provided, filter reviews by the 'reviewed' attribute
        if reviewed:
            try:
                reviewed_id = int(reviewed)
                reviews = Review.objects.filter(reviewed_id=reviewed_id)
            except ValueError:
                reviews = Review.objects.filter(Q(reviewed__username=reviewed) | Q(reviewed__id=reviewed))
        else:
            # Get all reviews
            reviews = Review.objects.all()

        # Serialize the reviews
        serializer = ReviewSerializer(reviews, many=True)

```

```
# Return the serialized reviews
return Response(serializer.data)

# Check if the request is a POST request
elif request.method == 'POST':
    # Create a new serializer instance
    serializer = ReviewSerializer(data=request.data)

    # Check if the serializer is valid
    if serializer.is_valid():
        # Save the new review
        serializer.save()

        # Return a success message
        return Response(serializer.data, status=status.HTTP_201_CREATED)
    else:
        # Return an error message
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

APP.TSX

const [loggedUser, setLoggedUser] = useState({id: 0, username: "", password: ""});
const [currentUser, setCurrentUser] = useState({id: 0, username: "", password: ""});

const [newReviewGrade, setNewReviewGrade] = useState(0);
const [newReviewText, setNewReviewText] = useState<string>();

const createReview = () => {
};
```

	DIFICULTAD	NOTAS
LLAMA	fácil.	no ha necesitado intervención.
COPILOT	fácil.	no ha necesitado intervención.

C10

```
Given the following context, could you complete the function "createHouse" using "fetch"?

It's meant to be called after a button click, it uses the entered values "newHouseTitle", "newHouseText",
"newHouseImage", "newHousePrice" and "newHouseCountry" and the value "loggedUser['id']" to create a new house, and
should recall "getHousesByUser(loggedUser['id'])".

SERIALIZERS.PY

class HouseSerializer(serializers.ModelSerializer):
    class Meta:
        model = House
        fields = ('id', 'user', 'title', 'text', 'image', 'price', 'country')

URLS.PY

urlpatterns = [
    path('houses/', views.handle_houses)
]

VIEWS.PY

@api_view(['GET', 'POST'])
def handle_houses(request):
    """
    Handles house CRUD operations.

    GET: Retrieves all houses.
    POST: Creates a new house.
    """

    # Check if the request is a GET request
    if request.method == 'GET':
        # Get the query parameters 'user' and 'country' from the URL
        user = request.GET.get('user')
        country = request.GET.get('country')

        # If 'user' is provided, filter houses by the 'user' attribute
        if user:
            try:
                user_id = int(user)
                houses = House.objects.filter(user_id=user_id)
            except ValueError:
                houses = House.objects.filter(Q(user__username=user) | Q(user__id=user))
        # If 'country' is provided, filter houses by the 'country' attribute
        elif country:
            houses = House.objects.filter(country=country)
        else:
```



```

        # Get all houses
        houses = House.objects.all()

        # Serialize the houses
        serializer = HouseSerializer(houses, many=True)

        # Return the serialized houses
        return Response(serializer.data)

    # Check if the request is a POST request
    elif request.method == 'POST':
        # Create a new serializer instance
        serializer = HouseSerializer(data=request.data)

        # Check if the serializer is valid
        if serializer.is_valid():
            # Save the new house
            serializer.save()

            # Return a success message
            return Response(serializer.data, status=status.HTTP_201_CREATED)
        else:
            # Return an error message
            return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

APP.TSX

const [loggedUser, setLoggedUser] = useState({id: 0, username: "", password: ""});

const [newHouseTitle, setNewHouseTitle] = useState<string>();
const [newHouseText, setNewHouseText] = useState<string>();
const [newHouseImage, setNewHouseImage] = useState<string>();
const [newHousePrice, setNewHousePrice] = useState(0);
const [newHouseCountry, setNewHouseCountry] = useState<string>();

const createHouse = () => {
};

```

	DIFICULTAD	NOTAS
LLAMA	fácil.	no ha necesitado intervención.
COPILOT	fácil.	no ha necesitado intervención.

C11

Given the following context, could you complete the function "createOrder" using "fetch"?

It's meant to be called after a button click, it uses the entered value "newOrderText" and the values "pending" (literal), "loggedUser['id']" and "currentHouse['id']" to create a new order, and should recall "getOrders(loggedUser['id'])".

SERIALIZERS.PY

```

class OrderSerializer(serializers.ModelSerializer):
    class Meta:
        model = Order
        fields = ('id', 'text', 'status', 'user', 'house')

```

URLS.PY

```

urlpatterns = [
    path('orders/', views.handle_orders),
]

```

VIEWS.PY

```

@api_view(['GET', 'POST'])
def handle_orders(request):
    """
    Handles order CRUD operations.

    GET: Retrieves all orders.
    POST: Creates a new order.
    """

    # Check if the request is a GET request
    if request.method == 'GET':
        # Get the query parameters 'user' and 'house' from the URL
        user = request.GET.get('user')
        house = request.GET.get('house')

        # If 'user' is provided, filter orders by the 'user' attribute
        if user:
            try:
                user_id = int(user)

```

```
        orders = Order.objects.filter(user_id=user_id)
    except ValueError:
        orders = Order.objects.filter(Q(user__username=user) | Q(user__id=user))
# If 'house' is provided, filter orders by the 'house' attribute
elif house:
    try:
        house_id = int(house)
        orders = Order.objects.filter(house_id=house_id)
    except ValueError:
        orders = Order.objects.filter(Q(house__title=house) | Q(house__id=house))
else:
    # Get all orders
    orders = Order.objects.all()

# Serialize the orders
serializer = OrderSerializer(orders, many=True)

# Return the serialized orders
return Response(serializer.data)

# Check if the request is a POST request
elif request.method == 'POST':
    # Create a new serializer instance
    serializer = OrderSerializer(data=request.data)

    # Check if the serializer is valid
    if serializer.is_valid():
        # Save the new order
        serializer.save()

        # Return a success message
        return Response(serializer.data, status=status.HTTP_201_CREATED)
    else:
        # Return an error message
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

APP.TSX

const [loggedUser, setLoggedUser] = useState({id: 0, username: "", password: ""});

const [currentHouse, setCurrentHouse] = useState({id: 0, user: 0, title: "", text: "", image: "", price: 0,
country: "", username: ""});
const [newOrderText, setNewOrderText] = useState<string>();

const createOrder = () => {
};
```

	DIFICULTAD	NOTAS
LLAMA	fácil.	no ha necesitado intervención.
COPILLOT	fácil.	no ha necesitado intervención.

C12

```
Given the following context, could you complete the function "editUsername" using "fetch"?

It's meant to be called after a button click, using "newUserUsername" and "loggedUser", it edits the field
"username" and leaves "password" with the same value, and at the end call "getUser(loggedUser['id'])" (already
implemented).

SERIALIZERS.PY

class UserSerializer(serializers.ModelSerializer):
    class Meta:
        model = User
        fields = ('id', 'username', 'password')

URLS.PY

urlpatterns = [
    path('users/<int:pk>/', views.handle_users_by_id)
]

VIEWS.PY

@api_view(['GET', 'PUT', 'DELETE'])
def handle_users_by_id(request, pk):
    """
    Handles user CRUD operations by ID.

    GET: Retrieves a user by ID.
    PUT: Updates a user by ID.
    DELETE: Deletes a user by ID.
    """
```

```
# Try to get the user by ID
try:
    user = User.objects.get(pk=pk)
except User.DoesNotExist:
    # Return a 404 error if the user does not exist
    return Response({'error': 'User not found'}, status=status.HTTP_404_NOT_FOUND)

# Check if the request is a GET request
if request.method == 'GET':
    # Serialize the user
    serializer = UserSerializer(user)

    # Return the serialized user
    return Response(serializer.data)

# Check if the request is a PUT request
elif request.method == 'PUT':
    # Create a new serializer instance
    serializer = UserSerializer(user, data=request.data)

    # Check if the serializer is valid
    if serializer.is_valid():
        # Save the updated user
        serializer.save()

        # Return a success message
        return Response(serializer.data)
    else:
        # Return an error message
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

# Check if the request is a DELETE request
elif request.method == 'DELETE':
    # Delete the user
    user.delete()

    # Return a success message
    return Response({'message': 'User deleted successfully'}, status=status.HTTP_204_NO_CONTENT)

APP.TSX

const [currentUser, setCurrentUser] = useState({id: 0, username: "", password: ""});
const [newUserUsername, setNewUserUsername] = useState<string>();

const editUsername = () => {
}
```

	DIFICULTAD	NOTAS
LLAMA	fácil.	no ha necesitado intervención.
COPILLOT	fácil.	no ha necesitado intervención.

C13

```
Given the following context, could you complete the function "editReview" using "fetch"?

It's meant to be called after a button click, using "newReviewGrade" and "newReviewText", it edits the fields
"grade" and "text" and leaves the rest with the same value, and at the end recall "getUser(currentUser['id'])".

SERIALIZERS.PY

class ReviewSerializer(serializers.ModelSerializer):
    class Meta:
        model = Review
        fields = ('id', 'grade', 'text', 'reviewer', 'reviewed')

URLS.PY

urlpatterns = [
    path('reviews/<int:pk>/', views.handle_reviews_by_id)
]

VIEWS.PY

@api_view(['GET', 'PUT', 'DELETE'])
def handle_reviews_by_id(request, pk):
    """
    Handles review CRUD operations by ID.

    GET: Retrieves a review by ID.
    PUT: Updates a review by ID.
    DELETE: Deletes a review by ID.
    """

    # Try to get the review by ID
```

```
try:
    review = Review.objects.get(pk=pk)
except Review.DoesNotExist:
    # Return a 404 error if the review does not exist
    return Response({'error': 'Review not found'}, status=status.HTTP_404_NOT_FOUND)

# Check if the request is a GET request
if request.method == 'GET':
    # Serialize the review
    serializer = ReviewSerializer(review)

    # Return the serialized review
    return Response(serializer.data)

# Check if the request is a PUT request
elif request.method == 'PUT':
    # Create a new serializer instance
    serializer = ReviewSerializer(review, data=request.data)

    # Check if the serializer is valid
    if serializer.is_valid():
        # Save the updated review
        serializer.save()

        # Return a success message
        return Response(serializer.data)
    else:
        # Return an error message
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

# Check if the request is a DELETE request
elif request.method == 'DELETE':
    # Delete the review
    review.delete()

    # Return a success message
    return Response({'message': 'Review deleted successfully'}, status=status.HTTP_204_NO_CONTENT)

APP.TSX

const [currentReview, setCurrentReview] = useState({id: 0, grade: 0, text: "", reviewer: 0, reviewed: 0});

const [newReviewGrade, setNewReviewGrade] = useState(0);
const [newReviewText, setNewReviewText] = useState<string>();

const editReview = () => {
}
```

	DIFICULTAD	NOTAS
LLAMA	fácil.	no ha necesitado intervención.
COPILOT	fácil.	no ha necesitado intervención.

C14

Given the following context, could you complete the function "editHouse" using "fetch"?

It's meant to be called after a button click over, using "currentHouse" and the variables "newHouseTitle", "newHouseText", "newHouseImage", "newHousePrice" and "newHouseCountry", it edits the fields "title", "text", "image", "price" and "country" and leaves the rest with the same value (values from the object "currentHouse"), and at the end recall "getHouse(currentHouse['id'])".

SERIALIZERS.PY

```
class HouseSerializer(serializers.ModelSerializer):
    class Meta:
        model = House
        fields = ('id', 'user', 'title', 'text', 'image', 'price', 'country')
```

URLS.PY

```
urlpatterns = [
    path('houses/<int:pk>/', views.handle_houses_by_id)
]
```

VIEWS.PY

```
@api_view(['GET', 'PUT', 'DELETE'])
def handle_houses_by_id(request, pk):
    """
    Handles house CRUD operations by ID.

    GET: Retrieves a house by ID.
    PUT: Updates a house by ID.
    DELETE: Deletes a house by ID.
```

```
"""

# Try to get the house by ID
try:
    house = House.objects.get(pk=pk)
except House.DoesNotExist:
    # Return a 404 error if the house does not exist
    return Response({'error': 'House not found'}, status=status.HTTP_404_NOT_FOUND)

# Check if the request is a GET request
if request.method == 'GET':
    # Serialize the house
    serializer = HouseSerializer(house)

    # Return the serialized house
    return Response(serializer.data)

# Check if the request is a PUT request
elif request.method == 'PUT':
    # Create a new serializer instance
    serializer = HouseSerializer(house, data=request.data)

    # Check if the serializer is valid
    if serializer.is_valid():
        # Save the updated house
        serializer.save()

        # Return a success message
        return Response(serializer.data)
    else:
        # Return an error message
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

# Check if the request is a DELETE request
elif request.method == 'DELETE':
    # Delete the house
    house.delete()

    # Return a success message
    return Response({'message': 'House deleted successfully'}, status=status.HTTP_204_NO_CONTENT)

APP.TSX

const [currentHouse, setCurrentHouse] = useState({id: 0, user: 0, title: "", text: "", image: "", price: 0,
country: "", username: ""});

const [newHouseTitle, setNewHouseTitle] = useState<string>();
const [newHouseText, setNewHouseText] = useState<string>();
const [newHouseImage, setNewHouseImage] = useState<string>();
const [newHousePrice, setNewHousePrice] = useState(0);
const [newHouseCountry, setNewHouseCountry] = useState<string>();

const editHouse = () => {
}
```

	DIFICULTAD	NOTAS
LLAMA	fácil.	no ha necesitado intervención.
COPILOT	fácil.	no ha necesitado intervención.

C15

```
Given the following context, could you complete the function "acceptOrder" using "fetch"?

It's meant to be called after a button click over a specific order, given the parameters "id", "text", "user" and "house", it edits the field "status" as "accepted" and sets "text", "user" and "house" to the value from the parameters; and at the end recalls "getHouse(currentHouse['id'])".

SERIALIZERS.PY

class OrderSerializer(serializers.ModelSerializer):
    class Meta:
        model = Order
        fields = ('id', 'text', 'status', 'user', 'house')

URLS.PY

urlpatterns = [
    path('orders/<int:pk>/', views.handle_orders_by_id)
]

VIEWS.PY

@api_view(['GET', 'PUT', 'DELETE'])
def handle_orders_by_id(request, pk):
```

```

"""
Handles order CRUD operations by ID.

GET: Retrieves an order by ID.
PUT: Updates an order by ID.
DELETE: Deletes an order by ID.
"""

# Try to get the order by ID
try:
    order = Order.objects.get(pk=pk)
except Order.DoesNotExist:
    # Return a 404 error if the order does not exist
    return Response({'error': 'Order not found'}, status=status.HTTP_404_NOT_FOUND)

# Check if the request is a GET request
if request.method == 'GET':
    # Serialize the order
    serializer = OrderSerializer(order)

    # Return the serialized order
    return Response(serializer.data)

# Check if the request is a PUT request
elif request.method == 'PUT':
    # Create a new serializer instance
    serializer = OrderSerializer(order, data=request.data)

    # Check if the serializer is valid
    if serializer.is_valid():
        # Save the updated order
        serializer.save()

        # Return a success message
        return Response(serializer.data)
    else:
        # Return an error message
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

# Check if the request is a DELETE request
elif request.method == 'DELETE':
    # Delete the order
    order.delete()

    # Return a success message
    return Response({'message': 'Order deleted successfully'}, status=status.HTTP_204_NO_CONTENT)

APP.TSX

const [currentHouse, setCurrentHouse] = useState({id: 0, user: 0, title: "", text: "", image: "", price: 0,
country: "", username: ""});

const acceptOrder = () => {
}

```

	DIFICULTAD	NOTAS
LLAMA	fácil.	no ha necesitado intervención.
COPILOT	fácil.	no ha necesitado intervención.

C16

```

Given the following context, could you complete the function "rejectOrder" using "fetch"?

It's meant to be called after a button click over a specific order, given the parameters "id", "text", "user" and
"house", it edits the field "status" as "rejected" and sets "text", "user" and "house" to the value from the
parameters; and at the end recalls "getHouse(currentHouse['id'])" (these are already implemented).

SERIALIZERS.PY

class OrderSerializer(serializers.ModelSerializer):
    class Meta:
        model = Order
        fields = ('id', 'text', 'status', 'user', 'house')

URLS.PY

urlpatterns = [
    path('orders/<int:pk>/', views.handle_orders_by_id)
]

VIEWS.PY

@api_view(['GET', 'PUT', 'DELETE'])
def handle_orders_by_id(request, pk):

```

```

"""
Handles order CRUD operations by ID.

GET: Retrieves an order by ID.
PUT: Updates an order by ID.
DELETE: Deletes an order by ID.
"""

# Try to get the order by ID
try:
    order = Order.objects.get(pk=pk)
except Order.DoesNotExist:
    # Return a 404 error if the order does not exist
    return Response({'error': 'Order not found'}, status=status.HTTP_404_NOT_FOUND)

# Check if the request is a GET request
if request.method == 'GET':
    # Serialize the order
    serializer = OrderSerializer(order)

    # Return the serialized order
    return Response(serializer.data)

# Check if the request is a PUT request
elif request.method == 'PUT':
    # Create a new serializer instance
    serializer = OrderSerializer(order, data=request.data)

    # Check if the serializer is valid
    if serializer.is_valid():
        # Save the updated order
        serializer.save()

        # Return a success message
        return Response(serializer.data)
    else:
        # Return an error message
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

# Check if the request is a DELETE request
elif request.method == 'DELETE':
    # Delete the order
    order.delete()

    # Return a success message
    return Response({'message': 'Order deleted successfully'}, status=status.HTTP_204_NO_CONTENT)

APP.TSX

const rejectOrder = () => {

```

	DIFICULTAD	NOTAS
LLAMA	fácil.	no ha necesitado intervención.
COPILLOT	fácil.	no ha necesitado intervención.

C17

Given the following context, could you complete the function "deleteUser" using "fetch"?

It's meant to be called after a button click over a specific user (with the user's ID given as a parameter).

It follows the following steps.

1. Gets all reviews made by the user (user's ID in the attribute "reviewer") and deletes them.
2. Gets all the reviews others made to the user (user's ID in the attribute "reviewed") and deletes them.
3. Gets all the orders made by the user (user's ID in the attribute "user") and deletes them.
4. Gets all the houses made by the user (user's ID in the attribute "user").
 - 4.1. For each house, get all the orders others made to it (house's ID in the attribute "house") and deletes them.
 - 4.2. Deletes all previously obtained houses.
5. Deletes the user, and at the end calls "setLoggedUser({id: 0, username: '', password: ''})".

SERIALIZERS.PY

```

class UserSerializer(serializers.ModelSerializer):
    class Meta:
        model = User
        fields = ('id', 'username', 'password')

```

```

class ReviewSerializer(serializers.ModelSerializer):
    class Meta:
        model = Review
        fields = ('id', 'grade', 'text', 'reviewer', 'reviewed')

class HouseSerializer(serializers.ModelSerializer):
    class Meta:
        model = House
        fields = ('id', 'user', 'title', 'text', 'image', 'price', 'country')

class OrderSerializer(serializers.ModelSerializer):
    class Meta:
        model = Order
        fields = ('id', 'text', 'status', 'user', 'house')

URLS.PY

urlpatterns = [
    path('users/<int:pk>/', views.handle_users_by_id),
    path('reviews/', views.handle_reviews),
    path('reviews/<int:pk>/', views.handle_reviews_by_id),
    path('houses/', views.handle_houses),
    path('houses/<int:pk>/', views.handle_houses_by_id),
    path('orders/', views.handle_orders),
    path('orders/<int:pk>/', views.handle_orders_by_id)
]

VIEWS.PY

@api_view(['GET', 'PUT', 'DELETE'])
def handle_users_by_id(request, pk):
    """
    Handles user CRUD operations by ID.

    GET: Retrieves a user by ID.
    PUT: Updates a user by ID.
    DELETE: Deletes a user by ID.
    """

    # Try to get the user by ID
    try:
        user = User.objects.get(pk=pk)
    except User.DoesNotExist:
        # Return a 404 error if the user does not exist
        return Response({'error': 'User not found'}, status=status.HTTP_404_NOT_FOUND)

    # Check if the request is a GET request
    if request.method == 'GET':
        # Serialize the user
        serializer = UserSerializer(user)

        # Return the serialized user
        return Response(serializer.data)

    # Check if the request is a PUT request
    elif request.method == 'PUT':
        # Create a new serializer instance
        serializer = UserSerializer(user, data=request.data)

        # Check if the serializer is valid
        if serializer.is_valid():
            # Save the updated user
            serializer.save()

            # Return a success message
            return Response(serializer.data)
        else:
            # Return an error message
            return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

    # Check if the request is a DELETE request
    elif request.method == 'DELETE':
        # Delete the user
        user.delete()

        # Return a success message
        return Response({'message': 'User deleted successfully'}, status=status.HTTP_204_NO_CONTENT)

@api_view(['GET', 'POST'])
def handle_reviews(request):
    """
    Handles review CRUD operations.

    GET: Retrieves all reviews or reviews by reviewer or reviewed.

```



```

POST: Creates a new review.
"""

# Check if the request is a GET request
if request.method == 'GET':
    # Get the query parameters 'reviewer' and 'reviewed' from the URL
    reviewer = request.GET.get('reviewer')
    reviewed = request.GET.get('reviewed')

    # Filter reviews based on the query parameters
    if reviewer and reviewed:
        # Filter reviews by reviewer and reviewed
        reviews = Review.objects.filter(reviewer=reviewer, reviewed=reviewed)
    elif reviewer:
        # Filter reviews by reviewer
        reviews = Review.objects.filter(reviewer=reviewer)
    elif reviewed:
        # Filter reviews by reviewed
        reviews = Review.objects.filter(reviewed=reviewed)
    else:
        # Get all reviews
        reviews = Review.objects.all()

    # Serialize the reviews
    serializer = ReviewSerializer(reviews, many=True)

    # Return the serialized reviews
    return Response(serializer.data)

# Check if the request is a POST request
elif request.method == 'POST':
    # Create a new serializer instance
    serializer = ReviewSerializer(data=request.data)

    # Check if the serializer is valid
    if serializer.is_valid():
        # Save the new review
        serializer.save()

        # Return a success message
        return Response(serializer.data, status=status.HTTP_201_CREATED)
    else:
        # Return an error message
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

@api_view(['GET', 'PUT', 'DELETE'])
def handle_reviews_by_id(request, pk):
    """
    Handles review CRUD operations by ID.

    GET: Retrieves a review by ID.
    PUT: Updates a review by ID.
    DELETE: Deletes a review by ID.
    """

    # Try to get the review by ID
    try:
        review = Review.objects.get(pk=pk)
    except Review.DoesNotExist:
        # Return a 404 error if the review does not exist
        return Response({'error': 'Review not found'}, status=status.HTTP_404_NOT_FOUND)

    # Check if the request is a GET request
    if request.method == 'GET':
        # Serialize the review
        serializer = ReviewSerializer(review)

        # Return the serialized review
        return Response(serializer.data)

    # Check if the request is a PUT request
    elif request.method == 'PUT':
        # Create a new serializer instance
        serializer = ReviewSerializer(review, data=request.data)

        # Check if the serializer is valid
        if serializer.is_valid():
            # Save the updated review
            serializer.save()

            # Return a success message
            return Response(serializer.data)
        else:
            # Return an error message
            return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

```

```

# Check if the request is a DELETE request
elif request.method == 'DELETE':
    # Delete the review
    review.delete()

    # Return a success message
    return Response({'message': 'Review deleted successfully'}, status=status.HTTP_204_NO_CONTENT)

@api_view(['GET', 'POST'])
def handle_houses(request):
    """
    Handles house CRUD operations.

    GET: Retrieves all houses.
    POST: Creates a new house.
    """

    # Check if the request is a GET request
    if request.method == 'GET':
        # Get the query parameters 'user' and 'country' from the URL
        user = request.GET.get('user')
        country = request.GET.get('country')

        # If 'user' is provided, filter houses by the 'user' attribute
        if user:
            try:
                user_id = int(user)
                houses = House.objects.filter(user_id=user_id)
            except ValueError:
                houses = House.objects.filter(Q(user__username=user) | Q(user__id=user))
        # If 'country' is provided, filter houses by the 'country' attribute
        elif country:
            houses = House.objects.filter(country=country)
        else:
            # Get all houses
            houses = House.objects.all()

        # Serialize the houses
        serializer = HouseSerializer(houses, many=True)

        # Return the serialized houses
        return Response(serializer.data)

    # Check if the request is a POST request
    elif request.method == 'POST':
        # Create a new serializer instance
        serializer = HouseSerializer(data=request.data)

        # Check if the serializer is valid
        if serializer.is_valid():
            # Save the new house
            serializer.save()

            # Return a success message
            return Response(serializer.data, status=status.HTTP_201_CREATED)
        else:
            # Return an error message
            return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

@api_view(['GET', 'PUT', 'DELETE'])
def handle_houses_by_id(request, pk):
    """
    Handles house CRUD operations by ID.

    GET: Retrieves a house by ID.
    PUT: Updates a house by ID.
    DELETE: Deletes a house by ID.
    """

    # Try to get the house by ID
    try:
        house = House.objects.get(pk=pk)
    except House.DoesNotExist:
        # Return a 404 error if the house does not exist
        return Response({'error': 'House not found'}, status=status.HTTP_404_NOT_FOUND)

    # Check if the request is a GET request
    if request.method == 'GET':
        # Serialize the house
        serializer = HouseSerializer(house)

        # Return the serialized house
        return Response(serializer.data)

```

```

# Check if the request is a PUT request
elif request.method == 'PUT':
    # Create a new serializer instance
    serializer = HouseSerializer(house, data=request.data)

    # Check if the serializer is valid
    if serializer.is_valid():
        # Save the updated house
        serializer.save()

        # Return a success message
        return Response(serializer.data)
    else:
        # Return an error message
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

# Check if the request is a DELETE request
elif request.method == 'DELETE':
    # Delete the house
    house.delete()

    # Return a success message
    return Response({'message': 'House deleted successfully'}, status=status.HTTP_204_NO_CONTENT)

@api_view(['GET', 'POST'])
def handle_orders(request):
    """
    Handles order CRUD operations.

    GET: Retrieves all orders.
    POST: Creates a new order.
    """

    # Check if the request is a GET request
    if request.method == 'GET':
        # Get the query parameters 'user' and 'house' from the URL
        user = request.GET.get('user')
        house = request.GET.get('house')

        # If 'user' is provided, filter orders by the 'user' attribute
        if user:
            try:
                user_id = int(user)
                orders = Order.objects.filter(user_id=user_id)
            except ValueError:
                orders = Order.objects.filter(Q(user__username=user) | Q(user__id=user))
        # If 'house' is provided, filter orders by the 'house' attribute
        elif house:
            try:
                house_id = int(house)
                orders = Order.objects.filter(house_id=house_id)
            except ValueError:
                orders = Order.objects.filter(Q(house__title=house) | Q(house__id=house))
        else:
            # Get all orders
            orders = Order.objects.all()

        # Serialize the orders
        serializer = OrderSerializer(orders, many=True)

        # Return the serialized orders
        return Response(serializer.data)

    # Check if the request is a POST request
    elif request.method == 'POST':
        # Create a new serializer instance
        serializer = OrderSerializer(data=request.data)

        # Check if the serializer is valid
        if serializer.is_valid():
            # Save the new order
            serializer.save()

            # Return a success message
            return Response(serializer.data, status=status.HTTP_201_CREATED)
        else:
            # Return an error message
            return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

@api_view(['GET', 'PUT', 'DELETE'])
def handle_orders_by_id(request, pk):
    """
    Handles order CRUD operations by ID.

    GET: Retrieves an order by ID.

```

```
PUT: Updates an order by ID.
DELETE: Deletes an order by ID.
"""

# Try to get the order by ID
try:
    order = Order.objects.get(pk=pk)
except Order.DoesNotExist:
    # Return a 404 error if the order does not exist
    return Response({'error': 'Order not found'}, status=status.HTTP_404_NOT_FOUND)

# Check if the request is a GET request
if request.method == 'GET':
    # Serialize the order
    serializer = OrderSerializer(order)

    # Return the serialized order
    return Response(serializer.data)

# Check if the request is a PUT request
elif request.method == 'PUT':
    # Create a new serializer instance
    serializer = OrderSerializer(order, data=request.data)

    # Check if the serializer is valid
    if serializer.is_valid():
        # Save the updated order
        serializer.save()

        # Return a success message
        return Response(serializer.data)
    else:
        # Return an error message
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

# Check if the request is a DELETE request
elif request.method == 'DELETE':
    # Delete the order
    order.delete()

    # Return a success message
    return Response({'message': 'Order deleted successfully'}, status=status.HTTP_204_NO_CONTENT)

APP.TSX

const [loggedUser, setLoggedUser] = useState({id: 0, username: "", password: ""});

const deleteUser = () => {
}
```

	DIFICULTAD	NOTAS
LLAMA	difícil.	sí ha necesitado intervención; el modelo genera código parcialmente funcional, pero se confunde constantemente con el orden de las operaciones.
COPILOT	medio.	no ha necesitado intervención; pero si no se especifica el procedimiento, el modelo se inventa las peticiones al servidor ignorando el contexto proporcionado.

C18

```
Given the following context, could you complete the function "deleteReview" using "fetch"?

It's meant to be called after a button click over a specific review (with the review's ID given as a parameter),
it deletes it and calls "getUser(currentUser["id"])".

SERIALIZERS.PY

class ReviewSerializer(serializers.ModelSerializer):
    class Meta:
        model = Review
        fields = ('id', 'grade', 'text', 'reviewer', 'reviewed')

URLS.PY

urlpatterns = [
    path('reviews/<int:pk>/', views.handle_reviews_by_id)
]

VIEWS.PY

@api_view(['GET', 'PUT', 'DELETE'])
def handle_reviews_by_id(request, pk):
    """
    Handles review CRUD operations by ID.

    GET: Retrieves a review by ID.
```

```
PUT: Updates a review by ID.
DELETE: Deletes a review by ID.
"""

# Try to get the review by ID
try:
    review = Review.objects.get(pk=pk)
except Review.DoesNotExist:
    # Return a 404 error if the review does not exist
    return Response({'error': 'Review not found'}, status=status.HTTP_404_NOT_FOUND)

# Check if the request is a GET request
if request.method == 'GET':
    # Serialize the review
    serializer = ReviewSerializer(review)

    # Return the serialized review
    return Response(serializer.data)

# Check if the request is a PUT request
elif request.method == 'PUT':
    # Create a new serializer instance
    serializer = ReviewSerializer(review, data=request.data)

    # Check if the serializer is valid
    if serializer.is_valid():
        # Save the updated review
        serializer.save()

        # Return a success message
        return Response(serializer.data)
    else:
        # Return an error message
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

# Check if the request is a DELETE request
elif request.method == 'DELETE':
    # Delete the review
    review.delete()

    # Return a success message
    return Response({'message': 'Review deleted successfully'}, status=status.HTTP_204_NO_CONTENT)

APP.TSX

const deleteReview = () => {
}
```

	DIFICULTAD	NOTAS
LLAMA	fácil.	no ha necesitado intervención.
COPILOT	fácil.	no ha necesitado intervención.

C19

```
Given the following context, could you complete the function "deleteHouse" using "fetch"?

It's meant to be called after a button click over a specific house (with the order's ID given as a parameter).

It follows these steps.

1. Get all orders by the house's ID.
2. Deletes each order.
3. Deletes the house and calls "getHousesByUser(loggedUser["id"])" (already implemented).

SERIALIZERS.PY

class HouseSerializer(serializers.ModelSerializer):
    class Meta:
        model = House
        fields = ('id', 'user', 'title', 'text', 'image', 'price', 'country')

class OrderSerializer(serializers.ModelSerializer):
    class Meta:
        model = Order
        fields = ('id', 'text', 'status', 'user', 'house')

URLS.PY

urlpatterns = [
    path('houses/<int:pk>/', views.handle_houses_by_id),
    path('orders/', views.handle_orders),
    path('orders/<int:pk>/', views.handle_orders_by_id)
```

```
]
```

```
VIEWS.PY
```

```
@api_view(['GET', 'PUT', 'DELETE'])
def handle_houses_by_id(request, pk):
    """
    Handles house CRUD operations by ID.

    GET: Retrieves a house by ID.
    PUT: Updates a house by ID.
    DELETE: Deletes a house by ID.
    """

    # Try to get the house by ID
    try:
        house = House.objects.get(pk=pk)
    except House.DoesNotExist:
        # Return a 404 error if the house does not exist
        return Response({'error': 'House not found'}, status=status.HTTP_404_NOT_FOUND)

    # Check if the request is a GET request
    if request.method == 'GET':
        # Serialize the house
        serializer = HouseSerializer(house)

        # Return the serialized house
        return Response(serializer.data)

    # Check if the request is a PUT request
    elif request.method == 'PUT':
        # Create a new serializer instance
        serializer = HouseSerializer(house, data=request.data)

        # Check if the serializer is valid
        if serializer.is_valid():
            # Save the updated house
            serializer.save()

            # Return a success message
            return Response(serializer.data)
        else:
            # Return an error message
            return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

    # Check if the request is a DELETE request
    elif request.method == 'DELETE':
        # Delete the house
        house.delete()

        # Return a success message
        return Response({'message': 'House deleted successfully'}, status=status.HTTP_204_NO_CONTENT)

@api_view(['GET', 'POST'])
def handle_orders(request):
    """
    Handles order CRUD operations.

    GET: Retrieves all orders.
    POST: Creates a new order.
    """

    # Check if the request is a GET request
    if request.method == 'GET':
        # Get the query parameters 'user' and 'house' from the URL
        user = request.GET.get('user')
        house = request.GET.get('house')

        # If 'user' is provided, filter orders by the 'user' attribute
        if user:
            try:
                user_id = int(user)
                orders = Order.objects.filter(user_id=user_id)
            except ValueError:
                orders = Order.objects.filter(Q(user__username=user) | Q(user__id=user))
        # If 'house' is provided, filter orders by the 'house' attribute
        elif house:
            try:
                house_id = int(house)
                orders = Order.objects.filter(house_id=house_id)
            except ValueError:
                orders = Order.objects.filter(Q(house__title=house) | Q(house__id=house))
        else:
            # Get all orders
            orders = Order.objects.all()
```

```

# Serialize the orders
serializer = OrderSerializer(orders, many=True)

# Return the serialized orders
return Response(serializer.data)

# Check if the request is a POST request
elif request.method == 'POST':
    # Create a new serializer instance
    serializer = OrderSerializer(data=request.data)

    # Check if the serializer is valid
    if serializer.is_valid():
        # Save the new order
        serializer.save()

        # Return a success message
        return Response(serializer.data, status=status.HTTP_201_CREATED)
    else:
        # Return an error message
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

@api_view(['GET', 'PUT', 'DELETE'])
def handle_orders_by_id(request, pk):
    """
    Handles order CRUD operations by ID.

    GET: Retrieves an order by ID.
    PUT: Updates an order by ID.
    DELETE: Deletes an order by ID.
    """

    # Try to get the order by ID
    try:
        order = Order.objects.get(pk=pk)
    except Order.DoesNotExist:
        # Return a 404 error if the order does not exist
        return Response({'error': 'Order not found'}, status=status.HTTP_404_NOT_FOUND)

    # Check if the request is a GET request
    if request.method == 'GET':
        # Serialize the order
        serializer = OrderSerializer(order)

        # Return the serialized order
        return Response(serializer.data)

    # Check if the request is a PUT request
    elif request.method == 'PUT':
        # Create a new serializer instance
        serializer = OrderSerializer(order, data=request.data)

        # Check if the serializer is valid
        if serializer.is_valid():
            # Save the updated order
            serializer.save()

            # Return a success message
            return Response(serializer.data)
        else:
            # Return an error message
            return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

    # Check if the request is a DELETE request
    elif request.method == 'DELETE':
        # Delete the order
        order.delete()

        # Return a success message
        return Response({'message': 'Order deleted successfully'}, status=status.HTTP_204_NO_CONTENT)

APP.TSX

const deleteHouse = () => {
}

```

	DIFICULTAD	NOTAS
LLAMA	medio.	no ha necesitado intervención; pero si no se especifica el procedimiento, el modelo se inventa las peticiones al servidor ignorando el contexto proporcionado.
COPILOT	fácil.	no ha necesitado intervención.

Given the following context, could you complete the function "deleteOrder" using "fetch"?

It's meant to be called after a button click over a specific order (with the order's ID given as a parameter), it deletes it and then calls "getOrders(loggedUser["id"])".

SERIALIZERS.PY

```
class OrderSerializer(serializers.ModelSerializer):
    class Meta:
        model = Order
        fields = ('id', 'text', 'status', 'user', 'house')
```

URLS.PY

```
urlpatterns = [
    path('orders/<int:pk>/', views.handle_orders_by_id)
]
```

VIEWS.PY

```
@api_view(['GET', 'PUT', 'DELETE'])
def handle_orders_by_id(request, pk):
    """
    Handles order CRUD operations by ID.

    GET: Retrieves an order by ID.
    PUT: Updates an order by ID.
    DELETE: Deletes an order by ID.
    """

    # Try to get the order by ID
    try:
        order = Order.objects.get(pk=pk)
    except Order.DoesNotExist:
        # Return a 404 error if the order does not exist
        return Response({'error': 'Order not found'}, status=status.HTTP_404_NOT_FOUND)

    # Check if the request is a GET request
    if request.method == 'GET':
        # Serialize the order
        serializer = OrderSerializer(order)

        # Return the serialized order
        return Response(serializer.data)

    # Check if the request is a PUT request
    elif request.method == 'PUT':
        # Create a new serializer instance
        serializer = OrderSerializer(order, data=request.data)

        # Check if the serializer is valid
        if serializer.is_valid():
            # Save the updated order
            serializer.save()

            # Return a success message
            return Response(serializer.data)
        else:
            # Return an error message
            return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

    # Check if the request is a DELETE request
    elif request.method == 'DELETE':
        # Delete the order
        order.delete()

        # Return a success message
        return Response({'message': 'Order deleted successfully'}, status=status.HTTP_204_NO_CONTENT)
```

APP.TSX

```
const [loggedUser, setLoggedUser] = useState({id: 0, username: "", password: ""});

const deleteOrder = () => {
}
```

	DIFICULTAD	NOTAS
LLAMA	fácil.	no ha necesitado intervención.
COPILOT	fácil.	no ha necesitado intervención.